

01P-Linux 命令基础习惯-Linux 系统编程

date 显示系统当前时间

```
shiyancelou:~/ $ date
2020年 05月 19日 星期二 20:49:15 CST
shiyancelou:~/ $
```

cat /etc/shells 查看当前可使用的 shell

```
shiyancelou:Code/ $ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/dash
/bin/bash
/bin/rbash
/bin/zsh
/usr/bin/zsh
/usr/bin/tmux
shiyancelou:Code/ $
```

echo \$SHELL 查看当前使用的 shell

```
shiyancelou:Code/ $ echo $SHELL
/usr/bin/zsh
shiyancelou:Code/ $
```

主键盘快捷键:

上	Ctrl-p	
下	Ctrl-n	
左	Ctrl-b	
右	Ctrl-f	
Del	Ctrl-d delete	光标后面的
Home	Ctrl-a	first letter
End	Ctrl-e	end
Backspace	Backspace	delete 光标前面的单个字符
清除整行	Ctrl-u	
删除光标到行末	Ctrl-k	
显示上滚	Shift-PgUp	
显示下滚	Shift-PgDn	
增大终端字体	Ctrl-Shift-+	
减小终端字体	Ctrl--	
新打开一个终端	Ctrl-Alt-T	
清屏	Ctrl-l	直接用 clear 也行

02P-类 Unix 系统目录

pwd 查看当前所在目录

```
shiyanolou:Code/ $ pwd
/home/shiyanolou/Code
shiyanolou:Code/ $
```

Linux 系统目录:

bin: 存放二进制可执行文件

boot: 存放开机启动程序

dev: 存放设备文件: 字符设备、块设备

home: 存放普通用户

etc: 用户信息和系统配置文件 passwd、group

lib: 库文件: libc.so.6

root: 管理员宿主目录 (家目录)

usr: 用户资源管理目录 unix software resource

查看鼠标日志

[illegible]

03P- 目录和文件操作 1

cd - 返回上一个目录

```
/dev
zhcode@ubuntu:/dev$ cd ~
zhcode@ubuntu:~$ cd -
/dev
zhcode@ubuntu:/dev$
```

Linux 系统文件类型： 7/8 种

普通文件： -

目录文件： d

字符设备文件： c

块设备文件： b

软连接： l

管道文件： p

套接字： s

未知文件。

ls 列出当前文件夹下目录项

```
shianlou:~/ $ ls
Code Desktop golang lib node_modules package.json src-gen webpack.config.js
```

ll 竖排显示目录项和详细信息，ls -l 的缩写

```
shianlou:~/ $ ll
总用量 496K
drwxrwxr-x  2 shianlou shianlou 4.0K 9月 29 2019 Code
drwxr-xr-x  2 shianlou shianlou 4.0K 5月 19 14:53 Desktop
drwxr-xr-x  5 shianlou shianlou 4.0K 9月 23 2019 golang
drwxr-xr-x  3 shianlou shianlou 4.0K 9月 23 2019 lib
drwxr-xr-x 983 shianlou shianlou 36K 9月 23 2019 node_modules
-rw-r--r--  1 shianlou shianlou 1.6K 9月 19 2019 package.json
drwxr-xr-x  4 shianlou shianlou 4.0K 9月 23 2019 src-gen
-rw-r--r--  1 shianlou shianlou 5.2K 9月 23 2019 webpack.config.js
-rw-r--r--  1 shianlou shianlou 421K 9月 23 2019 yarn.lock
```

ls -l 显示目录项详细信息

```
shiyancelou:~/ $ ls -l
总用量 496
drwxrwxr-x  2 shiyancelou shiyancelou  4096 9月  29  2019 Code
drwxr-xr-x  2 shiyancelou shiyancelou  4096 5月  19 17:47 Desktop
drwxr-xr-x  5 shiyancelou shiyancelou  4096 9月  23  2019 go1ang
drwxr-xr-x  3 shiyancelou shiyancelou  4096 9月  23  2019 lib
drwxr-xr-x 983 shiyancelou shiyancelou 36864 9月  23  2019 node_modules
-rw-r--r--  1 shiyancelou shiyancelou  1543 9月  19  2019 package.json
drwxr-xr-x  4 shiyancelou shiyancelou  4096 9月  23  2019 src-gen
-rw-r--r--  1 shiyancelou shiyancelou  5279 9月  23  2019 webpack.config.js
-rw-r--r--  1 shiyancelou shiyancelou 430185 9月  23  2019 yarn.lock
```

ls -l dirname 显示 dirname 中目录详细信息

```
shiyancelou:~/ $ ls -l
总用量 496
drwxrwxr-x  2 shiyancelou shiyancelou  4096 9月  29  2019 Code
drwxr-xr-x  2 shiyancelou shiyancelou  4096 5月  19 17:47 Desktop
drwxr-xr-x  5 shiyancelou shiyancelou  4096 9月  23  2019 go1ang
drwxr-xr-x  3 shiyancelou shiyancelou  4096 9月  23  2019 lib
drwxr-xr-x 983 shiyancelou shiyancelou 36864 9月  23  2019 node_modules
-rw-r--r--  1 shiyancelou shiyancelou  1543 9月  19  2019 package.json
drwxr-xr-x  4 shiyancelou shiyancelou  4096 9月  23  2019 src-gen
-rw-r--r--  1 shiyancelou shiyancelou  5279 9月  23  2019 webpack.config.js
-rw-r--r--  1 shiyancelou shiyancelou 430185 9月  23  2019 yarn.lock
```

ls -dl dirname 显示 dirname 本身的详细信息

```
shiyancelou:~/ $ ls -dl Code
drwxrwxr-x 2 shiyancelou shiyancelou 4096 9月  29  2019 Code
shiyancelou:~/ $
```

ls -R 递归查看目录

```
shiyancelou:Code/ $ ls -R
.:
file3 mycode

./mycode:
file1 file2
shiyancelou:Code/ $
```


ls -Rl 递归展示详细信息

```
shiyanolou:Code/ $ ls -Rl
.:
总用量 4
-rw-rw-r-- 1 shiyanolou shiyanolou 0 5月 19 17:53 file3
drwxrwxr-x 2 shiyanolou shiyanolou 4096 5月 19 17:53 mycode

./mycode:
总用量 0
-rw-rw-r-- 1 shiyanolou shiyanolou 0 5月 19 17:53 file1
-rw-rw-r-- 1 shiyanolou shiyanolou 0 5月 19 17:53 file2
```

文件权限说明

```
drwxr-xr-x 983 shiyanolou shiyanolou 36K 9月 23 2019 node_modules
-rw-r--r-- 1 shiyanolou shiyanolou 1.6K 9月 19 2019 package.json
drwxr-xr-x 4 shiyanolou shiyanolou 4.0K 9月 23 2019 src-gen
-rw-r--r-- 1 shiyanolou shiyanolou 5.2K 9月 23 2019 webpack.config.js
-rw-r--r-- 1 shiyanolou shiyanolou 421K 9月 23 2019 yarn.lock
```

目录项详细信息

文件权限 硬链接计数 所有者 所属组 大小 时间 文件名/文件夹名

权限具体展开

-rw-r--r--

1234567890

1 代表文件类型

234 代表所有者读写执行权限

567 代表同组用户读写执行权限

890 代表其他人读写执行权限

which instruct 查看 instruct 命令所在目录位置

```
shiyanolou:Code/ $ which date
/bin/date
shiyanolou:Code/ $
```

隐藏终端中的路径

vi ~/.bashrc 打开使用的 shell 环境配置文件

末尾添加 PS1=\$ 保存退出，重启终端即可

```
116 fi
117 fi
118 PS1=$
```

效果如下:

```
$cd Code
$ls
file1.c
$pwd
/home/shiyanlou/Code
$
```

mkdir dirname 新建目录

```
shiyanlou:Code/ $ mkdir mycode
shiyanlou:Code/ $ ls
file1.c  mycode
shiyanlou:Code/ $
```

rmdir dirname 删除空目录, 非空目录删不掉

```
shiyanlou:Code/ $ rmdir mycode
rmdir: 删除 'mycode' 失败: 目录非空
shiyanlou:Code/ $
```

touch filename 创建名为 name 的空文件

```
shiyanlou:Code/ $ touch file3
shiyanlou:Code/ $ ls
file1.c  file2  file3  mycode
shiyanlou:Code/ $
```

rm filename 删除文件

```
shiyanlou:Code/ $ ls
file1.c  file2  file3  mycode
shiyanlou:Code/ $ rm file3
shiyanlou:Code/ $ ls
file1.c  file2  mycode
shiyanlou:Code/ $
```

rm -r dirname 递归删除目录

rm -rf dirname 强制删除

```
shiyanlou:Code/ $ ls
file1.c  file2  mycode
shiyanlou:Code/ $ rm mycode -r
shiyanlou:Code/ $ ls
file1.c  file2
shiyanlou:Code/ $
```

`mv file1 file2 location`

将文件 1 和文件 2 移动到目标位置

```
zhcode@ubuntu:~/Code$ ls
hello  hello.c  Mydir
zhcode@ubuntu:~/Code$ mv hello hello.c Mydir
zhcode@ubuntu:~/Code$ ls
Mydir
zhcode@ubuntu:~/Code$
```

`cp filename dirname` 复制文件到目录

`cp filename1 filename2` 复制文件 1 并重命名为文件 2

`cp -a dirname1 dirname2` 复制目录 1 及其下所有文件到目录 2

`cp -r dirname1 dirname2` 递归复制目录 1 到目录 2

```
shianlou:Code/ $ ls
file3  mycode
shianlou:Code/ $ cp file3 file5
shianlou:Code/ $ ls
file3  file5  mycode
shianlou:Code/ $
```

这里 `-a` 和 `-r` 的差别在于，`-a` 是完全复制，文件权限，改动时间什么的也完全相同。

04P- 目录和文件操作 2

cat filename 查看文件内容

tac filename 逆转查看文件内容

```
shiyanolou:Code/ $ cat file3
my wife is megumin
lalalalalala
shiyanolou:Code/ $
```

```
shiyanolou:Code/ $ tac file3
lalalalalala
my wife is megumin
shiyanolou:Code/ $
```

cat 读取终端，就是回显

```
shiyanolou:Code/ $ cat
my wife is holo
my wife is holo

```

more filename 和 cat 差不多，但是对于大文件查看很强势
空格翻页，回车一行
用 q 或者 Ctrl-c 退出

less filename 也和 cat 差不多
空格翻页，回车一行
用 q 或者 Ctrl-c 退出

head -n filename 查看文件前 n 行
不加 -n 参数默认查看 10 行

```
shiyanolou:Code/ $ head -5 file3
my wife is megumin
lalalalalala
3
4
5
shiyanolou:Code/ $
```

tail -n filename 查看文件后 n 行
默认查看 10 行，顺序显示的，不是倒序显示，1-10 查看后 4 行是 78910

```
shiyanolou:Code/ $ tail -5 file3
9
10
11
12
```

tree 命令，查看当前目录结构树
需要安装 tree

```
shiyanolou:Code/ $ sudo apt-get install tree
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
tree 已经是最新版 (1.7.0-3)。
下列软件包是自动安装的并且现在不需要了：
  libpython3-dev libpython3.5-dev python3.5-dev
使用'sudo apt autoremove'来卸载它(它们)。
升级了 0 个软件包，新安装了 0 个软件包，要卸载 0 个软件包，有 0 个软件包未被升级。
```

```
shiyanolou:Code/ $ tree
.
├── file3
├── file5
└── mycode
    ├── file1
    └── file2

1 directory, 4 files
```

05P-软链接和硬链接

`ln -s file file.s` 创建一个软链接

```
zhcode@ubuntu:~/Code$ ln -s file1 file.s
zhcode@ubuntu:~/Code$ ls
file1  file2  file.s  Mydir
zhcode@ubuntu:~/Code$
```

软链接就像 windows 下的快捷方式

```
zhcode@ubuntu:~/Code$ cat file1
my wife is megumin!!!
zhcode@ubuntu:~/Code$ cat file.s
my wife is megumin!!!
zhcode@ubuntu:~/Code$
```

```
lrwxrwxrwx  1 zhcode zhcode   5 May 20 12:01 file.s -> file1
```

这里软链接的大小是文件路径

Linux 下的软链接行为和 windows 下的快捷方式差不多，但是如果是用相对路径创建的软链接，在软链接移动之后就会失效，无法访问。这一点和 windows 快捷方式不同，windows 快捷方式随便放哪里都行。

失效的软链接

```
zhcode@ubuntu:~/Code$ mv file.s Mydir
zhcode@ubuntu:~/Code$ ls
file1  file2  file.s2  Mydir
zhcode@ubuntu:~/Code$ cd Mydir
zhcode@ubuntu:~/Code/Mydir$ ls
file.s  hello  hello.c
zhcode@ubuntu:~/Code/Mydir$
```

所以，创建软链接最好使用绝对路径

```
zhcode@ubuntu:~/Code$ ls
file1  file2  file.s  Mydir
zhcode@ubuntu:~/Code$ ln -s /home/zhcode/Code/file1 file.s2
zhcode@ubuntu:~/Code$ ls
file1  file2  file.s  file.s2  Mydir
zhcode@ubuntu:~/Code$
```

移动后，绝对路径创建的软链接不会失效

```

zhcode@ubuntu:~/Code$ mv file.s2 Mydir
zhcode@ubuntu:~/Code$ cd Mydir
zhcode@ubuntu:~/Code/Mydir$ ls
file.s  file.s2  hello  hello.c
zhcode@ubuntu:~/Code/Mydir$ ll
total 24
drwxrwxr-x 2 zhcode zhcode 4096 May 20 12:19 ./
drwxrwxr-x 3 zhcode zhcode 4096 May 20 12:19 ../
lrwxrwxrwx 1 zhcode zhcode 5 May 20 12:01 file.s -> file1
lrwxrwxrwx 1 zhcode zhcode 23 May 20 12:16 file.s2 -> /home/zhcode/Code/file1
-rwxrwxr-x 1 zhcode zhcode 8600 May 20 00:02 hello*
-rw-rw-r-- 1 zhcode zhcode 81 May 20 00:02 hello.c
zhcode@ubuntu:~/Code/Mydir$

```

上图要注意，软链接的权限指的是这个软链接本身的权限，不是软链接指向文件的权限

ln file file.h 创建一个硬链接

```

zhcode@ubuntu:~/Code$ ln file1 file.h
zhcode@ubuntu:~/Code$ ls
file1  file2  file.h  Mydir
zhcode@ubuntu:~/Code$

```

创建硬链接后，文件的硬链接计数+1

```

zhcode@ubuntu:~/Code$ ll
total 20
drwxrwxr-x 3 zhcode zhcode 4096 May 20 12:25 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:03 ../
-rw-rw-r-- 2 zhcode zhcode 22 May 20 12:03 file1
-rw-rw-r-- 1 zhcode zhcode 0 May 20 11:56 file2
-rw-rw-r-- 2 zhcode zhcode 22 May 20 12:03 file.h
drwxrwxr-x 2 zhcode zhcode 4096 May 20 12:19 Mydir/
zhcode@ubuntu:~/Code$

```

再创建一个硬链接

```

zhcode@ubuntu:~/Code$ ln file1 file.hard
zhcode@ubuntu:~/Code$ ll
total 24
drwxrwxr-x 3 zhcode zhcode 4096 May 20 12:30 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:03 ../
-rw-rw-r-- 3 zhcode zhcode 22 May 20 12:03 file1
-rw-rw-r-- 1 zhcode zhcode 0 May 20 11:56 file2
-rw-rw-r-- 3 zhcode zhcode 22 May 20 12:03 file.h
-rw-rw-r-- 3 zhcode zhcode 22 May 20 12:03 file.hard
drwxrwxr-x 2 zhcode zhcode 4096 May 20 12:19 Mydir/
zhcode@ubuntu:~/Code$

```

这里对于 file1，有 2 个硬链接 file.h 和 file.hard，无论更改哪个硬链接或者文件本身，这三个文件的变化同步

```
zhcode@ubuntu:~/Code$ vi file.h
zhcode@ubuntu:~/Code$ cat file1
my wife is megumin!!!
my wife is rin!!!
zhcode@ubuntu:~/Code$ cat file.h
my wife is megumin!!!
my wife is rin!!!
zhcode@ubuntu:~/Code$ cat file.hard
my wife is megumin!!!
my wife is rin!!!
zhcode@ubuntu:~/Code$
```

产生这种同步变化的原因:

```
zhcode@ubuntu:~/Code$ stat file1
  File: 'file1'
  Size: 40          Blocks: 8          IO Block: 4096   regular file
Device: 801h/2049d Inode: 926368        Links: 3
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  zhcode)   Gid: ( 1000/  zhcode)
Access: 2020-05-20 12:33:56.120177685 +0800
Modify: 2020-05-20 12:33:48.433077616 +0800
Change: 2020-05-20 12:33:48.433077616 +0800
 Birth: -
zhcode@ubuntu:~/Code$ stat file.h
  File: 'file.h'
  Size: 40          Blocks: 8          IO Block: 4096   regular file
Device: 801h/2049d Inode: 926368        Links: 3
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  zhcode)   Gid: ( 1000/  zhcode)
Access: 2020-05-20 12:33:56.120177685 +0800
Modify: 2020-05-20 12:33:48.433077616 +0800
Change: 2020-05-20 12:33:48.433077616 +0800
 Birth: -
zhcode@ubuntu:~/Code$
```

可以看到文件和硬链接的 Inode 是相同的, 每个文件都有唯一的 Inode, 直观理解起来就像洗佳佳里面的引用, 对于同一个文件, 无论有多少引用, 在访问时, 都是这个文件, 所以修改就是同步的。

当删除一个硬链接时, 文件的硬链接计数-1, 当这个计数减为 0 时, 才会删除这个文件


```

zhcode@ubuntu:~/Code$ ll
total 24
drwxrwxr-x  3 zhcode zhcode 4096 May 20 12:33 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:33 ../
-rw-rw-r--  3 zhcode zhcode  40 May 20 12:33 file1
-rw-rw-r--  1 zhcode zhcode   0 May 20 11:56 file2
-rw-rw-r--  3 zhcode zhcode  40 May 20 12:33 file.h
-rw-rw-r--  3 zhcode zhcode  40 May 20 12:33 file.hard
drwxrwxr-x  2 zhcode zhcode 4096 May 20 12:19 Mydir/
zhcode@ubuntu:~/Code$ rm file.h
zhcode@ubuntu:~/Code$ ll
total 20
drwxrwxr-x  3 zhcode zhcode 4096 May 20 12:44 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:33 ../
-rw-rw-r--  2 zhcode zhcode  40 May 20 12:33 file1
-rw-rw-r--  1 zhcode zhcode   0 May 20 11:56 file2
-rw-rw-r--  2 zhcode zhcode  40 May 20 12:33 file.hard
drwxrwxr-x  2 zhcode zhcode 4096 May 20 12:19 Mydir/
zhcode@ubuntu:~/Code$ █

```

即使删除硬链接指向的文件，也只会让硬链接计数-1

```

zhcode@ubuntu:~/Code$ rm file1
zhcode@ubuntu:~/Code$ ll
total 16
drwxrwxr-x  3 zhcode zhcode 4096 May 20 12:46 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:33 ../
-rw-rw-r--  1 zhcode zhcode   0 May 20 11:56 file2
-rw-rw-r--  1 zhcode zhcode  40 May 20 12:33 file.hard
drwxrwxr-x  2 zhcode zhcode 4096 May 20 12:19 Mydir/
zhcode@ubuntu:~/Code$ cat file.hard
my wife is megumin!!!
my wife is rin!!!
zhcode@ubuntu:~/Code$ █

```

06P-创建修改用户和用户组

whoami 查看当前用户

```
zhcode@ubuntu:~/Code$ whoami
zhcode
zhcode@ubuntu:~/Code$
```

chmod 修改权限操作

第一种，文字设定法

chmod [who] [+|-|=] [mode] filename

操作对象 who 可以是下述字母中的任一个或者它们的组合

- u 表示” 用户(user)” ， 即文件或目录的所有者
- g 表示” 同组(group)用户” ， 即与文件所有者有相同组 ID 的所有用户
- o 表示” 其他(others)用户”
- a 表示” 所有(all)用户” ， 它是系统默认值

操作符号可以是：

- + 添加某个权限
- 取消某个权限
- = 赋予给定权限并取消其他所有权限（如果有的话）

如下所示，给 file2 文件添加执行权限

```
zhcode@ubuntu:~/Code$ ll file2
-rw-rw-r-- 1 zhcode zhcode 0 May 20 11:56 file2
zhcode@ubuntu:~/Code$ chmod u+x file2
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 zhcode zhcode 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$
```

第二种，数字设定法

chmod 操作码 filename 直接用操作码修改文件权限

对于 file2 的权限

-rw-rw-r--
421421421

三个组的权限都用二进制编号，比如要设置当前用户对文件的读写和执行权限，则当前用户的操作权限为 4（读）+ 2（写）+ 1（执行） = 7

用户组和其他用户的权限设置也是一样的

对于 file2 的权限-rw-rw-r--

我们设置如下：

- 所有者 rwx = 7
- 所有者所在组 rw = 6
- 其他用户 r = 4

操作码就是 764

```
zhcode@ubuntu:~/Code$ ll file2
-rw-rw-r-- 1 zhcode zhcode 0 May 20 11:56 file2
zhcode@ubuntu:~/Code$ chmod 764 file2
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 zhcode zhcode 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$
```

sudo adduser newusername 添加新用户

```
zhcode@ubuntu:~/Code$ sudo adduser holo
Adding user `holo' ...
Adding new group `holo' (1002) ...
Adding new user `holo' (1002) with group `holo' ...
Creating home directory `/home/holo' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for holo
Enter the new value, or press ENTER for the default
    Full Name []: holo
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
zhcode@ubuntu:~/Code$
```

chown username filename 修改文件所有者

```
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 zhcode zhcode 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$ chown megumin file2
chown: changing ownership of 'file2': Operation not permitted
zhcode@ubuntu:~/Code$
zhcode@ubuntu:~/Code$ sudo chown megumin file2
[sudo] password for zhcode:
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 megumin zhcode 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$
```

su username 切换当前用户为 username

```
zhcode@ubuntu:~/Code$ su megumin
Password:
megumin@ubuntu:~/Code$
```

sudo addgroup groupname 添加新的用户组

```
zhcode@ubuntu:~/Code$ sudo addgroup mywife
Adding group `mywife' (GID 1003) ...
Done.
zhcode@ubuntu:~/Code$
```

`sudo chgrp groupname filename` 修改文件所属用户组

```
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 megumin zhcode 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$ sudo chgrp mywife file2
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 megumin mywife 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$
```

`sudo chown username:groupname filename` 同时修改文件所属用户和用户组

```
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 megumin mywife 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$ sudo chown holo:nogroup file2
zhcode@ubuntu:~/Code$ ll file2
-rwxrw-r-- 1 holo nogroup 0 May 20 11:56 file2*
zhcode@ubuntu:~/Code$
```

`sudo deluser username` 删除用户

```
zhcode@ubuntu:~/Code$ sudo deluser holo
Removing user `holo' ...
Warning: group `holo' has no more members.
Done.
zhcode@ubuntu:~/Code$
```

`sudo delgroup groupname` 删除用户组

```
zhcode@ubuntu:~/Code$ sudo delgroup mywife
Removing group `mywife' ...
Done.
zhcode@ubuntu:~/Code$
```

07P-find 命令 1

find 命令：找文件

-type 按文件类型搜索 d/p/s/c/b/l/ f:文件

```
zhcode@ubuntu:~/Code$ ll
total 16
drwxrwxr-x  3 zhcode zhcode 4096 May 20 15:18 ./
drwxr-xr-x 19 zhcode zhcode 4096 May 20 12:33 ../
-rwxrw-r--  1 1002 nogroup  0 May 20 11:56 file2*
-rw-rw-r--  1 zhcode zhcode  40 May 20 12:33 file.hard
drwxrwxr-x  2 zhcode zhcode 4096 May 20 15:18 Mydir/
-rw-rw-r--  1 zhcode zhcode  0 May 20 15:18 yuki.jpg
zhcode@ubuntu:~/Code$ find ./ -type d
./
./Mydir
zhcode@ubuntu:~/Code$
```

-name 按文件名搜索

find ./ -name "*file*.jpg"

```
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir yuki.jpg
zhcode@ubuntu:~/Code$ find ./ -name '*.jpg'
./Mydir/rin.jpg
./yuki.jpg
zhcode@ubuntu:~/Code$
```

-maxdepth 指定搜索深度。应作为第一个参数出现。

find ./ -maxdepth 1 -name "*file*.jpg"

```
zhcode@ubuntu:~/Code$ find ./ -maxdepth 1 -name '*.jpg'
./yuki.jpg
zhcode@ubuntu:~/Code$
```

-size 按文件大小搜索。单位：k、M、G

find /home/itcast -size +20M -size -50M

这里要注意，两个 size 一个都不能少，还有就是文件大小单位对大小写敏感

```
zhcode@ubuntu:/bin$ find ./ -size +1M -size -10M
./busybox
zhcode@ubuntu:/bin$ ls -l -h ./busybox
-rwxr-xr-x 1 root root 1.9M Aug 19 2015 ./busybox
zhcode@ubuntu:/bin$
```

-atime、mtime、ctime 天 amin、mmin、cmin 分钟。

按照时间搜索

a 表示最近访问时间

m 表示最近更改时间，指更改文件属性一类的

c 表示最近改动时间，指更改文件内容

08P-午后复习

09P-find 命令 2

-exec: 将 find 搜索的结果集执行某一指定命令。

```
find /usr/ -name '*tmp*' -exec ls -ld {} \;
```

```
zhcode@ubuntu:~/Code$ find ./ -name '*.jpg' -exec ls -l {} \;  
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./Mydir/rin.jpg  
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./yuki.jpg  
zhcode@ubuntu:~/Code$
```

-ok: 以交互式的方式 将 find 搜索的结果集执行某一指定命令

```
zhcode@ubuntu:~/Code$ find ./ -name '*.jpg' -ok ls -l {} \;  
< ls ... ./Mydir/rin.jpg > ? y  
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./Mydir/rin.jpg  
< ls ... ./yuki.jpg > ? y  
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./yuki.jpg  
zhcode@ubuntu:~/Code$
```

10P-grep 和 xargs

grep 命令：找文件内容

grep -r 'copy' ./ -n

-n 参数：:显示行号

```
zhcode@ubuntu:~/Code$ grep -r 'holo' ./ -n
./Mydir/holo.c:1:my wife is holo
./file.hard:4:my wife is holo!!!
zhcode@ubuntu:~/Code$
```

ps 监控后台进程工作情况，默认只显示当前可以和用户交互的进程

```
zhcode@ubuntu:~/Code$ ps
  PID TTY          TIME CMD
 2959 pts/1        00:00:00 bash
 6570 pts/1        00:00:00 bash
 7835 pts/1        00:00:00 ps
zhcode@ubuntu:~/Code$
```

ps aux | grep 'cupsd' -- 检索进程结果集。

下图显示和内核相关进程

```
zhcode@ubuntu:~/Code$ ps aux | grep kernel
zhcode      7845  0.0  0.0 15968  948 pts/1    S+   18:44   0:00 grep --color=auto kernel
zhcode@ubuntu:~/Code$
```

使用 grep 搜索进程，有一条结果是搜索进程本身

find ... | xargs ls -l 对 find 操作的结果集进行操作

等价于

find ... -exec ls -l {} \;

两者差别在于当结果集合很大的时候，xargs 会对结果进行分段处理，所以性能好些

但 xargs 也有缺陷，xargs 默认用空格来分割结果集，当文件名有空格的时候，会因为文件名被切割失效

```
zhcode@ubuntu:~/Code$ find ./ -name '*.jpg' | xargs ls -l
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./Mydir/rin.jpg
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./yuki.jpg
zhcode@ubuntu:~/Code$
```

-xargs: 将 find 搜索的结果集执行某一指定命令。 当结果集数量过大时，可以分片映射。

find /usr/ -name '*tmp*' | xargs ls -ld

创建名字带空格的文件方法

第一个方法，文件名加引号

```
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir yuki.jpg
zhcode@ubuntu:~/Code$ touch 'ab cd'
zhcode@ubuntu:~/Code$ ls
ab cd file2 file.hard Mydir yuki.jpg
zhcode@ubuntu:~/Code$
```

第二个方法，转义

```
zhcode@ubuntu:~/Code$ ls
ab cd file2 file.hard Mydir yuki.jpg
zhcode@ubuntu:~/Code$ touch ef\ gh
zhcode@ubuntu:~/Code$ ls
ab cd ef gh file2 file.hard Mydir yuki.jpg
zhcode@ubuntu:~/Code$
```

xargs 缺陷演示

```
zhcode@ubuntu:~/Code$ find ./ -maxdepth 1 -type f -exec ls -l {} \;
-rw-rw-r-- 1 zhcode zhcode 0 May 20 19:28 ./ef gh
-rw-rw-r-- 1 zhcode zhcode 0 May 20 19:25 ./ab cd
-rw-rw-r-- 1 zhcode zhcode 78 May 20 18:37 ./file.hard
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./yuki.jpg
-rwxrw-r-- 1 1002 nogroup 0 May 20 11:56 ./file2
zhcode@ubuntu:~/Code$ find ./ -maxdepth 1 -type f | xargs ls -l
ls: cannot access './ef': No such file or directory
ls: cannot access 'gh': No such file or directory
ls: cannot access './ab': No such file or directory
ls: cannot access 'cd': No such file or directory
-rwxrw-r-- 1 1002 nogroup 0 May 20 11:56 ./file2
-rw-rw-r-- 1 zhcode zhcode 78 May 20 18:37 ./file.hard
-rw-rw-r-- 1 zhcode zhcode 0 May 20 15:18 ./yuki.jpg
zhcode@ubuntu:~/Code$
```

解决 xargs 的缺陷，指定切分符号位 null

-print0:

```
find /usr/ -name '*tmp*' -print0 | xargs -print0 ls -l
```

第一个 print0 指定结果集分隔为 null，第二个 print0 指定 xargs 分隔为 null

```
zhcode@ubuntu:~/Code$ find ./ -maxdepth 1 -type f -print0 | xargs -print0 ls -l
xargs: WARNING: a NUL character occurred in the input. It cannot be passed through in the arguments. Did you mean to use the --null option?
ls -l ?...y
total 8
-rw-rw-r-- 1 zhcode zhcode      0 May 20 19:25 ab cd
-rw-rw-r-- 1 zhcode zhcode      0 May 20 19:28 ef gh
-rwxrw-r-- 1    1002 nogroup      0 May 20 11:56 file2
-rw-rw-r-- 1 zhcode zhcode     78 May 20 18:37 file.hard
drwxrwxr-x 2 zhcode zhcode  4096 May 20 18:36 Mydir
-rw-rw-r-- 1 zhcode zhcode      0 May 20 15:18 yuki.jpg
zhcode@ubuntu:~/Code$
```

11P-xargs 加强和 awk 说明

12P-软件包安装

`sudo apt-get install softname`

安装软件

```
zhcode@ubuntu:~/Code$ sudo apt-get install tree
[sudo] password for zhcode:
Reading package lists... Done
Building dependency tree
Reading state information... Done
tree is already the newest version (1.7.0-3).
0 upgraded, 0 newly installed, 0 to remove and 813 not upgraded.
zhcode@ubuntu:~/Code$
```

`sudo apt-get update` 更新软件列表

```
zhcode@ubuntu:~/Code$ sudo apt-get update
Hit:1 http://mirrors.aliyun.com/ubuntu xenial InRelease
Get:2 http://mirrors.aliyun.com/ubuntu xenial-updates InRelease [109 kB]
Get:3 http://mirrors.aliyun.com/ubuntu xenial-backports InRelease [107 kB]
Get:4 http://mirrors.aliyun.com/ubuntu xenial-security InRelease [109 kB]
Get:5 http://mirrors.aliyun.com/ubuntu xenial-updates/main amd64 Packages [1,150 kB]
```

更换软件源:

系统设置->软件和更新->下载自...

换软件源过后要更新软件列表

`sudo apt-get remove softname` 卸载软件

使用安装包进行软件安装

安装deb软件包命令: `sudo dpkg -i xxx.deb`

删除软件包命令: `sudo dpkg -r xxx.deb`

连同配置文件一起删除命令: `sudo dpkg -r --purge xxx.deb`

查看软件包信息命令: `sudo dpkg -info xxx.deb`

查看文件拷贝详情命令: `sudo dpkg -L xxx.deb`

查看系统中已安装软件包信息命令: `sudo dpkg -l`

重新配置软件包命令: `sudo dpkg-reconfigure xxx`

13P-压缩命令 gzip 和 bzip2

两者都是配合 tar 打包命令使用

这两个压缩的缺陷都是只能对单个文件进行压缩，一来不能压目录，二来不能打包

第一种压缩方式：gzip

tar zcvf 要生成的压缩包名 压缩材料，这里压缩包名一般以.tar.gz 结尾

```
zhcode@ubuntu:~/Code$ ls
ab cd ef gh file2 file.hard Mydir spacefile yuki.jpg
zhcode@ubuntu:~/Code$ tar zcvf spacefile.tar.gz ab\ cd ef\ gh
ab cd
ef gh
zhcode@ubuntu:~/Code$ ls
ab cd ef gh file2 file.hard Mydir spacefile spacefile.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$
```

上述命令实际上执行了两步，

一个是 gzip 进行压缩

gzip filename

```
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir spacefile spacefile.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$ gzip file2
zhcode@ubuntu:~/Code$ ls
file2.gz file.hard Mydir spacefile spacefile.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$
```

解压

gunzip zipfile

```
zhcode@ubuntu:~/Code$ ls
file2.gz file.hard Mydir spacefile spacefile.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$ gunzip file2.gz
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir spacefile spacefile.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$
```

另一个是打包命令

tar file... tarname

所以 tar zcvf 是两条指令的结合版本

对 zcvf 进行解释：

z:zip, 压缩

c:create, 创建

v:vision, 显示压缩过程，可以去掉，直接用 zcf，但这样不显示压缩过程

f:file, 文件

file filename 查看文件来源

```
zhcode@ubuntu:~/Code$ file spacefile.tar.gz
spacefile.tar.gz: gzip compressed data, last modified: Wed May 20 14:18:57 2020, from Unix
zhcode@ubuntu:~/Code$
```

第二种压缩方式：使用 bzip2 方式压缩
tar jcvf 要生成的压缩包名 压缩材料

```
zhcode@ubuntu:~/Code$ tar jcvf test.tar.gz file2 file.hard
file2
file.hard
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir spacefile spacefile.tar.gz test.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$
```

解压：

将压缩命令中的 c --> x

tar zxvf 压缩材料 使用 gzip 解压

tar jxvf 压缩材料 使用 bzip2 解压

```
zhcode@ubuntu:~/Code$ ls
file2 file.hard Mydir spacefile spacefile.tar.gz test.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$ tar zxvf spacefile.tar.gz
ab cd
ef gh
zhcode@ubuntu:~/Code$ ls
ab cd ef gh file2 file.hard Mydir spacefile spacefile.tar.gz test.tar.gz yuki.jpg
zhcode@ubuntu:~/Code$
```

14P-rar 压缩和 zip 压缩

rar 压缩，需要安装 rar

```
zhcode@ubuntu:~/Code$ sudo apt-get install rar
[sudo] password for zhcode:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  unrar
```

rar a -r newdir dir

打包，把 dir 压缩成 newdir.rar，如果压缩材料里没有目录，-r 参数可以省去

```
zhcode@ubuntu:~/Code$ rar a -r dzip.rar Mydir ab\ cd ef\ gh

RAR 5.30 beta 2   Copyright (c) 1993-2015 Alexander Roshal   4 Aug 2015
Trial version                               Type RAR -? for help

Evaluation copy. Please register.

zhcode@ubuntu:~/Code$ ls
ab cd  dzip.rar  ef gh  file2  file.hard  Mydir  spacefile  spacefile.tar.gz
zhcode@ubuntu:~/Code$
```

unrar x newdir.rar 解压 rar 文件

```
zhcode@ubuntu:~/Code/rartest$ ls
dzip.rar
zhcode@ubuntu:~/Code/rartest$ unrar x dzip.rar

UNRAR 5.30 beta 2 freeware           Copyright (c) 1993-2015 Alexander Roshal

Extracting from dzip.rar

Creating      Mydir                                OK
Extracting   Mydir/hello                          OK
Extracting   Mydir/hello.c                        OK
Extracting   Mydir/holo.c                         OK
Extracting   Mydir/rin.jpg                        OK
Extracting   Mydir/kikyo.c                        OK
Extracting   ab cd                                OK
Extracting   ef gh                                OK
All OK
zhcode@ubuntu:~/Code/rartest$
```

sudo aptitude show softname 查看软件安装信息

```
zhcode@ubuntu:~/Code/rartest$ sudo aptitude show tree
Package: tree
State: installed
Automatically installed: no
Version: 1.7.0-3
Priority: optional
Section: universe/utils
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Uncompressed Size: 138 k
Depends: libc6 (>= 2.14)
Conflicts: tree:i386
Description: displays an indented directory tree, in color
 Tree is a recursive directory listing command that produces a depth indented
  colored ala dircolors if the LS_COLORS environment variable is set and out
Homepage: http://mama.indstate.edu/users/ice/tree/

zhcode@ubuntu:~/Code/rartest$
```

zip 压缩:

zip -r dir.zip dir

```
zhcode@ubuntu:~/Code/rartest$ ls
ab cd ef gh Mydir
zhcode@ubuntu:~/Code/rartest$ zip -r testzip.zip ab\ cd ef\ gh Mydir
  adding: ab cd (stored 0%)
  adding: ef gh (stored 0%)
  adding: Mydir/ (stored 0%)
  adding: Mydir/hello (deflated 72%)
  adding: Mydir/hello.c (deflated 5%)
  adding: Mydir/holo.c (deflated 19%)
  adding: Mydir/rin.jpg (stored 0%)
  adding: Mydir/kikyo.c (stored 0%)
zhcode@ubuntu:~/Code/rartest$ ls
ab cd ef gh Mydir testzip.zip
zhcode@ubuntu:~/Code/rartest$
```

zip 解压

unzip dir.zip


```
zhcode@ubuntu:~/Code/rartest/test$ ls
testzip.zip
zhcode@ubuntu:~/Code/rartest/test$ unzip testzip.zip
Archive:  testzip.zip
  extracting: ab cd
  extracting: ef gh
    creating: Mydir/
  inflating: Mydir/hello
  inflating: Mydir/hello.c
  inflating: Mydir/holo.c
  extracting: Mydir/rin.jpg
  extracting: Mydir/kikyo.c
zhcode@ubuntu:~/Code/rartest/test$ ls
ab cd  ef gh  Mydir  testzip.zip
zhcode@ubuntu:~/Code/rartest/test$
```

zip 文件在 windows 和 linux 下通用

15P-其他命令

who

查看当前在线上的用户情况

```
zhcode@ubuntu:~/Code/rartest/test$ who
zhcode    tty7          2020-05-20 09:08 (:0)
zhcode@ubuntu:~/Code/rartest/test$
```

whoami

查看当前用户，不带有进程

```
zhcode@ubuntu:~/Code/rartest/test$ whoami
zhcode
zhcode@ubuntu:~/Code/rartest/test$
```

ps aux | grep 条件 结果至少有一个，就是当前查询进程

```
zhcode@ubuntu:~/Code/rartest/test$ ps aux | grep Mycode
zhcode    11306  0.0  0.0 15968  924 pts/1    S+   23:17   0:00 grep --color=auto Mycode
zhcode@ubuntu:~/Code/rartest/test$
```

jobs

查看操作系统当前运行了哪些用户作业

```
zhcode@ubuntu:~/Code/rartest/test$ cat
my wife is megumin
my wife is megumin
^C
zhcode@ubuntu:~/Code/rartest/test$ cat &
[1] 11317
zhcode@ubuntu:~/Code/rartest/test$ jobs
[1]+  Stopped                  cat
zhcode@ubuntu:~/Code/rartest/test$
```

kill 杀死进程

env 环境变量

```
zhcode@ubuntu:~/Code/rartest/test$ env | grep SHELL
SHELL=/bin/bash
zhcode@ubuntu:~/Code/rartest/test$
```

top 文字版任务管理器

sudo passwd username 设置用户密码

```
zhcode@ubuntu:~/Code/rartest/test$ sudo passwd zhcode
[sudo] password for zhcode:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
zhcode@ubuntu:~/Code/rartest/test$
```

sudo su 切换 root 用户

```
zhcode@ubuntu:~/Code/rartest/test$ sudo su
root@ubuntu:/home/zhcode/Code/rartest/test#
```

ifconfig 查看网卡信息

```
zhcode@ubuntu:~/Code/rartest/test$ ifconfig
ens33      Link encap:Ethernet  HWaddr 00:0c:29:4f:cb:81
            inet addr:192.168.184.129  Bcast:192.168.184.255  Mask:255.255.255.0
            inet6 addr: fe80::525d:80fc:bb19:7e88/64 Scope:Link
            UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
            RX packets:418775 errors:0 dropped:0 overruns:0 frame:0
            TX packets:62129 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1000
            RX bytes:598835134 (598.8 MB)  TX bytes:3928415 (3.9 MB)

lo         Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            inet6 addr: ::1/128 Scope:Host
            UP LOOPBACK RUNNING  MTU:65536  Metric:1
            RX packets:433 errors:0 dropped:0 overruns:0 frame:0
            TX packets:433 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1
            RX bytes:48854 (48.8 KB)  TX bytes:48854 (48.8 KB)

zhcode@ubuntu:~/Code/rartest/test$
```

man 系统参考手册

man n name 在系统手册第 n 章查看 name

alias 给命令起别名

```
zhcode@ubuntu:~/Code/rartest/test$ alias
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]} && echo terminal || echo $0" -n1|sed -e '\''s/^\\s*[0-9]\\+\\s*//;s/[;&|]\\s*alert$/'\''\''"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -aLF'
alias ls='ls --color=auto'
zhcode@ubuntu:~/Code/rartest/test$
```

图中，ll 实际指令里的 F 意味着带文件标识符，目录末尾有/，可执行文件末尾带*

alias 别名=' 指令'

这里以管道查询别名为例

```
zhcode@ubuntu:~/Code/rartest/test$ alias pg='ps aux | grep'
zhcode@ubuntu:~/Code/rartest/test$ pg abc
zhcode    11431  0.0  0.0 15968  940 pts/1    S+   23:43   0:00 grep --color=auto abc
zhcode@ubuntu:~/Code/rartest/test$
```

16P-总结

Linux 系统：“所见皆文件”

Linux 系统目录：

bin：存放二进制可执行文件

boot：存放开机启动程序

dev：存放设备文件： 字符设备、块设备

home：存放普通用户

etc：用户信息和系统配置文件 passwd、group

lib：库文件： libc.so.6

root：管理员宿主目录（家目录）

usr：用户资源管理目录

Linux 系统文件类型： 7/8 种

普通文件： -

目录文件： d

字符设备文件： c

块设备文件： b

软连接： l

管道文件： p

套接字： s

未知文件。

软连接：快捷方式

为保证软连接可以任意搬移， 创建时务必对源文件使用 绝对路径。

硬链接：

```
ln file file.hard
```

操作系统给每一个文件赋予唯一的 inode，当有相同 inode 的文件存在时，彼此同步。

删除时，只将硬链接计数减一。减为 0 时，inode 被释放。

创建用户：

```
sudo adduser 新用户名 --- useradd
```

修改文件所属用户：

```
sudo chown 新用户名 待修改文件。
```

```
sudo chown wangwu a.c
```

删除用户：

```
sudo deluser 用户名
```

创建用户组：

```
sudo addgroup 新组名
```

修改文件所属用户组：

```
sudo chgrp 新用户组名 待修改文件。
```

```
sudo chgrp g88 a.c
```

删除组：

```
sudo delgroup 用户组名
```

使用 chown 一次修改所有者和所属组：

```
sudo chown 所有者：所属组 待操作文件。
```

find 命令：找文件

```
-type 按文件类型搜索 d/p/s/c/b/l/ f:文件
```

```
-name 按文件名搜索
```

```
find ./ -name "*file*.jpg"
```

```
-maxdepth 指定搜索深度。应作为第一个参数出现。
```

```
find ./ -maxdepth 1 -name "*file*.jpg"
```

-size 按文件大小搜索。 单位：k、M、G

```
find /home/itcast -size +20M -size -50M
```

-atime、mtime、ctime 天 amin、mmin、cmin 分钟。

-exec: 将 find 搜索的结果集执行某一指定命令。

```
find /usr/ -name '*tmp*' -exec ls -ld {} \;
```

-ok: 以交互式的方式 将 find 搜索的结果集执行某一指定命令

-xargs: 将 find 搜索的结果集执行某一指定命令。 当结果集数量过大时，可以分片映射。

```
find /usr/ -name '*tmp*' | xargs ls -ld
```

-print0:

```
find /usr/ -name '*tmp*' -print0 | xargs -0 ls -ld
```

grep 命令：找文件内容

```
grep -r 'copy' ./ -n
```

-n 参数：:显示行号

```
ps aux | grep 'cupsd' -- 检索进程结果集。
```

软件安装：

1. 联网
2. 更新软件资源列表到本地。 `sudo apt-get update`
3. 安装 `sudo apt-get install 软件名`
4. 卸载 `sudo apt-get remove 软件名`
5. 使用软件包 (.deb) 安装： `sudo dpkg -i 安装包名。`

tar 压缩：

1. `tar -zcvf` 要生成的压缩包名 压缩材料。

`tar zcvf test.tar.gz file1 dir2` 使用 gzip 方式压缩。

`tar jcvf test.tar.gz file1 dir2` 使用 bzip2 方式压缩。

tar 解压:

将 压缩命令中的 `c` --> `x`

`tar zxvf test.tar.gz` 使用 gzip 方式解压缩。

`tar jxvf test.tar.gz` 使用 bzip2 方式解压缩。

rar 压缩:

`rar a -r 压缩包名 (带.rar 后缀) 压缩材料。`

`rar a -r testrar.rar studio.h test2.mp3`

rar 解压:

`unrar x 压缩包名 (带.rar 后缀)`

zip 压缩:

`zip -r 压缩包名 (带.zip 后缀) 压缩材料。`

`zip -r testzip.zip dir studio.h test2.mp3`

zip 解压:

`unzip 压缩包名 (带.zip 后缀)`

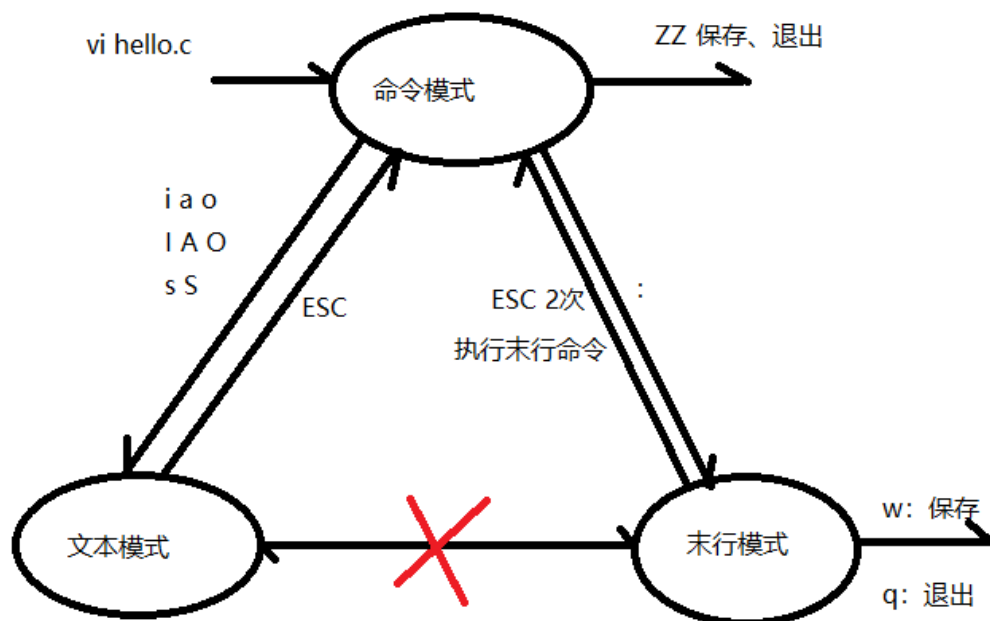
`unzip testzip.zip`

17P-复习

创建一个目录，大小默认是 4096k

```
zhcode@ubuntu:~/Code$ mkdir test1
zhcode@ubuntu:~/Code$ ll test1
total 8
drwxrwxr-x 2 zhcode zhcode 4096 May 21 10:38 ./
drwxrwxr-x 6 zhcode zhcode 4096 May 21 10:38 ../
```

18P-vim 的三种工作模式



19P-vim 基本操作-跳转和删字符

i 进入编辑模式，光标前插入字符
a 进入编辑模式，光标后插入字符
o 进入编辑模式，光标所在行的下一行插入
I 进入编辑模式，光标所在行的行首插入
A 进入编辑模式，光标所在行的行末插入字符
O 进入编辑模式，光标所在行的上一行插入字符
s 删除光标所在字符并进入编辑模式
S 删除光标所在行并进入编辑模式

x 删除光标所在字符，工作模式不变
dw 删除光标所在单词，要求光标在首字母上，如果不在首字母，只会删除当前位置到单词末，工作模式不变
D 删除光标所在位置到行末，工作模式不变
0(数字) 光标移到行首，工作模式不变
\$ 光标移到行尾，工作模式不变
d0 删除光标所在位置到行首，工作模式不变
d\$ 删除光标所在位置到行末，工作模式不变

命令模式下的光标移动

h 左移
j 下移
k 上移
l 右移

命令模式下行跳转

line-G 缺点是没有回显

末行模式下行跳转

:line-回车

跳转首行

gg (命令模式)

跳转末行

G (命令模式)

自动缩进

在这之前要进行 vimrc 修改，不然自动缩进是 8 个空格

ubuntu 的 vimrc 位置在 /etc/vim/vimrc

在文件末尾添加三行：

```
set tabstop=4 //设置制表符宽度为 4
```

```
set softtabstop=4 // 设置软制表符宽度为 4
```

set shiftwidth=4 // 设置缩进空格数为 4
gg=G (命令模式), 无回显

```
22
23 #ifndef _STDIO_H
24
25 if(i == 10) {
26     for(i =0;i<100;i++) {
27         printf("%d", i);
28     }
29 }
30
```

```
22
23 #ifndef _STDIO_H
24
25 if(i == 10) {
26     for(i =0;i<100;i++) {
27         printf("%d", i);
28     }
29 }
30
```

大括号跳转

命令模式下, 光标处于左大括号时, 使用%跳转到对应右大括号, 再按%跳回去。
其他括号也可以这样

20P-vim 基本操作-删除

替换单个字符

r 命令模式下替换光标选中字符

一段删除，即删除指定区域

光标选中要删除的首字符，按 v 进入可视模式，再使用 hjkl 移动到要删除的末尾，按 d 删除

删除整行：

dd，删除光标所在行

n+dd ，删除从光标开始的 n 行

21P-vim 基本操作-复制粘贴

yy 复制光标所在行

p 向后粘贴剪切板内容，如果复制整行，这里是粘贴在光标所在位置的下一行

P 向前粘贴剪切板内容，如果是整行，这里是粘贴在光标所在位置的上一行

这里提一下，上一节里的 dd，不是删除，而是剪切，小时的内容去了剪切板，而不是删掉了

p 和 P 粘贴会出现换行，主要原因是复制整行时，会把行末的换行符也复制下来。

n-yy 复制光标所在位置的 n 行，包括光标所在行

22p-vim 基本操作-查找和替换

查找

/+findname 命令模式下查找

按回车键启动查找后，按 n，会自动找下一个，N 跳到上一个

查找光标所在单词

光标在目标单词上时，*或者#查找下一个，这里不要求光标必须在首字母上

替换：末行模式下进行

单行替换

光标置于待替换行， :s /待替换词/替换词

全文替换

:%s /待替换词/替换词

这个默认替换每行的首个，一行有多个目标词时，后面的不会变

:%s /待替换词/替换词/g

真正意义上的全局替换

区域替换

:24,35s /待替换词/替换词/g

替换 24-35 行之间的目标词

末行模式下历史命令

Ctrl-p 上一条命令

Ctrl-n 下一条命令

23P-vim 基本操作-其他

命令模式下

u 撤销操作

Ctrl-r 反撤销

分屏，末行模式下

:sp 水平分屏

:vsp 竖直分屏

分屏命令+filename，分屏并打开这个文件

分屏后屏幕切换，Ctrl-w-w

使用:q 退出光标所在窗口

使用:qall 退出所有窗口

从 vim 中跳转 manpage，命令模式下

将光标放在待查看单词上，按 K，默认看第一卷

n+K，查看第 n 卷

查看宏定义：命令模式

光标放在待查看词上，[+d 即可查看

vim 下使用 shell 命令：末行模式

:! + 命令

操作后，会切换至终端显示结果，出现如下画面，按 Enter 后回到 vim 界面

```
zhcode@ubuntu:~/Code/Mydir$ vi stdio.h

[No write since last change]
file.s  file.s2  hello  hello.c  holo.c  kikyo.c  rin.jpg  stdio.h

Press ENTER or type command to continue
```


24P-vim 配置思路

两个 vim 配置文件

1. `/etc/vim/vimrc`
2. `~/.vimrc`

其中，第二个配置文件会优先加载，属于用户配置

25P-gcc 编译 4 步骤

gcc 编译可以执行程序 4 步骤：预处理、编译、汇编、链接



26P-gcc 编译常用参数

当头文件和源码不在一个目录下时，需要指定头文件

下图是头文件和源码在同一个目录下

```
zhcode@ubuntu:~/Code/Mydir$ ls
file.s file.s2 hello.c hello.h holo.c kikyo.c rin.jpg stdio.h
zhcode@ubuntu:~/Code/Mydir$ gcc hello.c -o myhello
zhcode@ubuntu:~/Code/Mydir$ ./myhello
my wife is megumin
zhcode@ubuntu:~/Code/Mydir$
```

将 hello.h 放入新建的文件夹 hellodir 之后，编译会失败

```
zhcode@ubuntu:~/Code/Mydir$ ls
file.s file.s2 hello.c hellodir holo.c kikyo.c rin.jpg stdio.h
zhcode@ubuntu:~/Code/Mydir$ gcc hello.c -o myhello
hello.c:1:18: fatal error: hello.h: No such file or directory
compilation terminated.
zhcode@ubuntu:~/Code/Mydir$
```

```
gcc -I ./hellodir hello.c -o hello
```

其中-I 参数指定头文件所在位置，位置可以在编译文件前，也可以在后面

```
zhcode@ubuntu:~/Code/Mydir$ gcc -I ./hellodir hello.c -o myhello
zhcode@ubuntu:~/Code/Mydir$ ./myhello
my wife is megumin
zhcode@ubuntu:~/Code/Mydir$
```

- I 指定头文件所在目录位置
- c 只做预处理，编译，汇编。得到二进制文件
- g 编译时添加调试文件，用于 gdb 调试
- Wall 显示所有警告信息
- D 向程序中“动态”注册宏定义
- l 指定动态库库名
- L 指定动态库路径

```
itcast@itcast:~/28_Linux/gcctest$ gcc hello.c -D HELLO
itcast@itcast:~/28_Linux/gcctest$ cat hello.c
#include "hello.h"

#ifdef HELLO

#define HI 20

#endif

int main(int argc, char *argv[])
{
    int b = 10;
    int a = 10;

    printf("-----%d\n", HI);

    return 0;
}
itcast@itcast:~/28_Linux/gcctest$ █
```

27P-午后复习

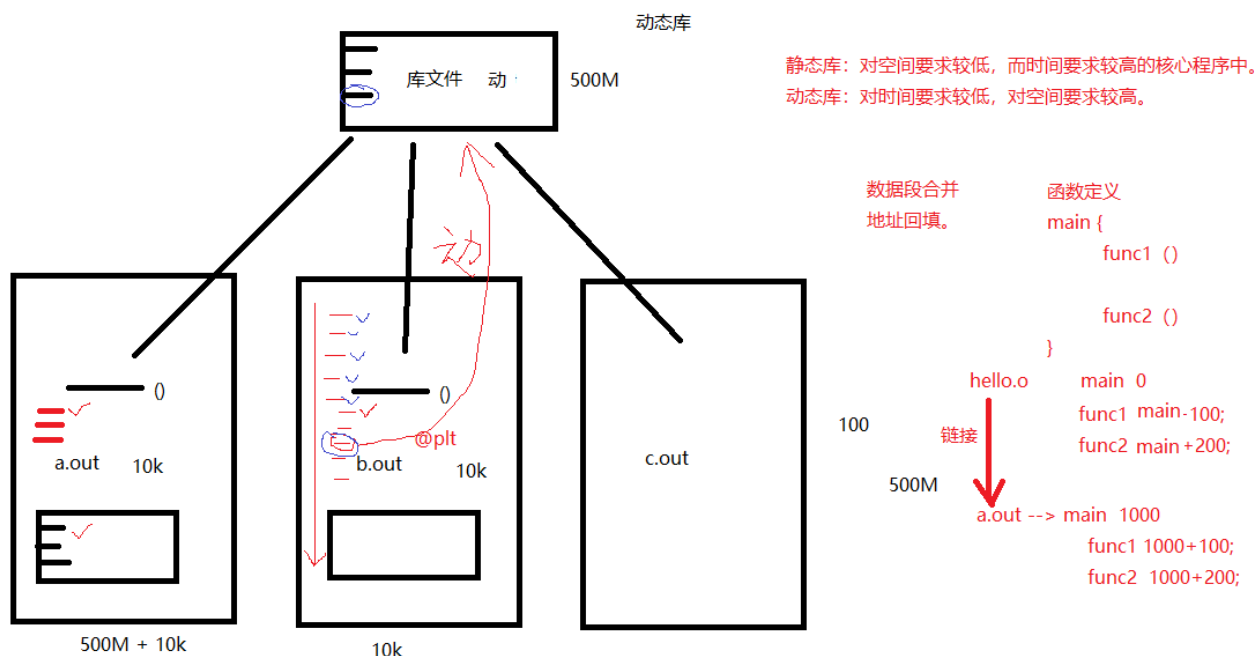
28P-动态库和静态库理论对比

静态库在文件中静态展开，所以有多少文件就展开多少次，非常吃内存，100M 展开 100 次，就是 1G，

但是这样的好处就是静态加载的速度快

使用动态库会将动态库加载到内存，10 个文件也只需要加载一次，然后这些文件用到库的时候临时去加载，速度慢一些，但是很省内存

动态库和静态库各有优劣，根据实际情况合理选用即可。



29P-静态库制作

静态库名字以 lib 开头，以 .a 结尾

例如：libmylib.a

静态库生成指令

```
ar rcs libmylib.a file1.o
```

步骤一：

写好源代码

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
add.c  div1.c  sub.c
zhcode@ubuntu:~/Code/Mydir/staticLib$ cat add.c
int add(int a, int b){
    return a+b;
}
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

步骤二：

编译源代码生成 .o 文件

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc -c add.c -o add.o
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc -c sub.c -o sub.o
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc -c div1.c -o div1.o
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
add.c  add.o  div1.c  div1.o  sub.c  sub.o
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

步骤三：

制作静态库

```
ar rcs libname.a file1.o file2.o ...
```

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ ar rcs libmymath.a add.o sub.o div1.o
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
add.c  add.o  div1.c  div1.o  libmymath.a  sub.c  sub.o
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

静态库的使用：

```
gcc test.c lib库名.a -o a.out
```

```

zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc test.c libmymath.a -o test
test.c: In function 'main':
test.c:9:33: warning: implicit declaration of function 'add' [-Wimplicit-function-declaration]
printf("%d + %d = %d\n", a, b, add(a, b));
                              ^
test.c:10:33: warning: implicit declaration of function 'sub' [-Wimplicit-function-declaration]
printf("%d - %d = %d\n", a, b, sub(a, b));
                              ^
test.c:11:33: warning: implicit declaration of function 'div1' [-Wimplicit-function-declaration]
printf("%d / %d = %d\n", a, b, div1(a, b));
                              ^
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
libmymath.a  resource  test  test.c
zhcode@ubuntu:~/Code/Mydir/staticLib$ ./test
9 + 3 = 12
9 - 3 = 6
9 / 3 = 3
zhcode@ubuntu:~/Code/Mydir/staticLib$

```

上图中，test.c 直接使用了 mymath 库中的 add, sub, div1 函数，所以在编译时要导入静态库。编译时出现了函数未定义的警告，可以忽略，让系统生成默认的定义。

```

-rw-rw-r-- 1 zhcode zhcode 3990 May 21 19:01 libmymath.a
drwxrwxr-x 2 zhcode zhcode 4096 May 21 19:06 resource/
-rwxrwxr-x 1 zhcode zhcode 8776 May 21 19:20 test*
-rw-rw-r-- 1 zhcode zhcode 296 May 21 19:17 test.c
zhcode@ubuntu:~/Code/Mydir/staticLib$

```

test.c 只有 296，然而 test 有 8776，所以静态库使用时，是直接编译到文件里的。

30P-静态库使用及头文件对应

上一节出现的警告，用编译器隐式声明来解决的

编译器只能隐式声明返回值为 `int` 的函数形式：`int add(int ,int );`

如果函数不是返回的 `int`，则隐式声明失效，会报错

在 `test.c` 中加入函数声明：

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<string.h>
4 #include<unistd.h>
5 #include<pthread.h>
6
7 int add(int, int);
8 int sub(int, int);
9 int div1(int, int);
10
11 int main(int argc, char *argv[]) {
12     int a = 9, b = 3;
13     printf("%d + %d = %d\n", a, b, add(a, b));
14     printf("%d - %d = %d\n", a, b, sub(a, b));
15     printf("%d / %d = %d\n", a, b, div1(a, b));
16     return 0;
17 }
```

再次进行编译，就不会报错了：

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc test.c libmymath.a -o test
zhcode@ubuntu:~/Code/Mydir/staticLib$ ./test
9 + 3 = 12
9 - 3 = 6
9 / 3 = 3
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

上面这个方法，需要库的使用者知道库里的函数，完事儿一个一个加到代码里，不太行

下面使用头文件的方法来加载静态库

```
1 #ifndef MYMATH_H
2 #define MYMATH_H
3
4 int add(int, int);
5 int sub(int, int);
6 int div1(int, int);
7
8 #endif
~
~
~
~
~
~
~
~
~
~
mymath.h 8,6 All test.c 8,0-1 All
```

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<string.h>
4 #include<unistd.h>
5 #include<pthread.h>
6
7 #include "mymath.h"
8
9 int main(int argc, char *argv[]){
10     int a = 9, b = 3;
11     printf("%d + %d = %d\n", a, b, add(a, b));
12     printf("%d - %d = %d\n", a, b, sub(a, b));
13     printf("%d / %d = %d\n", a, b, div1(a, b));
14
15     return 0;
16 }
```

左边的 define 为头文件守卫，防止在代码中多次 include 头文件，多次展开静态库，带来的额外开销

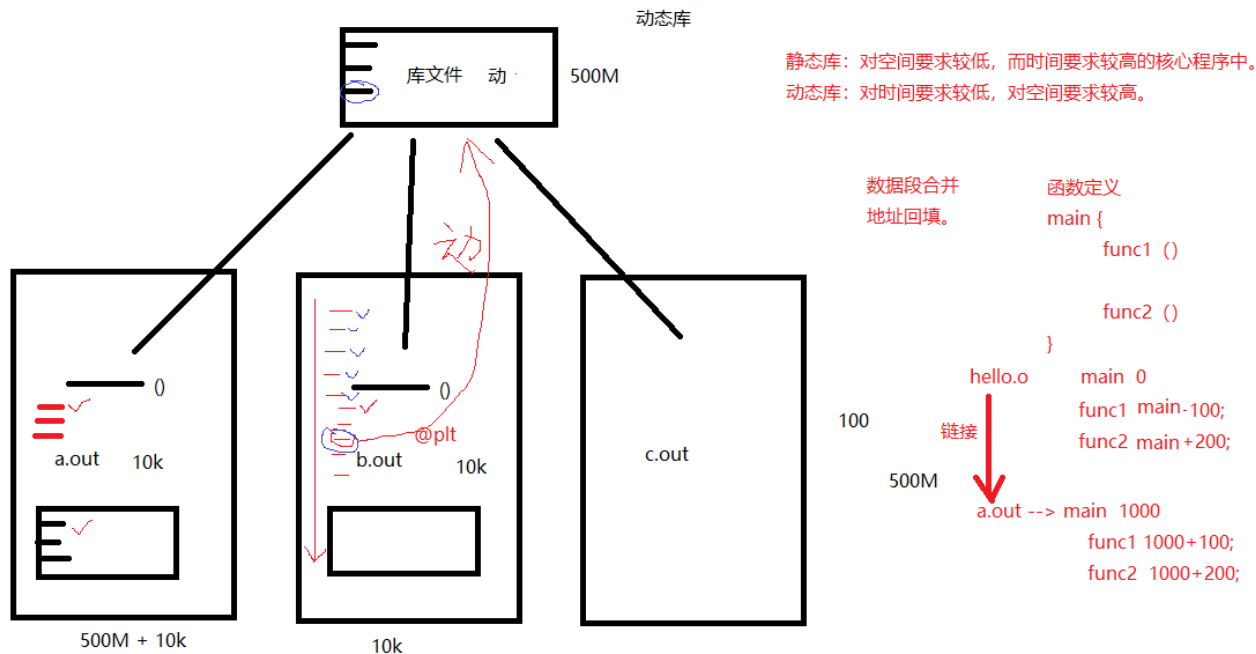
这样也不会报错了，而且更加科学

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc test.c libmymath.a -o test
zhcode@ubuntu:~/Code/Mydir/staticLib$ ./test
9 + 3 = 12
9 - 3 = 6
9 / 3 = 3
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

下面将静态库和头文件分别放至其他目录下
运行过程如下

```
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
libmymath.a mymath.h resource test test.c
zhcode@ubuntu:~/Code/Mydir/staticLib$ rm test
zhcode@ubuntu:~/Code/Mydir/staticLib$ mkdir inc
zhcode@ubuntu:~/Code/Mydir/staticLib$ mkdir lib
zhcode@ubuntu:~/Code/Mydir/staticLib$ mv *.h inc
zhcode@ubuntu:~/Code/Mydir/staticLib$ mv *.a lib
zhcode@ubuntu:~/Code/Mydir/staticLib$ gcc test.c ./lib/libmymath.a -o a.out -I ./inc
zhcode@ubuntu:~/Code/Mydir/staticLib$ ls
a.out inc lib resource test.c
zhcode@ubuntu:~/Code/Mydir/staticLib$ ./a.out
9 + 3 = 12
9 - 3 = 6
9 / 3 = 3
zhcode@ubuntu:~/Code/Mydir/staticLib$
```

31P-动态库制作-生成与位置无关代码



写在源代码里的函数，相对 main 函数偏移是一定的，链接时，回填 main 函数地址之后，其他源代码里的函数也就得到了地址。

动态库里的函数会用一个@plt 来标识，当动态库加载到内存时，再用加载进去的地址将@plt 替换掉。

制作动态库的步骤

步骤一：生成位置无关的.o 文件

```
gcc -c add.c -o add.o -fPIC
```

使用这个参数过后，生成的函数就和位置无关，挂上@plt 标识，等待动态绑定

32P-动态库制作-演示

制作动态库的步骤

1. 生成位置无关的.o 文件
gcc -c add.c -o add.o -fPIC
使用这个参数过后，生成的函数就和位置无关，挂上@plt 标识，等待动态绑定
2. 使用 gcc -shared 制作动态库
gcc -shared -o lib 库名.so add.o sub.o div.o
3. 编译可执行程序时指定所使用的动态库。-l:指定库名 -L:指定库路径
gcc test.c -o a.out -l mymath -L ./lib
4. 运行可执行程序./a.out

过程演示：

步骤一：生成位置无关的.o 文件

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ ls
add.c  div1.c  sub.c
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ gcc -c add.c -o add.o -fPIC
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ gcc -c sub.c -o sub.o -fPIC
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ gcc -c div1.c -o div1.o -fPIC
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ ls
add.c  add.o  div1.c  div1.o  sub.c  sub.o
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$
```

步骤二：制作动态库 gcc -shared -o lib 库名.so add.o sub.o div.o

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ gcc -shared -o libmymath.so add.o sub.o div1.o
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$ ls
add.c  add.o  div1.c  div1.o  libmymath.so  sub.c  sub.o
zhcode@ubuntu:~/Code/Mydir/dynamicLib/resource$
```

步骤三：编译程序

文件分布如下：动态库在 lib 目录下，头文件在 inc 目录下

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ls
inc  lib  resource  test.c
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ls inc
mymath.h
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ls lib
libmymath.so
zhcode@ubuntu:~/Code/Mydir/dynamicLib$
```

下面编译文件

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ gcc test.c -o a.out -l mymath -L ./lib -I ./inc
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ls
a.out  inc  lib  resource  test.c
zhcode@ubuntu:~/Code/Mydir/dynamicLib$
```

步骤四：执行文件，出错

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ./a.out
./a.out: error while loading shared libraries: libmymath.so: cannot open shared object file: No such file or directory
zhcode@ubuntu:~/Code/Mydir/dynamicLib$
```

33P-动态库加载错误原因及解决方式

出错原因分析：

连接器： 工作于链接阶段，工作时需要 `-l` 和 `-L`

动态链接器： 工作于程序运行阶段，工作时需要提供动态库所在目录位置

指定动态库路径并使其生效，然后再执行文件

通过环境变量指定动态库所在位置：`export LD_LIBRARY_PATH=动态库路径`

```
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ export LD_LIBRARY_PATH=./lib
zhcode@ubuntu:~/Code/Mydir/dynamicLib$ ./a.out
9 + 3 = 12
9 - 3 = 6
9 / 3 = 3
zhcode@ubuntu:~/Code/Mydir/dynamicLib$
```

当关闭终端，再次执行 `a.out` 时，又报错。

这是因为，环境变量是进程的概念，关闭终端之后再打开，是两个进程，环境变量发生了变化。

要想永久生效，需要修改 `bash` 的配置文件：`vi ~/.bashrc`

修改后要使配置文件立即生效：`source ~/.bashrc` 或者 `source .bashrc` 或者重开终端让其自己加载
这下再执行 `a.out` 就不会报错了

34P-动态库加载错误原因及解决方式 2

解决方式：

【1】 通过环境变量： `export LD_LIBRARY_PATH=动态库路径`
`./a.out` 成功!!! （临时生效， 终端重启环境变量失效）

【2】 永久生效： 写入 终端配置文件。 `.bashrc` 建议使用绝对路径。

1) `vi ~/.bashrc`

2) 写入 `export LD_LIBRARY_PATH=动态库路径` 保存

3) `source ~/.bashrc` / `source .bashrc` / 重启 终端 ---> 让修改后的.bashrc 生效

4) `./a.out` 成功!!!

【3】 拷贝自定义动态库 到 `/lib`（标准 C 库所在目录位置）

【4】 配置文件法

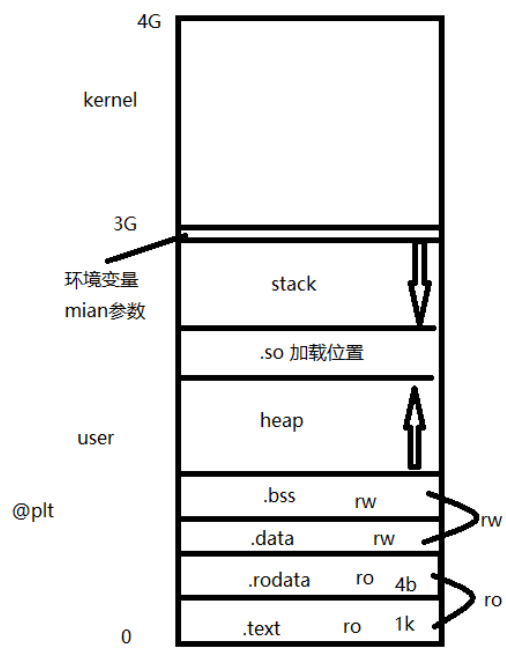
1) `sudo vi /etc/ld.so.conf`

2) 写入 动态库绝对路径 保存

3) `sudo ldconfig -v` 使配置文件生效。

4) `./a.out` 成功!!! --- 使用 `ldd a.out` 查看

35P-扩展讲解-数据段合并



36P-总结

跳转到指定行：

1. 88G （命令模式）
2. :88 （末行模式）

跳转文件首：

gg （命令模式）

跳转文件尾：

G （命令模式）

自动格式化程序：

gg=G （命令模式）

大括号对应：

% （命令模式）

光标移至行首：

0 （命令模式）执行结束，工作模式不变。

光标移至行尾：

\$ （命令模式）执行结束，工作模式不变。

删除单个字符：

x （命令模式）执行结束，工作模式不变。

替换单个字符：

将待替换的字符用光标选中， r （命令模式），再按欲替换的字符

删除一个单词：

dw （命令模式）光标置于单词的首字母进行操作。

删除光标至行尾：

D 或者 d\$（命令模式）

删除光标至行首：

d0（命令模式）

删除指定区域：

按 V（命令模式）切换为“可视模式”，使用 hjkl 挪移光标来选中待删除区域。按 d 删除该区域数据。

删除指定 1 行：

在光标所在行，按 dd（命令模式）

删除指定 N 行：

在光标所待删除首行，按 Ndd（命令模式）

复制一行：

yy

粘贴：

p：向后、P：向前。

查找：

1. 找 设想 内容：

命令模式下，按 “/” 输入欲搜索关键字，回车。使用 n 检索下一个。

2. 找 看到的内容：

命令模式下，将光标置于单词任意一个字符上，按 “*” / “#”

单行替换：

将光标置于待替换行上，进入末行模式，输入 :s /原数据/新数据

通篇替换：

末行模式，:%s /原数据/新数据/g g:不加，只替换每行首个。 sed

指定行的替换：

末行模式， : 起始行号，终止行号 s /原数据/新数据/g g:不加，只替换每行首个。

:29,35s /printf/println/g

撤销、反撤销:

u、ctrl+r (命令模式)

分屏:

sp: 横屏分。Ctrl+ww 切换。

vsp: 竖屏分。Ctrl+ww 切换。

跳转至 man 手册:

将光标置于待查看函数单词上，使用 K (命令模式) 跳转。指定卷， nK

查看宏定义:

将光标置于待查看宏定义单词上，使用 [d 查看定义语句。

在末行模式执行 shell 命令:

:!命令 :! ls -l

gcc 编译:

4 步骤: 预处理、编译、汇编、连接。

-I: 指定头文件所在目录位置。

-c: 只做预处理、编译、汇编。得到 二进制 文件!!!

-g: 编译时添加调试语句。 主要支持 gdb 调试。

-Wall: 显示所有警告信息。

-D: 向程序中“动态”注册宏定义。 #define NAME VALUE

静态库制作及使用步骤:

1. 将 .c 生成 .o 文件

gcc -c add.c -o add.o

2. 使用 ar 工具制作静态库

```
ar rcs lib库名.a add.o sub.o div.o
```

3. 编译静态库到可执行文件中:

```
gcc test.c lib库名.a -o a.out
```

头文件守卫: 防止头文件被重复包含

```
#ifndef _HEAD_H_

#define _HEAD_H_

.....

#endif
```

动态库制作及使用:

1. 将 .c 生成 .o 文件, (生成与位置无关的代码 -fPIC)

```
gcc -c add.c -o add.o -fPIC
```

2. 使用 gcc -shared 制作动态库

```
gcc -shared -o lib库名.so add.o sub.o div.o
```

3. 编译可执行程序时, 指定所使用的动态库。 -l: 指定库名(去掉 lib 前缀和 .so 后缀) -L: 指定库路径。

```
gcc test.c -o a.out -lmymath -L./lib
```

4. 运行可以执行程序 ./a.out 出错!!!! --- ldd a.out --> "not found"

error while loading shared libraries: libxxx.so: cannot open shared object file:
No such file or directory

原因:

链接器: 工作于链接阶段, 工作时需要 -l 和 -L

动态链接器: 工作于程序运行阶段, 工作时需要提供动态库所在目录位置。

解决方式:

【1】 通过环境变量: export LD_LIBRARY_PATH=动态库路径

./a.out 成功!!! (临时生效, 终端重启环境变量失效)

【2】 永久生效： 写入 终端配置文件。 `.bashrc` 建议使用绝对路径。

1) `vi ~/.bashrc`

2) 写入 `export LD_LIBRARY_PATH=动态库路径` 保存

3) `source .bashrc` / `source ~/.bashrc` / 重启 终端 ---> 让修改后的 `.bashrc` 生效

4) `./a.out` 成功!!!

【3】 拷贝自定义动态库 到 `/lib` (标准 C 库所在目录位置)

【4】 配置文件法

1) `sudo vi /etc/ld.so.conf`

2) 写入 动态库绝对路径 保存

3) `sudo ldconfig -v` 使配置文件生效。

4) `./a.out` 成功!!! --- 使用 `ldd a.out` 查看

37P-复习

38P-gdb 调试基础指令

使用 gdb 之前，要求对文件进行编译时增加-g 参数，加了这个参数过后生成的编译文件会大一些，这是因为增加了 gdb 调试内容

gdb 调试工具： 大前提：程序是你自己写的。 ——逻辑错误

基础指令：

-g：使用该参数编译可以执行文件，得到调试表。

gdb ./a.out

list: list 1 列出源码。根据源码指定 行号设置断点。

b: b 20 在 20 行位置设置断点。

run/r: 运行程序

n/next: 下一条指令（会越过函数）

s/step: 下一条指令（会进入函数）

p/print: p i 查看变量的值。

continue: 继续执行断点后续指令。

finish: 结束当前函数调用。

quit: 退出 gdb 当前调试。

其他指令：

run: 使用 run 查找段错误出现位置。

set args: 设置 main 函数命令行参数 （在 start、run 之前）

run 字符串1 字符串2 ...: 设置 main 函数命令行参数

info b: 查看断点信息表

b 20 if i = 5: 设置条件断点。

ptype: 查看变量类型。

bt: 列出当前程序正存活着的栈帧。

frame: 根据栈帧编号，切换栈帧。

display: 设置跟踪变量

undisplay: 取消设置跟踪变量。 使用跟踪变量的编号。

```
gcc gdbtest.c -o a.out -g
```

```
zhcode@ubuntu:~/Code/Mydir/gdbtest$ gcc test.c -o test -g
zhcode@ubuntu:~/Code/Mydir/gdbtest$ ls
test  test.c
zhcode@ubuntu:~/Code/Mydir/gdbtest$
```

`gdb test` 启动对 test 的调试

```
zhcode@ubuntu:~/Code/Mydir/gdbtest$ gdb test
GNU gdb (Ubuntu 7.11-0ubuntu1) 7.11
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test...done.
(gdb)
```

`1/list n` 从第 n 行开始显示程序，后续继续输入 `list/1`，就可以显示后面的代码

```
(gdb) list 1
1      #include <stdio.h>
2
3      void myprint(int);
4
5      int main(){
6          int i=0;
7          printf("my wife is alice\n");
8
9          for(i=0; i<10; i++)
10             myprint(i);
(gdb) list
11
12             return 0;
13     }
14
15     void myprint(int i){
16         if(i%2 == 1)
17             printf("this turn, i = %d\n", i);
18
19         return;
20     }
(gdb)
```


break/b n 在第 n 行设置断点，断点那一行不会执行

```
(gdb) break 8
Breakpoint 1 at 0x40057f: file test.c, line 8.
(gdb) █
```

run/r 运行程序

```
(gdb) run
Starting program: /home/zhcode/Code/Mydir/gdbtest/test
my wife is alice

Breakpoint 1, main () at test.c:9
9             for(i=0; i<10; i++)
(gdb) █
```

接下来按 next/n/step/s 继续向下执行

next/n :下一个，调用函数就跑

step/s :单步，会进入调用的函数

要注意的是，如果是系统函数，按 s 就出不来了，这时用 until+行号直接执行到行号处

```
Breakpoint 1, main () at test.c:9
9             for(i=0; i<10; i++)
(gdb) step
10                 myprint(i);
(gdb) step
myprint (i=0) at test.c:16
16             if(i%2 == 1)
(gdb) step
20         }
(gdb) step
main () at test.c:9
9             for(i=0; i<10; i++)
(gdb) next
10                 myprint(i);
(gdb) next
this turn, i = 1
9             for(i=0; i<10; i++)
(gdb) next
10                 myprint(i);
(gdb) next
9             for(i=0; i<10; i++)
(gdb) █
```

出不来的示例

```
myprint (i=5) at test.c:16
16             if(i%2 == 1)
(gdb) s
17             printf("this turn, i = %d\n", i);
(gdb) s
__printf (format=0x400675 "this turn, i = %d\n") at printf.c:28
28 printf.c: No such file or directory.
(gdb) s
32 in printf.c
(gdb) s
33 in printf.c
(gdb) █
```

使用 until 出来

```
33 in printf.c
(gdb) s
32 in printf.c
(gdb) until 18
this turn, i = 5
myprint (i=5) at test.c:19
19 return;
(gdb) n
20 }
(gdb) n
main () at test.c:9
9 for(i=0; i<10; i++)
(gdb)
```

print/p i 查看 i 变量的值

```
this turn, i = 7
9 for(i=0; i<10; i++)
(gdb) p i
$3 = 7
(gdb)
```

continue 直接运行到结束

```
(gdb) continue
Continuing.
this turn, i = 9
[Inferior 1 (process 2668) exited normally]
(gdb) █
```

39P-gdb 调试其他指令

其他指令：

run：使用 run 查找段错误出现位置。

set args： 设置 main 函数命令行参数 （在 start、run 之前）

run 字符串 1 字符串 2 ...： 设置 main 函数命令行参数

info b： 查看断点信息表

b 20 if i = 5： 设置条件断点。

ptype： 查看变量类型。

bt： 列出当前程序正存活着的栈帧。

frame： 根据栈帧编号，切换栈帧。

display： 设置跟踪变量

undisplay： 取消设置跟踪变量。 使用跟踪变量的编号。

40P-gdb 常见错误说明

没有符号被读取—编译时没加-g 参数

file 后面加使用-g 编译的文件，可以不用退出，gdb 直接读取后进行调试。

41P-makefile 基础规则

makefile: 管理项目。

命名: makefile Makefile --- make 命令

1 个规则:

目标: 依赖条件
(一个 tab 缩进) 命令

1. 目标的时间必须晚于依赖条件的时间, 否则, 更新目标
2. 依赖条件如果不存在, 找寻新的规则去产生依赖条件。

ALL: 指定 makefile 的终极目标。

2 个函数:

src = \$(wildcard ./*.c): 匹配当前工作目录下的所有.c 文件。将文件名组成列表, 赋值给变量 src。 src = add.c sub.c div1.c

obj = \$(patsubst %.c, %.o, \$(src)): 将参数 3 中, 包含参数 1 的部分, 替换为参数 2。
obj = add.o sub.o div1.o

clean: (没有依赖)

-rm -rf \$(obj) a.out “-”: 作用是, 删除不存在文件时, 不报错。顺序执行结束。

3 个自动变量:

\$\$: 在规则的命令中, 表示规则中的目标。

\$\$: 在规则的命令中, 表示所有依赖条件。

\$\$: 在规则的命令中, 表示第一个依赖条件。如果将该变量应用在模式规则中, 它可将依赖

条件列表中的依赖依次取出，套用模式规则。

模式规则：

```
%.o:%.c
gcc -c $< -o %@
```

静态模式规则：

```
$(obj):%.o:%.c
gcc -c $< -o %@
```

伪目标：

```
.PHONY: clean ALL
```

参数：

-n: 模拟执行 make、make clean 命令。

-f: 指定文件执行 make 命令。 xxxx.mk

下面来一步一步升级 makefile

第一个版本的 Makefile:

makefile 的依赖是从上至下的，换句话说就是目标文件是第一句里的目标，如果不满足执行依赖，就会继续向下执行。如果满足了生成目标的依赖，就不会再继续向下执行了。

make 会自动寻找规则里需要的材料文件，执行规则下面的行为生成规则中的目标。

```
1
2 hello:hello.o
3     gcc hello.o -o hello
4
5 hello.o:hello.c
6     gcc -c hello.c -o hello.o
```

执行 make 指令

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
hello.c  Makefile
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c hello.c -o hello.o
gcc hello.o -o hello
zhcode@ubuntu:~/Code/Mydir/make$ ls
hello  hello.c  hello.o  Makefile
zhcode@ubuntu:~/Code/Mydir/make$
```

运行 hello，没有问题

```
zhcode@ubuntu:~/Code/Mydir/make$ ./hello
my wife is megumin!!!
hello megumin!!!
my wife is alice!!!
hello alice!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

42P-makefile 的一个规则

修改 hello.c 如下:

```
1 #include<stdio.h>
2
3 int add(int , int);
4 int sub(int , int);
5 int div1(int , int);
6
7 int main(int argc, char *argv[]){
8     int a = 36, b = 12;
9
10    printf("%d + %d = %d\n", a, b, add(a, b));
11    printf("%d - %d = %d\n", a, b, sub(a, b));
12    printf("%d / %d = %d\n", a, b, div1(a, b));
13
14    printf("my wife is megumin!!!\n");
15
16    return 0;
17 }
```

此时要进行编译,则需要多文件联合编译:

```
gcc hello.c add.c sub.c div1.c -o a.out
```

```
zhcode@ubuntu:~/Code/Mydir/make$ gcc hello.c add.c sub.c div1.c -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 48
36 - 12 = 24
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

对这个新的代码,写出下面的 makefile

```
1 a.out:hello.c add.c sub.c div1.c
2     gcc hello.c add.c sub.c div1.c -o a.out
```

执行：

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c hello.o makefile sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc hello.c add.c sub.c div1.c -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 48
36 - 12 = 24
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

此时，修改 add.c 为下图

```
1 int add(int a, int b){
2     return a+b+1;
3 }
```

此时，再使用 make，发现了问题

```
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc hello.c add.c sub.c div1.c -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 24
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

可以看到，只修改 add.c，但是编译的时候，其他.c 文件也重新编译了，这不太科学。明明只改了一个，全部都重新编译了

于是将 makefile 改写如下：

```
1 a.out:hello.o add.o sub.o div1.o
2     gcc hello.o add.o sub.o div1.o -o a.out
3
4 hello.o:hello.c
5     gcc -c hello.c -o hello.o
6
7 add.o:add.c
8     gcc -c add.c -o add.o
9
10 sub.o:sub.c
11     gcc -c sub.c -o sub.o
12
13 div1.o:div1.c
14     gcc -c div1.c -o div1.o
15
```

执行 make 指令如下


```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c div1.c -o div1.o
gcc hello.o add.o sub.o div1.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c add.o a.out div1.c div1.o hello hello.c hello.o makefile mymake sub.c sub.o
zhcode@ubuntu:~/Code/Mydir/make$
```

执行程序输出如下：

```
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 24
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

此时修改 sub 为下图：

```
1 int sub(int a, int b){
2     return a-b+111;
3 }
```

再次 make

```
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c sub.c -o sub.o
gcc hello.o add.o sub.o div1.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$
```

可以看到，只重新编译了修改过的 sub.c 和最终目标

执行程序：

```
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

makefile 检测原理：

修改文件后，文件的修改时间发生变化，会出现目标文件的时间早于作为依赖材料的时间，出现这种情况的文件会重新编译。

修改 sub.c 后，sub.o 的时间就早于 sub.c，a.out 的时间也早于 sub.o 的时间了，于是重新编译这两文件了。

关于 makefile 指定目标问题，先修改 makefile 如下：

```
1 hello.o:hello.c
2     gcc -c hello.c -o hello.o
3
4 add.o:add.c
5     gcc -c add.c -o add.o
6
7 sub.o:sub.c
8     gcc -c sub.c -o sub.o
9
10 div1.o:div1.c
11     gcc -c div1.c -o div1.o
12
13 a.out:hello.o add.o sub.o div1.o
14     gcc hello.o add.o sub.o div1.o -o a.out
```

只是将 a.out 放在了文件末尾
执行 make，如下：

```
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c hello.c -o hello.o
zhcode@ubuntu:~/Code/Mydir/make$
```

这是因为，makefile 默认第一个目标文件为终极目标，生成就跑路，这时候可以用 ALL 来指定终极目标
指定目标的 makefile

```
1 ALL:a.out
2
3 hello.o:hello.c
4     gcc -c hello.c -o hello.o
5
6 add.o:add.c
7     gcc -c add.c -o add.o
8
9 sub.o:sub.c
10    gcc -c sub.c -o sub.o
11
12 div1.o:div1.c
13    gcc -c div1.c -o div1.o
14
15 a.out:hello.o add.o sub.o div1.o
16    gcc hello.o add.o sub.o div1.o -o a.out
```

执行:

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c div1.c -o div1.o
gcc hello.o add.o sub.o div1.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

43P-午后回顾

44P-makefile 两个函数和 clean

```
src = $(wildcard *.c)
```

找到当前目录下所有后缀为.c 的文件，赋值给 src

```
obj = $(patsubst %.c, %.o, $(src))
```

把 src 变量里所有后缀为.c 的文件替换成.o

用这两个函数修改 makefile 如下：

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 hello.o:hello.c
8     gcc -c hello.c -o hello.o
9
10 add.o:add.c
11     gcc -c add.c -o add.o
12
13 sub.o:sub.c
14     gcc -c sub.c -o sub.o
15
16 div1.o:div1.c
17     gcc -c div1.c -o div1.o
18
19 a.out: $(obj)
20     gcc $(obj) -o a.out
```

执行，make 指令，如下所示：

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc div1.o hello.o add.o sub.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

每次要删除.o 文件，很恶心，于是改写 makefile 如下：
加了 clean 部分

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 hello.o:hello.c
8     gcc -c hello.c -o hello.o
9
10 add.o:add.c
11     gcc -c add.c -o add.o
12
13 sub.o:sub.c
14     gcc -c sub.c -o sub.o
15
16 div1.o:div1.c
17     gcc -c div1.c -o div1.o
18
19 a.out: $(obj)
20     gcc $(obj) -o a.out
21
22 clean:
23     -rm -rf $(obj) a.out
```

rm 前面的-，代表出错依然执行。比如，待删除文件集合是 5 个，已经手动删除了 1 个，就只剩下 4 个，然而删除命令里面还是 5 个的集合，就会有删除不存在文件的问题，不加这-，就会报错，告诉你有一个文件找不到。加了-就不会因为这个报错。

执行 make：

```
zhcode@ubuntu:~/Code/Mydir/make$ make
make: Nothing to be done for 'ALL'.
zhcode@ubuntu:~/Code/Mydir/make$
```

由于没有文件变动，a.out 的时间戳晚于所有依赖文件，这里 make 就没干活

于是，执行时加新指令，先模拟执行 clean 部分：

```
zhcode@ubuntu:~/Code/Mydir/make$ make clean -n
rm -rf div1.o hello.o add.o sub.o a.out
zhcode@ubuntu:~/Code/Mydir/make$
```

可以看到模拟执行后，会删除哪些文件。

确定没有问题，执行

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c add.o a.out div1.c div1.o hello hello.c hello.o m2 makefile mymake sub.c sub.o
zhcode@ubuntu:~/Code/Mydir/make$ make clean
rm -rf div1.o hello.o add.o sub.o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c m2 makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$
```

45P-makefile3 个自动变量和模式规则

3 个自动变量

\$@ ： 在规则命令中，表示规则中的目标

\$< ： 在规则命令中，表示规则中的第一个条件，如果将该变量用在模式规则中，它可以将依赖条件列表中的依赖依次取出，套用模式规则

\$^ ： 在规则命令中，表示规则中的所有条件，组成一个列表，以空格隔开，如果这个列表中有重复项，则去重

用自动变量修改 makefile，如下：

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 hello.o:hello.c
8     gcc -c $< -o $@
9
10 add.o:add.c
11     gcc -c $< -o $@
12
13 sub.o:sub.c
14     gcc -c $< -o $@
15
16 div1.o:div1.c
17     gcc -c $< -o $@
18
19 a.out: $(obj)
20     gcc $^ -o $@
21
22 clean:
23     -rm -rf $(obj) a.out
```

sub, add 这些指令中使用\$<和\$^都能达到效果, 但是为了模式规则, 所以使用的\$< 执行 make, 如下:

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c m2 m3 makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc div1.o hello.o add.o sub.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

再来, 上面这个 makefile, 可扩展性不行。比如, 要添加一个乘法函数, 就需要在 makefile 里面增加乘法函数的部分。不科学, 所以, 模式规则就来了

```
%.o:%.c
    gcc -c $< -o $@
```

修改 makefile, 如下:

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 %.o:%.c
8     gcc -c $< -o $@
9
10 a.out: $(obj)
11     gcc $^ -o $@
12
13 clean:
14     -rm -rf $(obj) a.out
```

执行 make, 如下:

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c m2 m3 makefile mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc div1.o hello.o add.o sub.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```


这时，增加一个 mul 函数，并添加 mul.c 文件如下：

```
1 #include<stdio.h>
2
3 int add(int , int);
4 int sub(int , int);
5 int div1(int , int);
6 int mul(int, int);
7
8 int main(int argc, char *argv[]){
9     int a = 36, b = 12;
10
11     printf("%d + %d = %d\n", a, b, add(a, b));
12     printf("%d - %d = %d\n", a, b, sub(a, b));
13     printf("%d / %d = %d\n", a, b, div1(a, b));
14     printf("%d * %d = %d\n", a, b, mul(a, b));
15
16     printf("my wife is megumin!!!\n");
17
18     return 0;
19 }
```

mul.c 如下：

```
1 int mul(int a, int b){
2     return a*b;
3 }
```

直接执行 make：

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c m2 m3 makefile mul.c mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o
gcc -c mul.c -o mul.o
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc div1.o mul.o hello.o add.o sub.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
36 * 12 = 432
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

增加函数的时候，不用改 makefile，只需要增加.c 文件，改一下源码，就行。很强势。

继续优化 makefile，使用静态模式规则，就是指定模式规则给谁用，这里指定模式规则给 obj 用，以后文件多了，文件集合会有很多个，就需要指定哪个文件集合用什么规则

```
$(obj):%.o:%.c
```

```
gcc -c $< -o $@
```

修改后 makefile 如下：

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 $(obj):%.o:%.c
8     gcc -c $< -o $@
9
10 a.out: $(obj)
11     gcc $^ -o $@
12
13 clean:
14     -rm -rf $(obj) a.out
```

运行如下：

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c div1.c hello hello.c m2 m3 makefile mul.c mymake sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o
gcc -c mul.c -o mul.o
gcc -c hello.c -o hello.o
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc div1.o mul.o hello.o add.o sub.o -o a.out
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
36 * 12 = 432
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

再来一个扩展

当前文件夹下有 ALL 文件或者 clean 文件时，会导致 makefile 瘫痪，如下所示，make clean 没有工作

```
zhcode@ubuntu:~/Code/Mydir/make$ touch clean
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c a.out div1.c hello hello.o m3 mul.c mymake sub.o
add.o clean div1.o hello.c m2 makefile mul.o sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make clean
make: 'clean' is up to date.
zhcode@ubuntu:~/Code/Mydir/make$
```

用伪目标来解决，添加一行 `.PHONY: clean ALL`
makefile 如下所示：

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 ALL:a.out
6
7 $(obj):%.o:%.c
8     gcc -c $< -o $@
9
10 a.out: $(obj)
11     gcc $^ -o $@
12
13 clean:
14     -rm -rf $(obj) a.out
15
16 .PHONY: clean ALL
```

再来执行 `make clean`，就不会受到干扰了

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c  a.out  div1.c  hello  hello.o  m3      mul.c  mymake  sub.o
add.o  clean  div1.o  hello.c  m2      makefile  mul.o  sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make clean
rm -rf div1.o mul.o hello.o add.o sub.o a.out
zhcode@ubuntu:~/Code/Mydir/make$
```

还有一个扩展就是，编译时的参数，`-g`, `-Wall` 这些，可以放在 makefile 里面
修改后 makefile 如下：

```
1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 myArgs = -Wall -g
6 ALL:a.out
7
8 $(obj):%.o:%.c
9     gcc -c $< -o $@ $(myArgs)
10
11 a.out: $(obj)
12     gcc $^ -o $@ $(myArgs)
13
14 clean:
15     -rm -rf $(obj) a.out
16
17 .PHONY: clean ALL
```

执行 makefile, 如下:

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
add.c  clean  div1.c  hello  hello.c  m2  m3  makefile  mul.c  mymake  sub.c
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c div1.c -o div1.o -Wall -g
gcc -c mul.c -o mul.o -Wall -g
gcc -c hello.c -o hello.o -Wall -g
gcc -c add.c -o add.o -Wall -g
gcc -c sub.c -o sub.o -Wall -g
gcc div1.o mul.o hello.o add.o sub.o -o a.out -Wall -g
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
36 * 12 = 432
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

46P-习题和作业

源码 `add.c`, `sub.c` 这些在 `src` 目录下, `.o` 文件要放在 `obj` 目录下, 头文件 `head.h` 在 `inc` 目录下。如下所示:

```
zhcode@ubuntu:~/Code/Mydir/make$ ls
clean  hello  inc  m2  m3  makefile  mymake  obj  src
zhcode@ubuntu:~/Code/Mydir/make$
```

首先，将 `hello.c` 中的头文件单独拿出来如下：

The screenshot shows a code editor window with a menu bar at the top containing "Terminal", "File", "Edit", "View", "Search", "Terminal", and "Help". The main area displays two source files side-by-side.

The left file, named `head.h`, contains the following C header code:

```
1 #ifndef _HEAD_H_
2 #define _HEAD_H_
3
4 #include<stdio.h>
5
6 int add(int , int);
7 int sub(int , int);
8 int div1(int , int);
9 int mul(int, int);
10
11 #endif
```

The right file, named `hello.c`, contains the following C program code:

```
1 #include "head.h"
2
3 int main(int argc, char *argv[]){
4     int a = 36, b = 12;
5
6     printf("%d + %d = %d\n", a, b, add(a, b));
7     printf("%d - %d = %d\n", a, b, sub(a, b));
8     printf("%d / %d = %d\n", a, b, div1(a, b));
9     printf("%d * %d = %d\n", a, b, mul(a, b));
10
11     printf("my wife is megumin!!!\n");
12
13     return 0;
14 }
```

修改 makefile 如下，主要是注意%的匹配理解，只匹配文件名，目录位置要手动添加

```
1 src = $(wildcard ./src/*.c) # ./src/add.c ./src/sub.c ...
2 obj = $(patsubst ./src/%.c, ./obj/%.o, $(src))
3
4 inc_path = ./inc
5
6 myArgs = -Wall -g
7
8 ALL:a.out
9
10 $(obj):./obj/%.o:./src/%.c
11     gcc -c $< -o $@ $(myArgs) -I $(inc_path)
12
13 a.out: $(obj)
14     gcc $^ -o $@ $(myArgs)
15
16 clean:
17     -rm -rf $(obj) a.out
18
19 .PHONY: clean ALL
```

执行一波，如下：

```
zhcode@ubuntu:~/Code/Mydir/make$ make
gcc -c src/div1.c -o obj/div1.o -Wall -g -I ./inc
gcc -c src/mul.c -o obj/mul.o -Wall -g -I ./inc
gcc -c src/hello.c -o obj/hello.o -Wall -g -I ./inc
gcc -c src/add.c -o obj/add.o -Wall -g -I ./inc
gcc -c src/sub.c -o obj/sub.o -Wall -g -I ./inc
gcc obj/div1.o obj/mul.o obj/hello.o obj/add.o obj/sub.o -o a.out -Wall -g
zhcode@ubuntu:~/Code/Mydir/make$ ls
a.out clean hello inc m2 m3 makefile mymake obj src
zhcode@ubuntu:~/Code/Mydir/make$ ./a.out
36 + 12 = 49
36 - 12 = 135
36 / 12 = 3
36 * 12 = 432
my wife is megumin!!!
zhcode@ubuntu:~/Code/Mydir/make$
```

可以说是非常强势了

调用 clean 删除文件，直接用就行

```
zhcode@ubuntu:~/Code/Mydir/make$ make clean
rm -rf ./obj/div1.o ./obj/mul.o ./obj/hello.o ./obj/add.o ./obj/sub.o a.out
zhcode@ubuntu:~/Code/Mydir/make$
```

如果 makefile 的名字变化一下，比如，叫 m6

用 m6 执行 makefile, make -f m6

用 m6 执行 clean make -f m6 clean

47P-系统编程阶段说在前面的话

系统调用
库调用

内核提供的函数
程序库中的函数

48P-open 函数

manpage 第二卷，open 函数如下，有两个版本的

SYNOPSIS

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

返回一个文件描述符，理解为整数，出错返回-1

pathname 文件路径

flags 权限控制，只读，只写，读写。 O_RDONLY, O_WRONLY, O_RDWR

第二个 open

多了一个 mode 参数，用来指定文件的权限，数字设定法

文件权限 = mode & ~umask

open 常见错误：

1. 打开文件不存在
2. 以写方式打开只读文件（权限问题）
3. 以只写方式打开目录
- 4.

当 open 出错时，程序会自动设置 errno，可以通过 strerror(errno) 来查看报错数字的含义
以打开不存在文件为例：

```
1 #include <unistd.h>
2 #include <fcntl.h>
3 #include <stdio.h>
4 #include <errno.h>
5 #include <string.h>
6
7 int main(int argc, char *argv[]){
8     int fd;
9     fd = open("./wife.c", O_RDONLY);
10
11     printf("fd = %d, errno = %d:%s\n", fd, errno, strerror(errno));
12
13     close(fd);
14
15     return 0;
16 }
```

执行该代码，结果如下：

```
zhcode@ubuntu:~/Code/Mydir/open$ ./open
fd = -1, errno = 2:No such file or directory
zhcode@ubuntu:~/Code/Mydir/open$
```

open 函数：

```
int open(char *pathname, int flags)    #include <unistd.h>
```


参数:

pathname: 欲打开的文件路径名

flags: 文件打开方式: #include <fcntl.h>

O_RDONLY|O_WRONLY|O_RDWR O_CREAT|O_APPEND|O_TRUNC|O_EXCL|O_NONBLOCK

返回值:

成功: 打开文件所得到的文件描述符 (整数)

失败: -1, 设置 errno

int open(char *pathname, int flags, mode_t mode) 123 775

参数:

pathname: 欲打开的文件路径名

flags: 文件打开方式: O_RDONLY|O_WRONLY|O_RDWR
O_CREAT|O_APPEND|O_TRUNC|O_EXCL|O_NONBLOCK

mode: 参数 3 使用的前提, 参 2 指定了 O_CREAT。取值 8 进制数, 用来描述文件的访问权限。
rwx 0664

创建文件最终权限 = mode & ~umask

返回值:

成功: 打开文件所得到的文件描述符 (整数)

失败: -1, 设置 errno

close 函数:

int close(int fd);

49P-总结

gdb 调试工具： 大前提：程序是你自己写的。 ---逻辑错误

基础指令：

-g: 使用该参数编译可以执行文件，得到调试表。

gdb ./a.out

list: list 1 列出源码。根据源码指定 行号设置断点。

b: b 20 在 20 行位置设置断点。

run/r: 运行程序

n/next: 下一条指令（会越过函数）

s/step: 下一条指令（会进入函数）

p/print: p i 查看变量的值。

continue: 继续执行断点后续指令。

finish: 结束当前函数调用。

quit: 退出 gdb 当前调试。

其他指令：

run: 使用 run 查找段错误出现位置。

set args: 设置 main 函数命令行参数 （在 start、run 之前）

run 字符串1 字符串2 ...: 设置 main 函数命令行参数

info b: 查看断点信息表

b 20 if i = 5: 设置条件断点。

ptype: 查看变量类型。

bt: 列出当前程序正存活着的栈帧。

frame: 根据栈帧编号，切换栈帧。

display: 设置跟踪变量

`undisplay`: 取消设置跟踪变量。 使用跟踪变量的编号。

`makefile`: 管理项目。

命名: `makefile` `Makefile` --- `make` 命令

1 个规则:

目标: 依赖条件
(一个 `tab` 缩进) 命令

1. 目标的时间必须晚于依赖条件的时间, 否则, 更新目标
2. 依赖条件如果不存在, 找寻新的规则去产生依赖条件。

`ALL`: 指定 `makefile` 的终极目标。

2 个函数:

`src = $(wildcard ./*.c)`: 匹配当前工作目录下的所有 `.c` 文件。将文件名组成列表, 赋值给变量 `src`。 `src = add.c sub.c div1.c`

`obj = $(patsubst %.c, %.o, $(src))`: 将参数 3 中, 包含参数 1 的部分, 替换为参数 2。
`obj = add.o sub.o div1.o`

`clean`: (没有依赖)

`-rm -rf $(obj) a.out` “-”: 作用是, 删除不存在文件时, 不报错。顺序执行结束。

3 个自动变量:

`$@`: 在规则的命令中, 表示规则中的目标。

`$^`: 在规则的命令中, 表示所有依赖条件。

`$<`: 在规则的命令中, 表示第一个依赖条件。如果将该变量应用在模式规则中, 它可将依赖条件列表中的依赖依次取出, 套用模式规则。

模式规则:

`%.o:%.c`
`gcc -c $< -o %@`

静态模式规则:

`$(obj):%.o:%.c`

```
gcc -c $< -o %@
```

伪目标:

```
.PHONY: clean ALL
```

参数:

-n: 模拟执行 make、make clean 命令。

-f: 指定文件执行 make 命令。 xxxx.mk

作业: 编写一个 makefile 可以将其所在目录下的所有独立 .c 文件编译生成同名可执行文件。

open 函数:

50P-复习

51P-makefile 作业

将当前目录下所有.c 文件编译为可执行程序

```
1 src = $(wildcard *.c)
2 target = $(patsubst %.c, %, $(src))
3
4 ALL:$(target)
5
6 %:%.c
7     gcc $< -o $@
8
9 clean:
10     -rm -rf $(target)
11
12 .PHONY: clean ALL
```

测试程序:

```
1 #include<stdio.h>
2
3 int main(){
4
5     printf("megumin is my wife!!!\n");
6
7     return 0;
8 }
```

测试:

```
zhcode@ubuntu:~/Code/sysp$ make
gcc hello.c -o hello
zhcode@ubuntu:~/Code/sysp$ ./hello
megumin is my wife!!!
zhcode@ubuntu:~/Code/sysp$
```

52P-read 和 write 实现 cp

read 函数:

```
ssize_t read(int fd, void *buf, size_t count);
```

参数:

fd: 文件描述符

buf: 存数据的缓冲区

count: 缓冲区大小

返回值:

0: 读到文件末尾。

成功; > 0 读到的字节数。

失败: -1, 设置 errno

-1: 并且 errno = EAGAIN 或 EWOULDBLOCK, 说明不是 read 失败, 而是 read 在以非阻塞方式读一个设备文件 (网络文件), 并且文件无数据。

write 函数:

```
ssize_t write(int fd, const void *buf, size_t count);
```

参数:

fd: 文件描述符

buf: 待写出数据的缓冲区

count: 数据大小

返回值:

成功; 写入的字节数。

失败: -1, 设置 errno

用 read 和 write 实现一个 copy 函数:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     char buf[1024];
10
11     int n = 0;
12
13     int fd1 = open(argv[1], O_RDONLY);
14     int fd2 = open(argv[2], O_RDWR|O_CREAT|O_TRUNC, 0664);
15
16     while((n = read(fd1, buf, 1024)) != 0){
17         write(fd2, buf, n);
18     }
19
20     close(fd1);
21     close(fd2);
22
23     return 0;
24 }

```

编译一波

```

zhcode@ubuntu:~/Code/sysp$ make mycp
gcc mycp.c -o mycp
zhcode@ubuntu:~/Code/sysp$ █

```

复制文件一波:

```

zhcode@ubuntu:~/Code/sysp$ ls
hello hello.c makefile mycp mycp.c
zhcode@ubuntu:~/Code/sysp$ ./mycp hello.c hello2.c
zhcode@ubuntu:~/Code/sysp$ ls
hello hello2.c hello.c makefile mycp mycp.c
zhcode@ubuntu:~/Code/sysp$ ll
total 48
drwxrwxr-x 2 zhcode zhcode 4096 May 23 12:48 ./
drwxrwxr-x 8 zhcode zhcode 4096 May 23 11:09 ../
-rwxrwxr-x 1 zhcode zhcode 8600 May 23 11:16 hello*
-rw-rw-r-- 1 zhcode zhcode 83 May 23 12:48 hello2.c
-rw-rw-r-- 1 zhcode zhcode 83 May 23 11:16 hello.c
-rw-rw-r-- 1 zhcode zhcode 140 May 23 11:14 makefile
-rwxrwxr-x 1 zhcode zhcode 8816 May 23 12:47 mycp*
-rw-rw-r-- 1 zhcode zhcode 385 May 23 12:47 mycp.c
zhcode@ubuntu:~/Code/sysp$

```

可以在复制函数里加入错误检测:


```
if(fd1 == -1) {  
    perror( "open argv[1] error" );  
    exit(1);  
}
```

错误处理函数：与 `errno` 相关。

```
printf("xxx error: %d\n", errno);
```

```
char *strerror(int errnum);
```

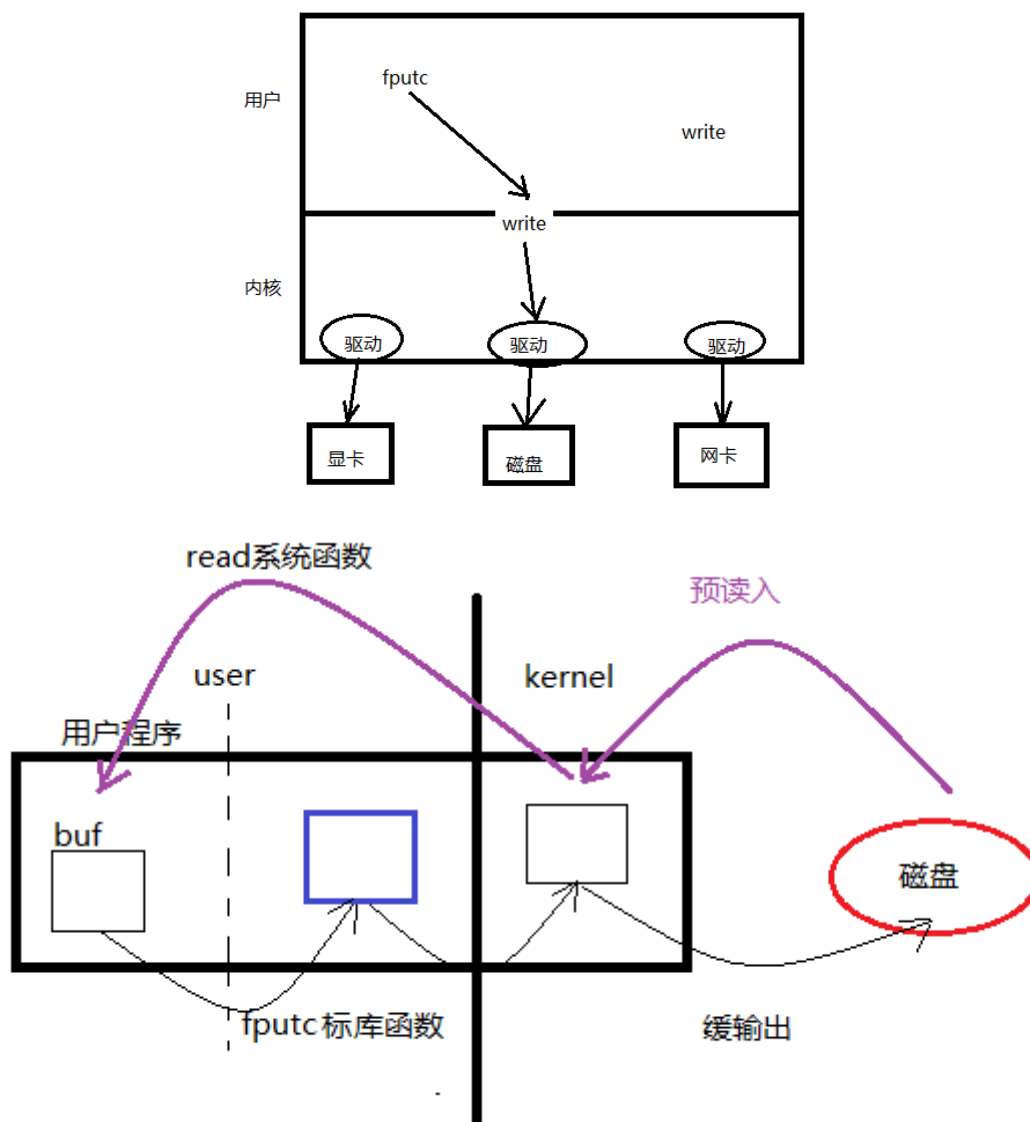
```
printf("xxx error: %s\n", strerror(errno));
```

```
void perror(const char *s);
```

```
perror("open error");
```

用 `read/write` 实现的 `copy` 和 `fgetc/fputc` 实现的 `copy` 对比：

53P-系统调用和库函数比较—预读入缓输出



下面写两个文件拷贝函数，一个用 read/write 实现，一个用 fputc/fgetc 实现，比较速度
首先是 fputc/fgetc

```
8 int main(void){
9     FILE *fp, *fp_out;
10    int n = 0;
11
12    fp = fopen("hello.c", "r");
13    if(fp == NULL){
14        perror("fopen error");
15        exit(1);
16    }
17
18    fp_out = fopen("hello.cp", "w");
19    if(fp_out == NULL){
20        perror("fopen error");
21        exit(1);
22    }
23
24    while((n = fgetc(fp)) != EOF){
25        fputc(n, fp_out);
26    }
27
28    fclose(fp);
29    fclose(fp_out);
30
31    return 0;
32 }
```

编译一波

```
zhcode@ubuntu:~/Code/sysp$ make getcmp
gcc getcmp.c -o getcmp
zhcode@ubuntu:~/Code/sysp$
```

运行一波

```
zhcode@ubuntu:~/Code/sysp$ ./getcmp
zhcode@ubuntu:~/Code/sysp$ ls
getcmp  getcmp.c  hello  hello2.c  hello.c  hello.cp  makefile  mycp  mycp.c
zhcode@ubuntu:~/Code/sysp$
```

很快就拷贝完成了。

下面修改 read 那边的缓冲区，一次拷贝一个字符。

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     char buf[1];
10
11     int n = 0;
12
13     int fd1 = open(argv[1], O_RDONLY);
14     int fd2 = open(argv[2], O_RDWR|O_CREAT|O_TRUNC, 0664);
15
16     while((n = read(fd1, buf, 1)) != 0){
17         write(fd2, buf, n);
18     }
19
20     close(fd1);
21     close(fd2);
22
23     return 0;
24 }

```

运行，如图：

```

zhcode@ubuntu:~/Code/sysp$ ls
getcmp getcmp.c hello hello2.c hello3.c hello.c hello.cp makefile mycp.c
zhcode@ubuntu:~/Code/sysp$ make mycp
gcc mycp.c -o mycp
zhcode@ubuntu:~/Code/sysp$ ls
getcmp getcmp.c hello hello2.c hello3.c hello.c hello.cp makefile mycp mycp.c
zhcode@ubuntu:~/Code/sysp$ ./mycp hello.c hello4.c
zhcode@ubuntu:~/Code/sysp$ ls
getcmp getcmp.c hello hello2.c hello3.c hello4.c hello.c hello.cp makefile mycp mycp.c
zhcode@ubuntu:~/Code/sysp$

```

速度缓慢。

原因分析：

read/write 这块，每次写一个字节，会疯狂进行内核态和用户态的切换，所以非常耗时。

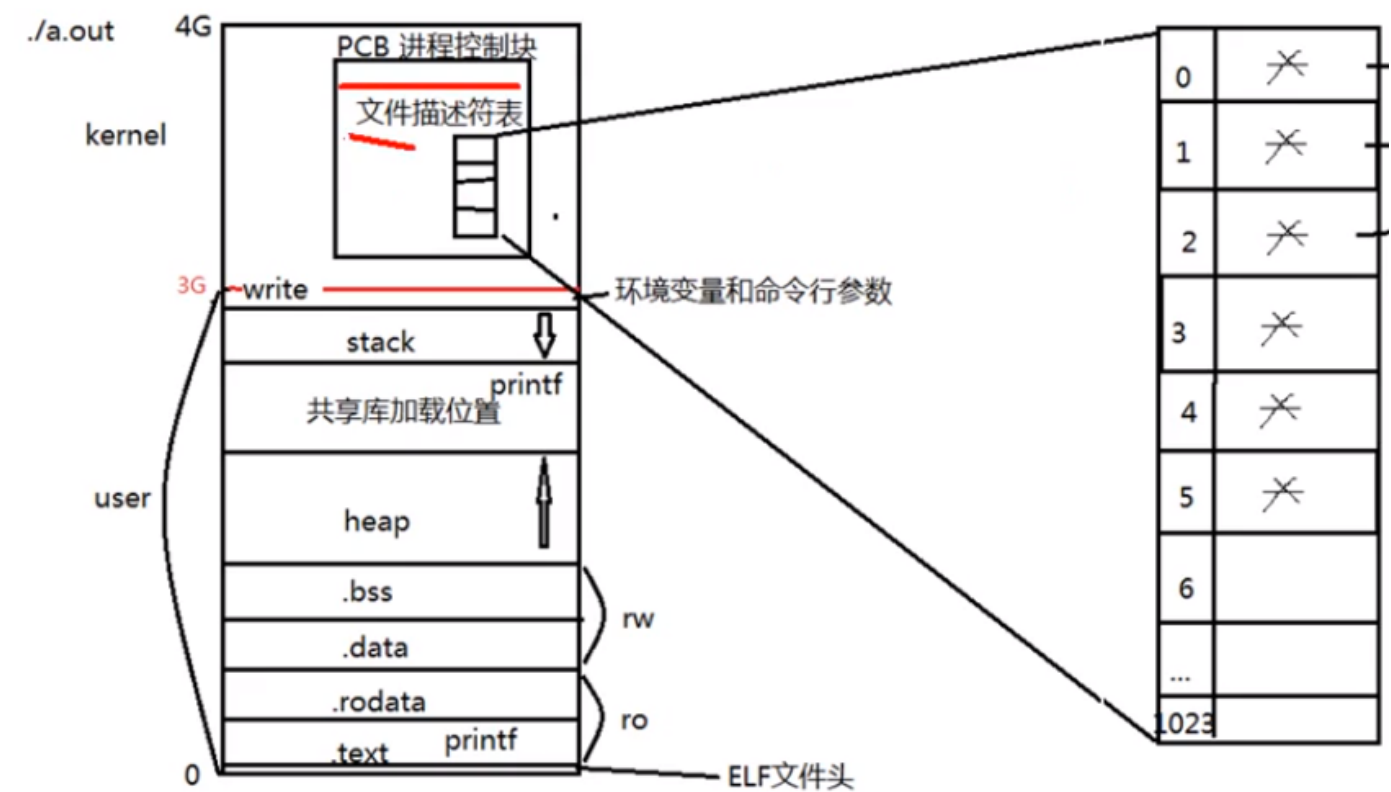
fgetc/fputc，有个缓冲区，4096，所以它并不是一个字节一个字节地写，内核和用户切换就比较少

预读入，缓输出机制。

所以系统函数并不是一定比库函数牛逼，能使用库函数的地方就使用库函数。

标准 IO 函数自带用户缓冲区，系统调用无用户级缓冲。系统缓冲区是都有的。

54P-文件描述符



文件描述符是指向一个文件结构体的指针
PCB 进程控制块：本质 结构体。

成员：文件描述符表。

文件描述符：0/1/2/3/4..... /1023 表中可用的最小的。

0 - STDIN_FILENO

1 - STDOUT_FILENO

2 - STDERR_FILENO

55P-阻塞和非阻塞

阻塞、非阻塞： 是设备文件、网络文件的属性。

产生阻塞的场景。 读设备文件。读网络文件。（读常规文件无阻塞概念。）

/dev/tty -- 终端文件。

`open("/dev/tty", O_RDWR|O_NONBLOCK)` --- 设置 /dev/tty 非阻塞状态。（默认为阻塞状态）

一个例子，从标准输入读，写到标准输出：

```
1 #include <unistd.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 int main(){
6     char buf[10];
7     int n;
8
9     n = read(STDIN_FILENO, buf, 10);
10    if(n < 0){
11        perror("read STDIN_FILENO");
12        exit(1);
13    }
14    write(STDOUT_FILENO, buf, n);
15
16    return 0;
17 }
```

执行程序，就会发现程序在阻塞等待输入

```
zhcode@ubuntu:~/Code/block$ ./block_readtty
hello
hello
zhcode@ubuntu:~/Code/block$
```

下面是一段更改非阻塞读取终端的代码：

```
1. #include <unistd.h>
2. #include <fcntl.h>
3. #include <stdlib.h>
4. #include <stdio.h>
5. #include <errno.h>
6. #include <string.h>
7.
8. #define MSG_TRY "try again\n"
9. #define MSG_TIMEOUT "time out\n"
10.
11. int main(void)
```

```

12. {
13.     char buf[10];
14.     int fd, n, i;
15.
16.     fd = open("/dev/tty", O_RDONLY|O_NONBLOCK);
17.     if(fd < 0){
18.         perror("open /dev/tty");
19.         exit(1);
20.     }
21.     printf("open /dev/tty ok... %d\n", fd);
22.
23.     for (i = 0; i < 5; i++){
24.         n = read(fd, buf, 10);
25.         if (n > 0) {                //说明读到了东西
26.             break;
27.         }
28.         if (errno != EAGAIN) {        //EWOULDBLOCK
29.             perror("read /dev/tty");
30.             exit(1);
31.         } else {
32.             write(STDOUT_FILENO, MSG_TRY, strlen(MSG_TRY));
33.             sleep(2);
34.         }
35.     }
36.
37.     if (i == 5) {
38.         write(STDOUT_FILENO, MSG_TIMEOUT, strlen(MSG_TIMEOUT));
39.     } else {
40.         write(STDOUT_FILENO, buf, n);
41.     }
42.
43.     close(fd);
44.
45.     return 0;
46. }

```

执行，如图所示：

```

zhcode@ubuntu:~/Code/block$ ls
block_readtty  block_readtty.c  makefile  nonblock_timeout.c
zhcode@ubuntu:~/Code/block$ make nonblock_timeout
gcc nonblock_timeout.c -o nonblock_timeout
zhcode@ubuntu:~/Code/block$ ./nonblock_timeout
open /dev/tty ok... 3
try again
try again
try again
try again
try again
time out
zhcode@ubuntu:~/Code/block$

```

56-fcntl 改文件属性

fcntl 用来改变一个【已经打开】的文件的 访问控制属性
重点掌握两个参数的使用， F_GETFL, F_SETFL

fcntl:

```
int (int fd, int cmd, ...)
```

fd 文件描述符

cmd 命令，决定了后续参数个数

```
int flgs = fcntl(fd, F_GETFL);
```

```
flgs |= O_NONBLOCK
```

```
fcntl(fd, F_SETFL, flgs);
```

获取文件状态： F_GETFL

设置文件状态： F_SETFL

终端文件默认是阻塞读的，这里用 fcntl 将其更改为非阻塞读

```
1. #include <unistd.h>
2. #include <fcntl.h>
3. #include <errno.h>
4. #include <stdio.h>
5. #include <stdlib.h>
6. #include <string.h>
7.
8. #define MSG_TRY "try again\n"
9.
10. int main(void)
11. {
12.     char buf[10];
13.     int flags, n;
14.
15.     flags = fcntl(STDIN_FILENO, F_GETFL); //获取 stdin 属性信息
16.     if(flags == -1){
17.         perror("fcntl error");
18.         exit(1);
19.     }
20.     flags |= O_NONBLOCK;
21.     int ret = fcntl(STDIN_FILENO, F_SETFL, flags);
22.     if(ret == -1){
23.         perror("fcntl error");
24.         exit(1);
25.     }
26.
```



```
27. tryagain:
28.     n = read(STDIN_FILENO, buf, 10);
29.     if(n < 0){
30.         if(errno != EAGAIN){
31.             perror("read /dev/tty");
32.             exit(1);
33.         }
34.         sleep(3);
35.         write(STDOUT_FILENO, MSG_TRY, strlen(MSG_TRY));
36.         goto tryagain;
37.     }
38.     write(STDOUT_FILENO, buf, n);
39.
40.     return 0;
41. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/block$ make fcntl
gcc fcntl.c -o fcntl
zhcode@ubuntu:~/Code/block$ ./fcntl
try again
try again
try again
try again
try again
try again
try again
try again
try again
try again
meguminty again
try again
try again
try again
^C
zhcode@ubuntu:~/Code/block$
```

可以看到，是非阻塞读取。

57P-午后回顾

58P-lseek 函数

lseek 函数：

```
off_t lseek(int fd, off_t offset, int whence);
```

参数：

fd：文件描述符

offset： 偏移量，就是将读写指针从 whence 指定位置向后偏移 offset 个单位

whence： 起始偏移位置： SEEK_SET/SEEK_CUR/SEEK_END

返回值：

成功：较起始位置偏移量

失败：-1 errno

应用场景：

1. 文件的“读”、“写”使用同一偏移位置。
2. 使用 lseek 获取文件大小
3. 使用 lseek 拓展文件大小：要想使文件大小真正拓展，必须引起 IO 操作。

使用 truncate 函数，直接拓展文件。 `int ret = truncate("dict.cp", 250);`

lseek 示例，写一个句子到空白文件，完事调整光标位置，读取刚才写那个文件。
这个示例中，如果不调整光标位置，是读取不到内容的，因为读写指针在内容的末尾
代码如下：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <unistd.h>
4. #include <string.h>
5. #include <fcntl.h>
6.
7. int main(void)
8. {
9.     int fd, n;
10.    char msg[] = "It's a test for lseek\n";
11.    char ch;
12.
13.    fd = open("lseek.txt", O_RDWR|O_CREAT, 0644);
14.    if(fd < 0){
15.        perror("open lseek.txt error");
16.        exit(1);
```

```

17.     }
18.
19.     write(fd, msg, strlen(msg));    //使用 fd 对打开的文件进行写操作，问价读写位置位于文件结尾处。
20.
21.     lseek(fd, 0, SEEK_SET);         //修改文件读写指针位置，位于文件开头。 注释该行会怎样呢？
22.
23.     while((n = read(fd, &ch, 1)){
24.         if(n < 0){
25.             perror("read error");
26.             exit(1);
27.         }
28.         write(STDOUT_FILENO, &ch, n);    //将文件内容按字节读出，写出到屏幕
29.     }
30.
31.     close(fd);
32.
33.     return 0;
34. }

```

编译运行，结果如图：

```

zhcode@ubuntu:~/Code/block$ make lseek
gcc lseek.c -o lseek
zhcode@ubuntu:~/Code/block$ ./lseek
It's a test for lseek
zhcode@ubuntu:~/Code/block$

```

下面这个代码用 lseek 的偏移来读取文件大小：

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <fcntl.h>
6
7 int main(int argc, char *argv[]){
8     int fd = open(argv[1], O_RDWR);
9     if(fd == -1){
10         perror("open error");
11         exit(1);
12     }
13
14     int lenth = lseek(fd, 0, SEEK_END);
15     printf("file size: %d\n", lenth);
16
17     close(fd);
18     return 0;
19 }
20

```

编译执行，结果如下：

```
zhcode@ubuntu:~/Code/block$ make lseek_test
gcc lseek_test.c -o lseek_test
zhcode@ubuntu:~/Code/block$ ./lseek_test ./fcntl.c
file size: 698
zhcode@ubuntu:~/Code/block$ ll fcntl.c
-rw-rw-r-- 1 zhcode zhcode 698 May 23 17:06 fcntl.c
zhcode@ubuntu:~/Code/block$
```

是没有问题的。

下面使用 lseek 将 fcntl.c 原本的 698 字节填充为 800 字节，差值 102 字节：
比起读取大小的代码，就改了个偏移量

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <fcntl.h>
6
7 int main(int argc, char *argv[]){
8     int fd = open(argv[1], O_RDWR);
9     if(fd == -1){
10         perror("open error");
11         exit(1);
12     }
13
14     int lenth = lseek(fd, 102, SEEK_END);
15     printf("file size: %d\n", lenth);
16
17     close(fd);
18     return 0;
19 }
20
```

编译执行，如下：

```
zhcode@ubuntu:~/Code/block$ make lseek_test
gcc lseek_test.c -o lseek_test
zhcode@ubuntu:~/Code/block$ ./lseek_test fcntl.c
file size: 800
zhcode@ubuntu:~/Code/block$
```

下面再用 ls 命令查看 fcntl.c，问题出来了：

```
zhcode@ubuntu:~/Code/block$ ls -l fcntl.c
-rw-rw-r-- 1 zhcode zhcode 698 May 23 17:06 fcntl.c
zhcode@ubuntu:~/Code/block$
```

它的大小不是 800。

原因是，要使文件大小真正拓展，必须引起 IO 操作。

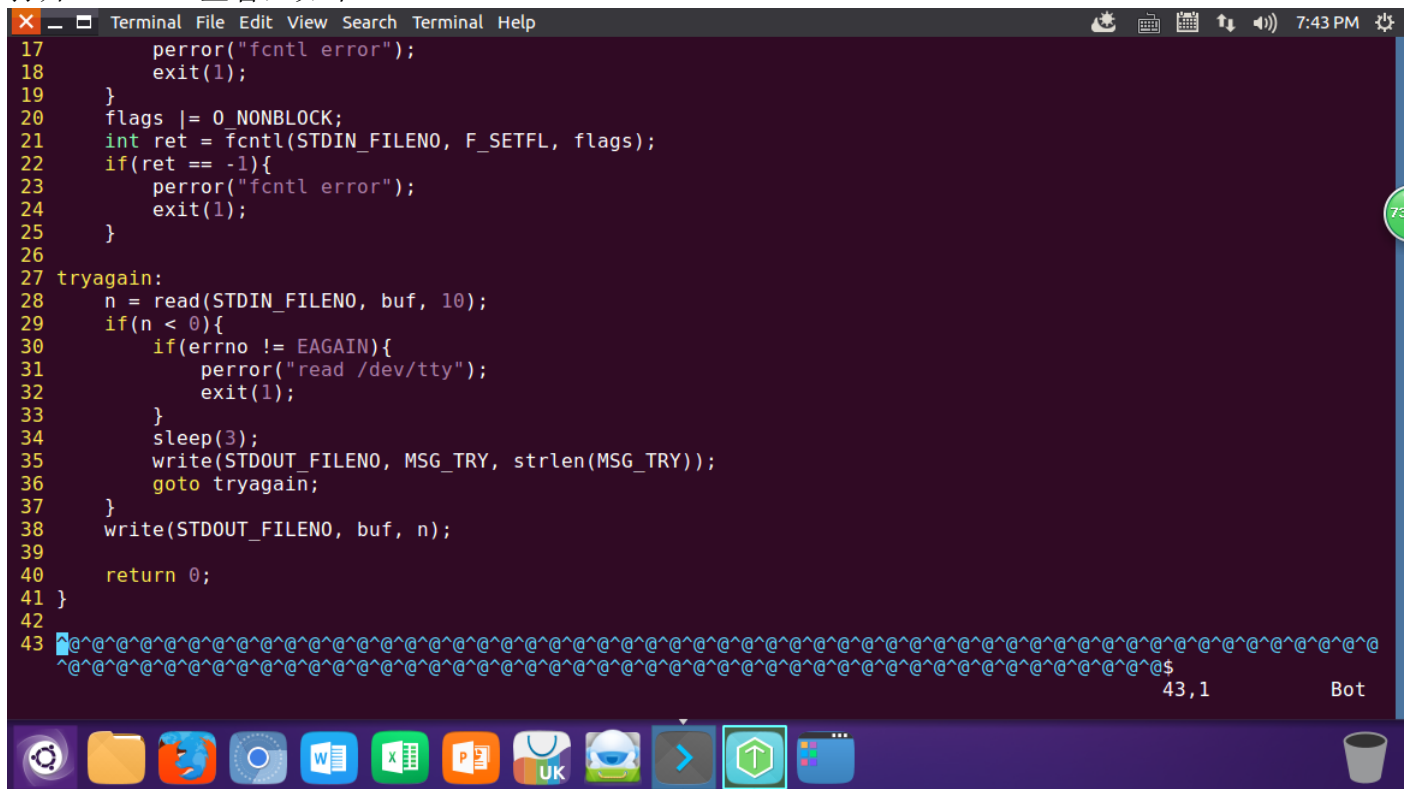
修改后的扩展文件代码如下：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <fcntl.h>
6
7 int main(int argc, char *argv[]){
8     int fd = open(argv[1], O_RDWR);
9     if(fd == -1){
10         perror("open error");
11         exit(1);
12     }
13
14     int lenth = lseek(fd, 101, SEEK_END);
15     printf("file size: %d\n", lenth);
16
17     write(fd, "$", 1);
18
19     close(fd);
20     return 0;
21 }
22
```

编译运行，如下：

```
zhcode@ubuntu:~/Code/block$ make lseek_test
gcc lseek_test.c -o lseek_test
zhcode@ubuntu:~/Code/block$ ./lseek_test fcntl.c
file size: 799
zhcode@ubuntu:~/Code/block$ ls -l fcntl.c
-rw-rw-r-- 1 zhcode zhcode 800 May 23 19:41 fcntl.c
zhcode@ubuntu:~/Code/block$
```

打开 fcntl.c 查看，如下：

A terminal window with a dark background and light-colored text. The window title is "Terminal" and it has a menu bar with "File", "Edit", "View", "Search", "Terminal", and "Help". The terminal shows the contents of the file fcntl.c, starting from line 17. The code includes error handling for file opening and reading, and a loop that reads data from the file and writes it to the standard output. The terminal also shows the file size of fcntl.c as 800 bytes. The bottom of the terminal shows a taskbar with various application icons and a system tray with a clock showing 7:43 PM and a battery icon.

```
17     perror("fcntl error");
18     exit(1);
19 }
20 flags |= O_NONBLOCK;
21 int ret = fcntl(STDIN_FILENO, F_SETFL, flags);
22 if(ret == -1){
23     perror("fcntl error");
24     exit(1);
25 }
26
27 tryagain:
28     n = read(STDIN_FILENO, buf, 10);
29     if(n < 0){
30         if(errno != EAGAIN){
31             perror("read /dev/tty");
32             exit(1);
33         }
34         sleep(3);
35         write(STDOUT_FILENO, MSG_TRY, strlen(MSG_TRY));
36         goto tryagain;
37     }
38     write(STDOUT_FILENO, buf, n);
39
40     return 0;
41 }
42
43
```

简单小结一下：

对于写文件再读取那个例子，由于文件写完之后未关闭，读写指针在文件末尾，所以不调节指针，直

接读取不到内容。

lseek 读取的文件大小总是相对文件头部而言。

用 lseek 读取文件大小实际用的是读写指针初末位置的偏移差，一个新开文件，读写指针初位置都在文件开头。如果用这个来扩展文件大小，必须引起 IO 才行，于是就至少要写入一个字符。上面代码出现 lseek 返回 799，ls 查看为 800 的原因是，lseek 读取到偏移差的时候，还没有写入最后的 '\$' 符号。

末尾那一大堆 ^@，是文件空洞，如果自己写进去的也想保持队形，就写入 "\0"。

拓展文件直接使用 truncate，简单粗暴：

使用 truncate 函数，直接拓展文件。 `int ret = truncate("dict.cp", 250);`

59P-传入传出参数

传入参数：

1. 指针作为函数参数。
2. 同常有 `const` 关键字修饰。
3. 指针指向有效区域， 在函数内部做读操作。

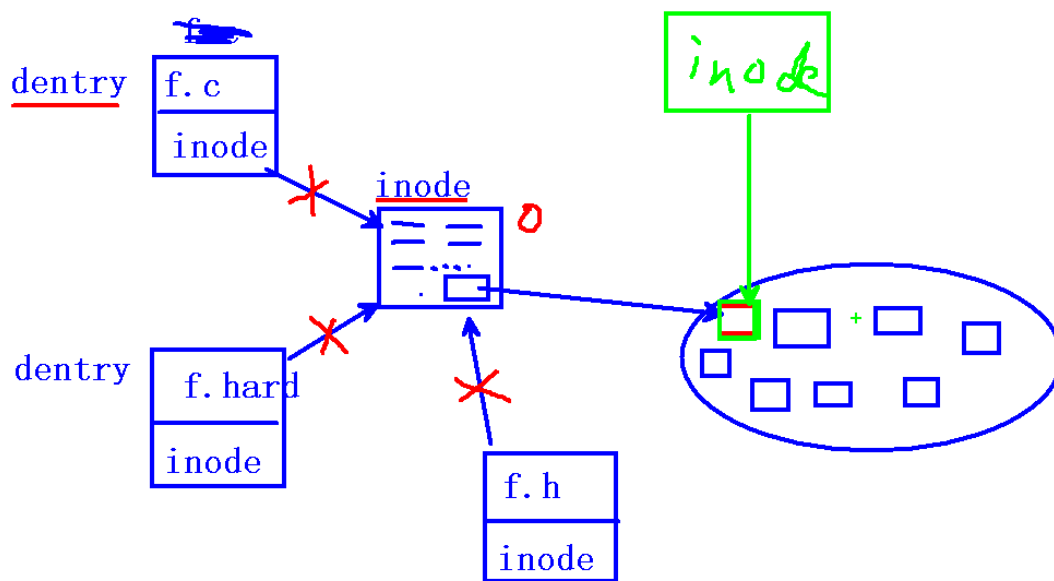
传出参数：

1. 指针作为函数参数。
2. 在函数调用之前，指针指向的空间可以无意义，但必须有效。
3. 在函数内部，做写操作。
4. 函数调用结束后，充当函数返回值。

传入传出参数：

1. 指针作为函数参数。
2. 在函数调用之前，指针指向的空间有实际意义。
3. 在函数内部，先做读操作，后做写操作。
4. 函数调用结束后，充当函数返回值。

60P- 目录项和 inode



一个文件主要由两部分组成，dentry(目录项)和 inode

inode 本质是结构体，存储文件的属性信息，如：权限、类型、大小、时间、用户、盘快位置…也叫做文件属性管理结构，大多数的 inode 都存储在磁盘上。

少量常用、近期使用的 inode 会被缓存到内存中。

所谓的删除文件，就是删除 inode，但是数据其实还是在硬盘上，以后会覆盖掉。

61P-stat 函数

获取文件属性，（从 inode 结构体中获取）

stat/lstat 函数：

```
int stat(const char *path, struct stat *buf);
```

参数：

path: 文件路径

buf:（传出参数） 存放文件属性，inode 结构体指针。

返回值：

成功: 0

失败: -1 errno

获取文件大小: buf.st_size

获取文件类型: buf.st_mode

获取文件权限: buf.st_mode

符号穿透: stat 会。lstat 不会。

下面这个例子是获取文件大小的正规军解法，用 stat：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <pthread.h>
6 #include <sys/stat.h>
7
8 int main(int argc, char *argv[]){
9     struct stat sbuf;
10
11     int ret = stat(argv[1], &sbuf);
12     if(ret == -1){
13         perror("stat error");
14         exit(1);
15     }
16
17     printf("file size: %ld\n", sbuf.st_size);
18
19     return 0;
20 }
```

编译运行，并查看 mystat.c 文件的大小，如下：

```
zhcode@ubuntu:~/Code/stat$ make mystat
gcc mystat.c -o mystat
zhcode@ubuntu:~/Code/stat$ ./mystat mystat.c
file size: 322
zhcode@ubuntu:~/Code/stat$ ls -l mystat.c
-rw-rw-r-- 1 zhcode zhcode 322 May 23 21:02 mystat.c
zhcode@ubuntu:~/Code/stat$
```

62P-lstat 和 stat

用 stat 查看文件类型，代码如下：

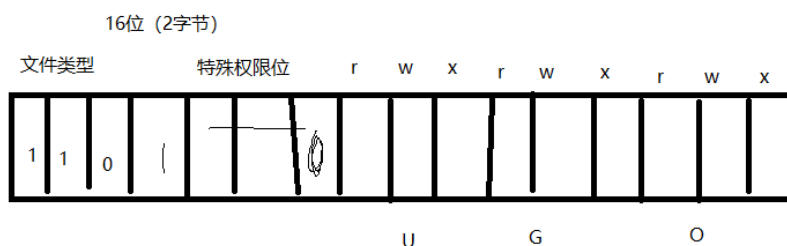
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <pthread.h>
6 #include <sys/stat.h>
7
8 int main(int argc, char *argv[]){
9     struct stat sbuf;
10
11     int ret = stat(argv[1], &sbuf);
12     if(ret == -1){
13         perror("stat error");
14         exit(1);
15     }
16
17     if(S_ISREG(sbuf.st_mode)){
18         printf("It's a regular\n");
19     }
20     else if(S_ISDIR(sbuf.st_mode)){
21         printf("It's a dir\n");
22     }
23     else{
24         printf("It's a unknown file\n");
25     }
26
27     return 0;
28 }
```

编译运行，查看一波 mystat.c 文件：

```
zhcode@ubuntu:~/Code/stat$ make mystat2
gcc mystat2.c -o mystat2
zhcode@ubuntu:~/Code/stat$ ./mystat2 mystat.c
It's a regular
zhcode@ubuntu:~/Code/stat$
```

stat 会拿到符号链接指向那个文件或目录的属性。

不想穿透符号就用 lstat



63P-传出参数当返回值

其实就是传出参数在函数中被改变,可以用来传出。而且传出参数数量不限,比单纯的返回值更灵活。

64P-link 和 Unlink 隐式回收

硬链接数就是 dentry 数目

link 就是用来创建硬链接的

link 可以用来实现 mv 命令

函数原型:

```
int link(const char *oldpath, const char *newpath)
```

用这个来实现 mv，用 oldpath 来创建 newpath，完事儿删除 oldpath 就行。

删除一个链接 `int unlink(const char *pathname)`

unlink 是删除一个文件的目录项 dentry，使硬链接数-1

unlink 函数的特征：清除文件时，如果文件的硬链接数到 0 了，没有 dentry 对应，但该文件仍不会马上被释放，要等到所有打开文件的进程关闭该文件，系统才会挑时间将该文件释放掉。

下面用一段代码来验证 unlink 是删除 dentry:

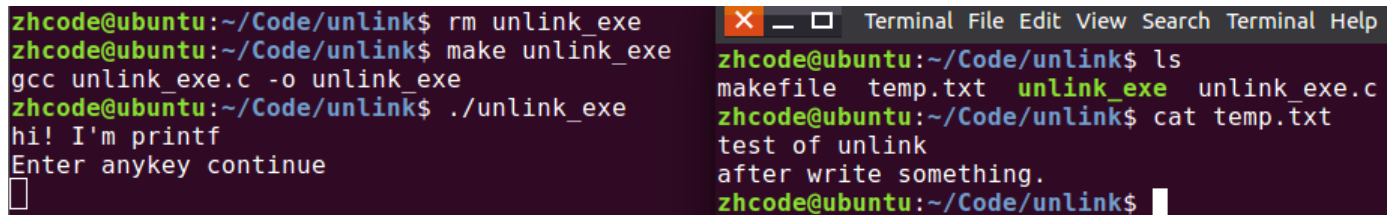
```
1.  /*
2.  *unlink 函数是删除一个 dentry
3.  */
4.  #include <unistd.h>
5.  #include <fcntl.h>
6.  #include <stdlib.h>
7.  #include <string.h>
8.  #include <stdio.h>
9.
10.
11. int main(void)
12. {
13.     int fd, ret;
14.     char *p = "test of unlink\n";
15.     char *p2 = "after write something.\n";
16.
17.     fd = open("temp.txt", O_RDWR|O_CREAT|O_TRUNC, 0644);
18.     if(fd < 0){
19.         perror("open temp error");
20.         exit(1);
21.     }
22.
23.     ret = write(fd, p, strlen(p));
24.     if (ret == -1) {
25.         perror("-----write error");
26.     }
27.
28.     printf("hi! I'm printf\n");
```

```

29.     ret = write(fd, p2, strlen(p2));
30.     if (ret == -1) {
31.         perror("-----write error");
32.     }
33.
34.     printf("Enter anykey continue\n");
35.     getchar();
36.
37.     ret = unlink("temp.txt");           //具备了被释放的条件
38.     if(ret < 0){
39.         perror("unlink error");
40.         exit(1);
41.     }
42.
43.     close(fd);
44.
45.     return 0;
46. }

```

编译程序并运行，程序阻塞，此时打开新终端查看临时文件 temp.c 如下：



```

zhcode@ubuntu:~/Code/unlink$ rm unlink_exe
zhcode@ubuntu:~/Code/unlink$ make unlink_exe
gcc unlink_exe.c -o unlink_exe
zhcode@ubuntu:~/Code/unlink$ ./unlink_exe
hi! I'm printf
Enter anykey continue

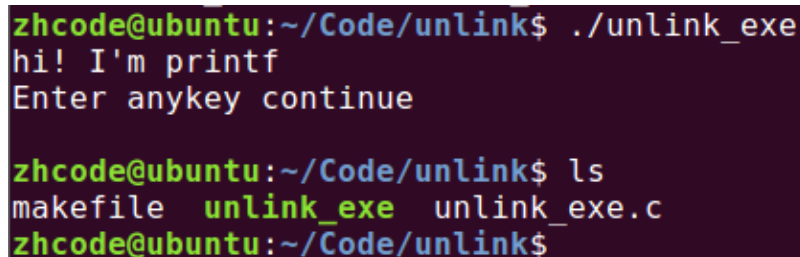
```

```

Terminal File Edit View Search Terminal Help
zhcode@ubuntu:~/Code/unlink$ ls
makefile temp.txt unlink_exe unlink_exe.c
zhcode@ubuntu:~/Code/unlink$ cat temp.txt
test of unlink
after write something.
zhcode@ubuntu:~/Code/unlink$

```

可以看到，临时文件没有被删除，这是因为当前进程没结束。
输入字符使当前进程结束后，temp.txt 就不见了



```

zhcode@ubuntu:~/Code/unlink$ ./unlink_exe
hi! I'm printf
Enter anykey continue

zhcode@ubuntu:~/Code/unlink$ ls
makefile unlink_exe unlink_exe.c
zhcode@ubuntu:~/Code/unlink$

```

下面开始搞事，在程序中加入段错误成分，段错误在 unlink 之前，由于发生段错误，程序后续删除 temp.txt 的 dentry 部分就不会再执行，temp.txt 就保留了下来，这是不科学的。

解决办法是检测 fd 有效性后，立即释放 temp.txt，由于进程未结束，虽然 temp.txt 的硬链接数已经为 0，但还不会立即释放，仍然存在，要等到程序执行完才会释放。这样就能避免程序出错导致临时文件保留下来。

因为文件创建后，硬链接数立马减为 0，即使程序异常退出，这个文件也会被清理掉。这时候的内容是写在内核空间的缓冲区。

修改后代码如下：

```

1.  /*

```

```

2.  *unlink 函数是删除一个 dentry
3.  */
4.  #include <unistd.h>
5.  #include <fcntl.h>
6.  #include <stdlib.h>
7.  #include <string.h>
8.  #include <stdio.h>
9.
10.
11. int main(void)
12. {
13.     int fd, ret;
14.     char *p = "test of unlink\n";
15.     char *p2 = "after write something.\n";
16.
17.     fd = open("temp.txt", O_RDWR|O_CREAT|O_TRUNC, 0644);
18.     if(fd < 0){
19.         perror("open temp error");
20.         exit(1);
21.     }
22.
23.     ret = unlink("temp.txt");          //具备了被释放的条件
24.     if(ret < 0){
25.         perror("unlink error");
26.         exit(1);
27.     }
28.
29.     ret = write(fd, p, strlen(p));
30.     if (ret == -1) {
31.         perror("-----write error");
32.     }
33.
34.     printf("hi! I'm printf\n");
35.     ret = write(fd, p2, strlen(p2));
36.     if (ret == -1) {
37.         perror("-----write error");
38.     }
39.
40.     printf("Enter anykey continue\n");
41.     getchar();
42.
43.     close(fd);
44.
45.     return 0;
46. }

```

隐式回收:

当进程结束运行时, 所有进程打开的文件会被关闭, 申请的内存空间会被释放。系统的这一特性

称之为隐式回收系统资源。

比如上面那个程序,要是没有在程序中关闭文件描述符,没有隐式回收的话,这个文件描述符会保留,多次出现这种情况会导致系统文件描述符耗尽。所以隐式回收会在程序结束时收回它打开的文件使用的文件描述符。

65P-文件目录 rwx 权限差异

vi 目录
会得到目录项的列表

注意：目录文件也是“文件”。其文件内容是该目录下所有子文件的目录项 dentry。可以尝试用 vim 打开一个目录。

	r	w	x
文件	文件的内容可以被查看。 cat、more、less...	内容可以被修改。 vi、> ...	可以运行产生一个进程。 <u>./</u> 文件名
目录	目录可以被浏览。 ls、tree...	创建、删除、修改文件。 mv、touch、mkdir...	可以被打开、进入。 cd

目录设置黏住位：若有 w 权限，创建不变，删除、修改只能由 root、目录所有者、文件所有者操作。

66P- 目录操作函数

目录操作函数：

```
DIR * opendir(char *name);

int closedir(DIR *dp);

struct dirent *readdir(DIR * dp);

struct dirent {

    inode

    char dname[256];
}
```

没有写目录操作，因为目录写操作就是创建文件。可以用 touch

下面用目录操作函数实现一个 ls 操作：

```
1 #include <dirent.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <stdlib.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     DIR *dp;
10    struct dirent *sdp;
11
12    dp = opendir(argv[1]);
13    if(dp == NULL){
14        perror("opendir error");
15        exit(1);
16    }
17
18    while((sdp = readdir(dp)) != NULL){
19        printf("%s\t", sdp->d_name);
20    }
21    printf("\n");
22    closedir(dp);
23
24    return 0;
25 }
```

编译执行，如下：

```
zhcode@ubuntu:~/Code/mydir$ make myls
gcc myls.c -o myls
zhcode@ubuntu:~/Code/mydir$ ./mysls ../unlink/
.      unlink_exe.c      ..      unlink_exe      makefile
zhcode@ubuntu:~/Code/mydir$
```

要隐藏这个.和..的话，在输出文件名的时候判定一下，只输出不是.和..的就行了

67P-总结

open 函数:

```
int open(char *pathname, int flags)    #include <unistd.h>
```

参数:

pathname: 欲打开的文件路径名

flags: 文件打开方式: #include <fcntl.h>

O_RDONLY|O_WRONLY|O_RDWR O_CREAT|O_APPEND|O_TRUNC|O_EXCL|O_NONBLOCK

返回值:

成功: 打开文件所得到的 文件描述符 (整数)

失败: -1, 设置 errno

```
int open(char *pathname, int flags, mode_t mode)    123 775
```

参数:

pathname: 欲打开的文件路径名

flags: 文件打开方式: O_RDONLY|O_WRONLY|O_RDWR
O_CREAT|O_APPEND|O_TRUNC|O_EXCL|O_NONBLOCK

mode: 参数 3 使用的前提, 参 2 指定了 O_CREAT。取值 8 进制数, 用来描述文件的 访问权限。 rwx 0664

创建文件最终权限 = mode & ~umask

返回值:

成功: 打开文件所得到的 文件描述符 (整数)

失败: -1, 设置 errno

close 函数:

```
int close(int fd);
```

错误处理函数: 与 errno 相关。

```
printf("xxx error: %d\n", errno);
```

```
char *strerror(int errnum);
```

```
printf("xxx error: %s\n", strerror(errno));
```

```
void perror(const char *s);
```

```
perror("open error");
```

read 函数:

```
ssize_t read(int fd, void *buf, size_t count);
```

参数:

fd: 文件描述符

buf: 存数据的缓冲区

count: 缓冲区大小

返回值:

0: 读到文件末尾。

成功: > 0 读到的字节数。

失败: -1 , 设置 `errno`

-1 : 并且 `errno = EAGAIN` 或 `EWOULDBLOCK`, 说明不是 read 失败, 而是 read 在以非阻塞方式读一个设备文件 (网络文件), 并且文件无数据。

write 函数:

```
ssize_t write(int fd, const void *buf, size_t count);
```

参数:

fd: 文件描述符

buf: 待写出数据的缓冲区

count: 数据大小

返回值:

成功: 写入的字节数。

失败: -1 , 设置 `errno`

文件描述符:

PCB 进程控制块: 本质 结构体。

成员：文件描述符表。

文件描述符：0/1/2/3/4。。。。/1023 表中可用的最小的。

0 - STDIN_FILENO

1 - STDOUT_FILENO

2 - STDERR_FILENO

阻塞、非阻塞： 是设备文件、网络文件的属性。

产生阻塞的场景。 读设备文件。读网络文件。（读常规文件无阻塞概念。）

/dev/tty -- 终端文件。

open("/dev/tty", O_RDWR|O_NONBLOCK) --- 设置 /dev/tty 非阻塞状态。（默认为阻塞状态）

fcntl:

int (int fd, int cmd, ...)

int flgs = fcntl(fd, F_GETFL);

flgs |= O_NONBLOCK

fcntl(fd, F_SETFL, flgs);

获取文件状态： F_GETFL

设置文件状态： F_SETFL

lseek 函数:

off_t lseek(int fd, off_t offset, int whence);

参数:

fd: 文件描述符

offset: 偏移量

whence: 起始偏移位置: SEEK_SET/SEEK_CUR/SEEK_END

返回值:

成功: 较起始位置偏移量

失败: -1 errno

应用场景：

1. 文件的“读”、“写”使用同一偏移位置。
2. 使用 lseek 获取文件大小
3. 使用 lseek 拓展文件大小：要想使文件大小真正拓展，必须引起 IO 操作。

使用 truncate 函数，直接拓展文件。 `int ret = truncate("dict.cp", 250);`

传入参数：

1. 指针作为函数参数。
2. 同常有 const 关键字修饰。
3. 指针指向有效区域， 在函数内部做读操作。

传出参数：

1. 指针作为函数参数。
2. 在函数调用之前，指针指向的空间可以无意义，但必须有效。
3. 在函数内部，做写操作。
4. 函数调用结束后，充当函数返回值。

传入传出参数：

1. 指针作为函数参数。
2. 在函数调用之前，指针指向的空间有实际意义。
3. 在函数内部，先做读操作，后做写操作。
4. 函数调用结束后，充当函数返回值。

```
void aaa();
```

```
int aaa(int *p, struct stat *p2, struct student *p3);
```

```
bbb()  
{  
    aaa();  
}
```

stat/lstat 函数：

```
int stat(const char *path, struct stat *buf);
```


参数:

path: 文件路径

buf: (传出参数) 存放文件属性。

返回值:

成功: 0

失败: -1 errno

获取文件大小: buf.st_size

获取文件类型: buf.st_mode

获取文件权限: buf.st_mode

符号穿透: stat 会。lstat 不会。

link/unlink:

隐式回收。

目录操作函数:

```
DIR * opendir(char *name);
```

```
int closedir(DIR *dp);
```

```
struct dirent *readdir(DIR * dp);
```

```
struct dirent {  
    inode  
  
    char dname[256];  
}
```

68P-复习

应用程序的系统调用过程

应用程序->标库函数->系统调用->驱动->硬件

69P-递归遍历目录思路分析

任务需求：使用 `opendir` `closedir` `readdir` `stat` 实现一个递归遍历目录的程序
输入一个指定目录，默认为当前目录。递归列出目录中的文件，同时显示文件大小。

思路分析

递归遍历目录：ls-R.c

1. 判断命令行参数，获取用户要查询的目录名。 `int argc, char *argv[1]`

`argc == 1 --> ./`

2. 判断用户指定的是否是目录。 `stat S_ISDIR()`; --> 封装函数 `isFile()` { }

3. 读目录： `read_dir()` {

`opendir (dir)`

`while (readdir ()) {`

普通文件，直接打印

目录：

拼接目录访问绝对路径。 `sprintf(path, "%s/%s", dir, d_name)`

递归调用自己。--》 `opendir (path) readdir closedir`

`}`

`closedir ()`

`}`

`read_dir()` --> `isFile()` ---> `read_dir()`

70P-递归遍历目录代码预览

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <sys/stat.h>
6. #include <dirent.h>
7. #include <pthread.h>
8.
9. void isFile(char *name);
10.
11. // 打开目录读取,处理目录
12. void read_dir(char *dir, void (*func)(char *))
13. {
14.     char path[256];
15.     DIR *dp;
16.     struct dirent *sdp;
17.
18.     dp = opendir(dir);
19.     if (dp == NULL) {
20.         perror("opendir error");
21.         return;
22.     }
23.     // 读取目录项
24.     while((sdp = readdir(dp)) != NULL) {
25.         if (strcmp(sdp->d_name, ".") == 0 || strcmp(sdp->d_name, "..") == 0) {
26.             continue;
27.         }
28.         //fprintf();
29.         // 目录项本身不可访问, 拼接. 目录/目录项
30.         sprintf(path, "%s/%s", dir, sdp->d_name);
31.
32.         // 判断文件类型, 目录递归进入, 文件显示名字/大小
33.         //isFile(path);
34.         (*func)(path);
35.     }
36.
37.     closedir(dp);
38.
39.     return ;
40. }
41.
42. void isFile(char *name)
43. {
44.     int ret = 0;
45.     struct stat sb;
46.
```

```
47. // 获取文件属性, 判断文件类型
48. ret = stat(name, &sb);
49. if (ret == -1) {
50.     perror("stat error");
51.     return ;
52. }
53. // 是目录文件
54. if (S_ISDIR(sb.st_mode)) {
55.     read_dir(name, isFile);
56. }
57. // 是普通文件, 显示名字/大小
58. printf("%10s\t\t%d\n", name, sb.st_size);
59.
60. return;
61. }
62.
63.
64. int main(int argc, char *argv[])
65. {
66.     // 判断命令行参数
67.     if (argc == 1) {
68.         isFile(".");
69.     } else {
70.         isFile(argv[1]);
71.     }
72.
73.     return 0;
74. }
```

71P-递归遍历目录实现

先写个简易版的，可以判定文件，读取文件大小：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <pthread.h>
6. #include <sys/stat.h>
7.
8. void isFile(char *name){
9.     int ret = 0;
10.    struct stat sb;
11.
12.    ret = stat(name, &sb);
13.    if(ret == -1){
14.        perror("stat error");
15.        return;
16.    }
17.
18.    if(S_ISDIR(sb.st_mode)){
19.
20.    }
21.    printf("%s\t%ld\n", name, sb.st_size);
22.
23.    return;
24. }
25. int main(int argc, char *argv[]){
26.    if(argc == 1) {
27.        isFile(".");
28.    }
29.    else {
30.        isFile(argv[1]);
31.    }
32.
33.    return 0;
34. }
```

编译运行，并查看一个文件，如下：

```
zhcode@ubuntu:~/Code/mylsr$ make ls-R
gcc ls-R.c -o ls-R
zhcode@ubuntu:~/Code/mylsr$ ./ls-R ls-R.c
ls-R.c 457
zhcode@ubuntu:~/Code/mylsr$
```

下面完善功能，把对目录的递归处理补全，如下：

```
75. #include <stdio.h>
```

```
76. #include <stdlib.h>
77. #include <string.h>
78. #include <unistd.h>
79. #include <sys/stat.h>
80. #include <dirent.h>
81. #include <pthread.h>
82.
83. void isFile(char *name);
84.
85. // 打开目录读取,处理目录
86. void read_dir(char *dir, void (*func)(char *))
87. {
88.     char path[256];
89.     DIR *dp;
90.     struct dirent *sdp;
91.
92.     dp = opendir(dir);
93.     if (dp == NULL) {
94.         perror("opendir error");
95.         return;
96.     }
97.     // 读取目录项
98.     while((sdp = readdir(dp)) != NULL) {
99.         if (strcmp(sdp->d_name, ".") == 0 || strcmp(sdp->d_name, "..") == 0) {
100.             continue;
101.         }
102.         //fprintf();
103.         // 目录项本身不可访问, 拼接. 目录/目录项
104.         sprintf(path, "%s/%s", dir, sdp->d_name);
105.
106.         // 判断文件类型,目录递归进入,文件显示名字/大小
107.         //isFile(path);
108.         (*func)(path);
109.     }
110.
111.     closedir(dp);
112.
113.     return ;
114.}
115.
116.void isFile(char *name)
117.{
118.    int ret = 0;
119.    struct stat sb;
120.
121.    // 获取文件属性, 判断文件类型
122.    ret = stat(name, &sb);
123.    if (ret == -1) {
124.        perror("stat error");
```

```

125.     return ;
126. }
127. // 是目录文件
128. if (S_ISDIR(sb.st_mode)) {
129.     read_dir(name, isFile);
130. }
131. // 是普通文件, 显示名字/大小
132. printf("%10s\t\t%ld\n", name, sb.st_size);
133.
134. return;
135.}
136.
137.
138.int main(int argc, char *argv[])
139.{
140.    // 判断命令行参数
141.    if (argc == 1) {
142.        isFile(".");
143.    } else {
144.        isFile(argv[1]);
145.    }
146.
147.    return 0;
148.}

```

这里和视频里有一点差异就是, 这里用的回调函数来实现对目录中目录项的处理, 视频里直接调用的 isFile, 差别不大。

编译运行, 如下:

```

zhcode@ubuntu:~/Code/mylsr$ ./ls-R
./ls-R          9136
./test/hello.c    0
./test/hello2.c  27
./test          4096
./makefile       140
./ls-R.c        1287
.               4096
zhcode@ubuntu:~/Code/mylsr$

```

如图, 基本达到了 ls-r 的功能。

72P-递归遍历目录总结

递归改回调，问题不大

73P-dup 和 dup2

用来做重定向，本质就是复制文件描述符：

dup 和 dup2：

`int dup(int oldfd);` 文件描述符复制。

`oldfd`：已有文件描述符

返回：新文件描述符，这个描述符和 `oldfd` 指向相同内容。

`int dup2(int oldfd, int newfd);` 文件描述符复制，`oldfd` 拷贝给 `newfd`。返回 `newfd`

一个小例子，给一个旧的文件描述符，返回一个新文件描述符：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]) {
9     int fd = open(argv[1], O_RDONLY);
10
11     int newfd = dup(fd);
12     printf("newfd = %d", newfd);
13
14     return 0;
15 }
```

编译运行，如下：

```
zhcode@ubuntu:~/Code/duptest$ make dup
gcc dup.c -o dup
zhcode@ubuntu:~/Code/duptest$ ./dup hello.c
newfd = 4
zhcode@ubuntu:~/Code/duptest$
```

dup 基本就这样了，后续使用也就起个保存作用。

下面讲 `dup2` (`dup2to`):

下面这个例子，将一个已有文件描述符 `fd1` 复制给另一个文件描述符 `fd2`，然后用 `fd2` 修改 `fd1` 指向的文件：

```

1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]) {
9     int fd1 = open(argv[1], O_RDWR);
10    int fd2 = open(argv[2], O_RDWR);
11
12    int fdret = dup2(fd1, fd2);
13    printf("fdret = %d\n", fdret);
14
15    int ret = write(fd2, "megumin is my wife!!!\n", 22);
16    printf("ret = %d\n", ret);
17
18    return 0;
19 }

```

编译运行，如下图：

```

zhcode@ubuntu:~/Code/duptest$ ls
dup dup2.c dup.c hello2.c hello.c makefile
zhcode@ubuntu:~/Code/duptest$ make dup2
gcc dup2.c -o dup2 -Wall -g
zhcode@ubuntu:~/Code/duptest$ ./dup2 hello.c hello2.c
fdret = 4
ret = 21
zhcode@ubuntu:~/Code/duptest$

```

下面查看 hello.c

```

zhcode@ubuntu:~/Code/duptest$ cat hello.c
megumin is my wife!!!
zhcode@ubuntu:~/Code/duptest$

```

没错，惠惠是我老婆。

上面那个例子，fd1 是打开 hello.c 的文件描述符，fd2 是打开 hello2.c 的文件描述符，用 dup2 将 fd1 复制给了 fd2，于是在对 fd2 指向的文件进行写操作时，实际上就是对 fd1 指向的 hello.c 进行写操作。

这里需要注意一个问题，由于 hello.c 和 hello2.c 都是空文件，所以直接写进去没关系。但如果 hello.c 是非空的，写进去的内容默认从文件头部开始写，会覆盖原有内容。

dup2 也可以用于标准输入输出的重定向。

下面这个例子，将输出到 STDOUT 的内容重定向到文件里：

```

1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]) {
9     int fd1 = open(argv[1], O_RDWR);
10    int fd2 = open(argv[2], O_RDWR);
11
12    int fdret = dup2(fd1, fd2);
13    printf("fdret = %d\n", fdret);
14
15    int ret = write(fd2, "megumin is my wife!!!\n", 22);
16    printf("ret = %d\n", ret);
17
18    dup2(fd1, STDOUT_FILENO);
19
20    printf("shinobi is my wife!!!\n");
21    return 0;
22 }

```

编译执行，如下：

```

zhcode@ubuntu:~/Code/dup2test$ make dup2
gcc dup2.c -o dup2 -Wall -g
zhcode@ubuntu:~/Code/dup2test$ ./dup2 hello.c hello2.c
fdret = 4
ret = 22
zhcode@ubuntu:~/Code/dup2test$ cat hello.c
megumin is my wife!!!
shinobi is my wife!!!
zhcode@ubuntu:~/Code/dup2test$

```

这个程序，将 fd1 的内容复制给了 fd2，使得原来指向 hello2.c 的 fd2 也指向了 hello.c 并通过 fd2 向 hello.c 里写入了惠惠是我老婆。完事儿将标准输出重定向至 fd1，就是将要显示在标准输出的内容，写入了 fd1 指向的文件，就是 hello.c 中

这里有一点和上面程序不同，就是 hello.c 是处于打开状态的，连续写入两段话，写入小忍是我老婆的时候，读写指针在这句话末尾，就不会覆盖惠惠是我老婆这句话，所以，都是我老婆，没有问题的。这里再强调一下，打开一个文件，读写指针默认在文件头，如果文件本身有内容，直接写入会覆盖原有内容。

74P-fcntl 实现 dup 描述符

fcntl 函数实现 dup:

```
int fcntl(int fd, int cmd, ....)
```

cmd: F_DUPFD

参 3: 被占用的, 返回最小可用的。

未被占用的, 返回=该值的文件描述符。

下面这个代码用 fcntl 来实现描述符的复制:

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int fd1 = open(argv[1], O_RDWR);
10
11     printf("fd1 = %d\n", fd1);
12
13     int newfd = fcntl(fd1, F_DUPFD, 0);
14     printf("newfd = %d\n", newfd);
15
16     return 0;
17 }
```

对于 fcntl 中的参数 0, 这个表示 0 被占用, fcntl 使用文件描述符表中的最小文件描述符返回
假设传入 0, 传一个 7, 且 7 未被占用, 则会返回 7

所以这个参数可以这样理解, 你传入一个文件描述符 k, 如果 k 没被占用, 则直接用 k 复制 fd1 的内容。如果 k 被占用, 则返回描述符表中最小可用描述符, 也就是自己指定一个一志愿, 如果行, 就返回这个。如果不行, 国家给你分配一个最小的。

编译执行, 如下:

```
zhcode@ubuntu:~/Code/code74$ make fcntl_dup
gcc fcntl_dup.c -o fcntl_dup -Wall -g
zhcode@ubuntu:~/Code/code74$ ./fcntl_dup hello.c
fd1 = 3
newfd = 4
zhcode@ubuntu:~/Code/code74$
```

如图可知, 原来指向 hello.c 的文件描述符是 3, 复制了一个, 新的文件描述符 4 也指向 hello.c

下面这个例子, 用 fcntl 复制 2 次文件描述符, 第一次使用默认分配, 就是传 0, 第二次使用自己选定文件描述符复制, 完事儿向文件里写入内容

```

1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int fd1 = open(argv[1], O_RDWR);
10
11     printf("fd1 = %d\n", fd1);
12
13     int newfd = fcntl(fd1, F_DUPFD, 0);
14     printf("newfd = %d\n", newfd);
15
16     int newfd2 = fcntl(fd1, F_DUPFD, 9);
17     printf("newfd2 = %d\n", newfd2);
18
19     int ret = write(newfd2, "megumin is my wife!!!\n", 22);
20     printf("ret = %d\n", ret);
21
22     return 0;
23 }

```

编译执行，结果如下：

```

zhcode@ubuntu:~/Code/code74$ make fcntl_dup
gcc fcntl_dup.c -o fcntl_dup -Wall -g
zhcode@ubuntu:~/Code/code74$ ./fcntl_dup hello.c
fd1 = 3
newfd = 4
newfd2 = 9
ret = 22
zhcode@ubuntu:~/Code/code74$ cat hello.c
megumin is my wife!!!
zhcode@ubuntu:~/Code/code74$

```

可见上述说明都是没有问题的。

75P-复习

76P-进程和程序以及 CPU 相关

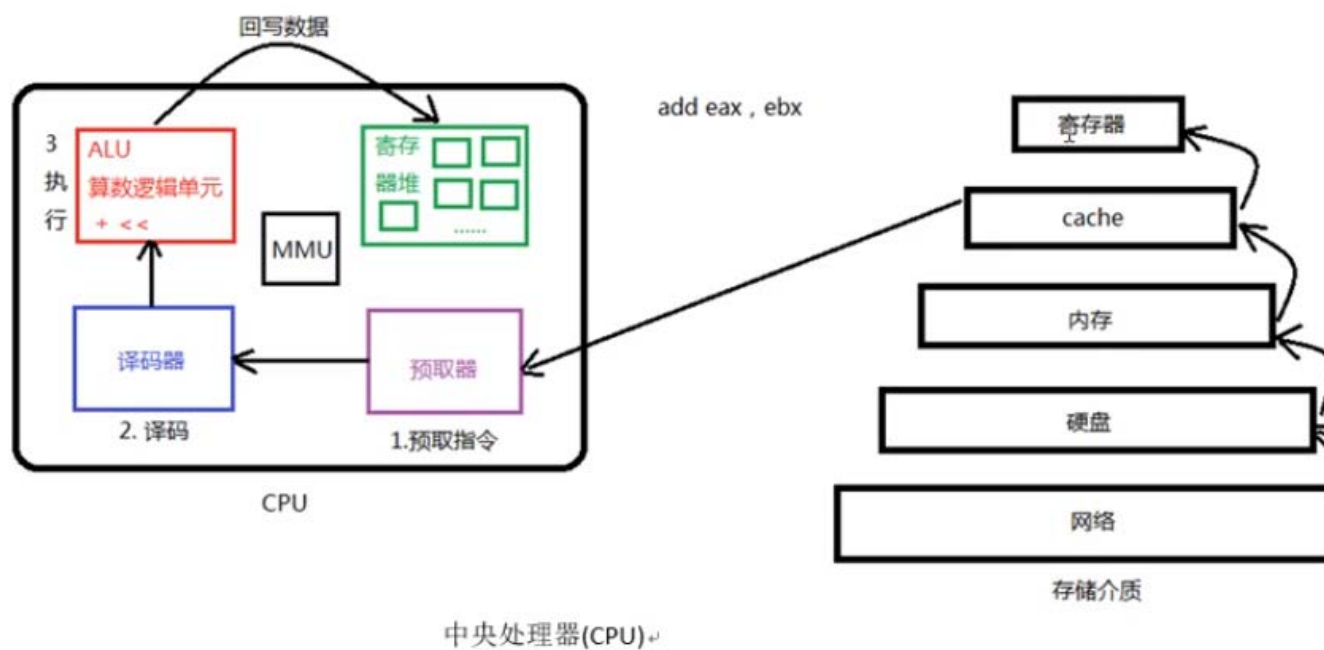
进程：

程序：死的。只占用磁盘空间。——剧本。

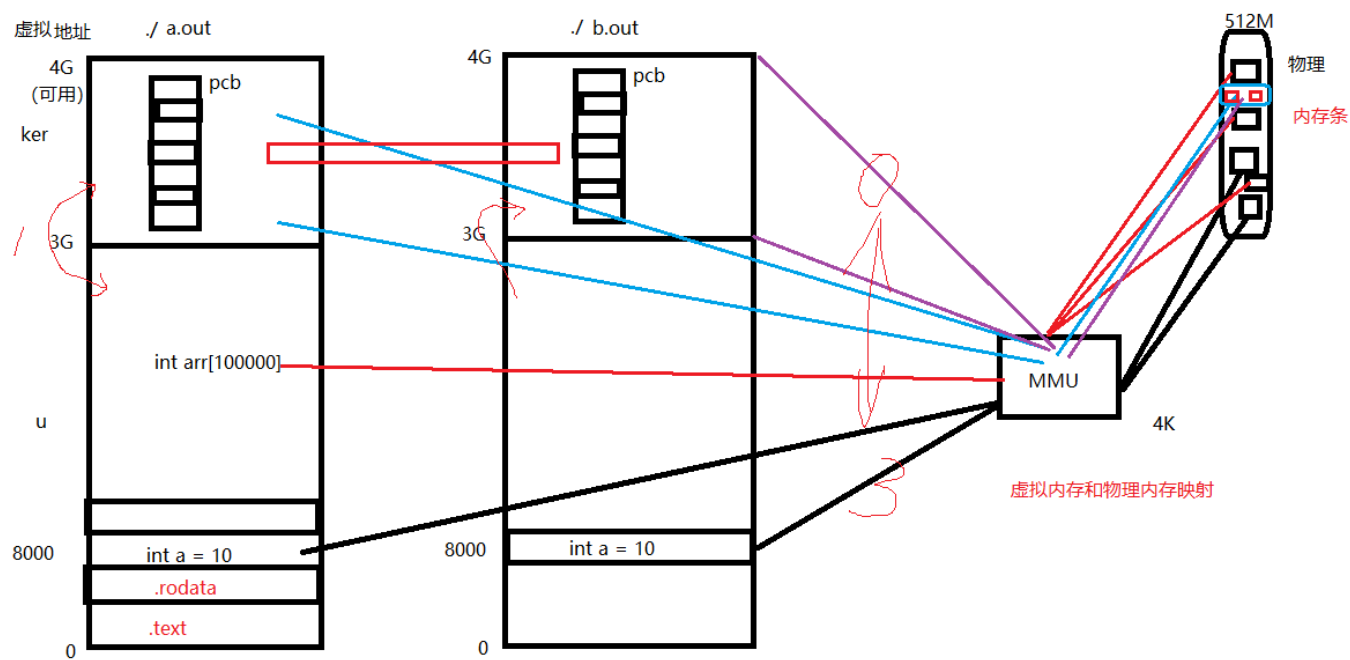
进程：活的。运行起来的程序。占用内存、cpu 等系统资源。——戏。

并发和并行：并行是宏观上并发，微观上串行

.CPU 和 MMU



77P-虚拟内存和物理内存映射关系



78P-pcb 进程控制块

PCB 进程控制块：

进程 id

文件描述符表

进程状态：初始态、就绪态、运行态、挂起态、终止态。

进程工作目录位置

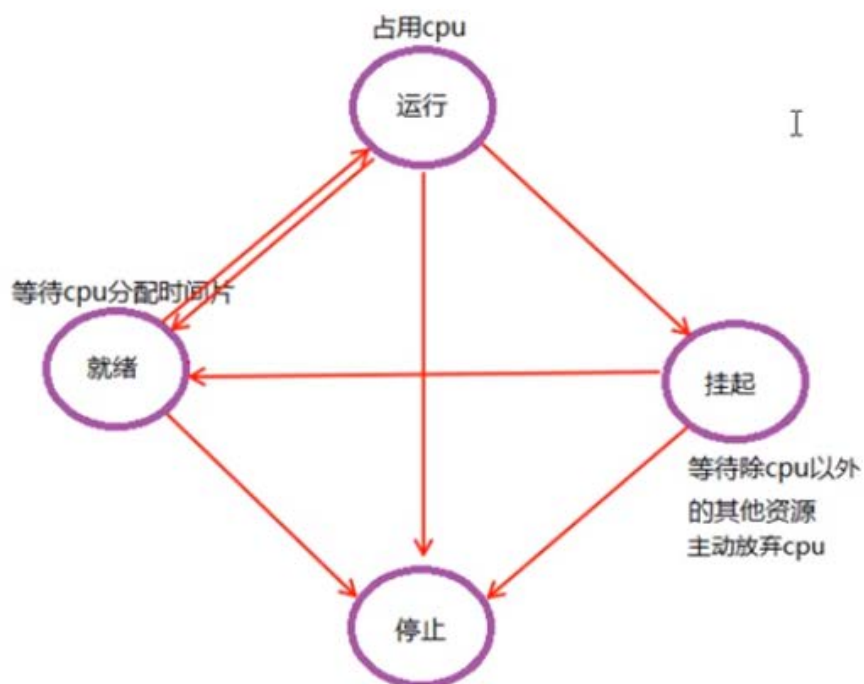
*umask 掩码（进程的概念）

信号相关信息资源。

用户 id 和组 id

ps aux 返回结果里，第二列是进程 id

进程基本的状态有 5 种。分别为初始态，就绪态，运行态，挂起态与终止态。其中初始态为进程准备阶段，常与就绪态结合来看。



79P-环境变量

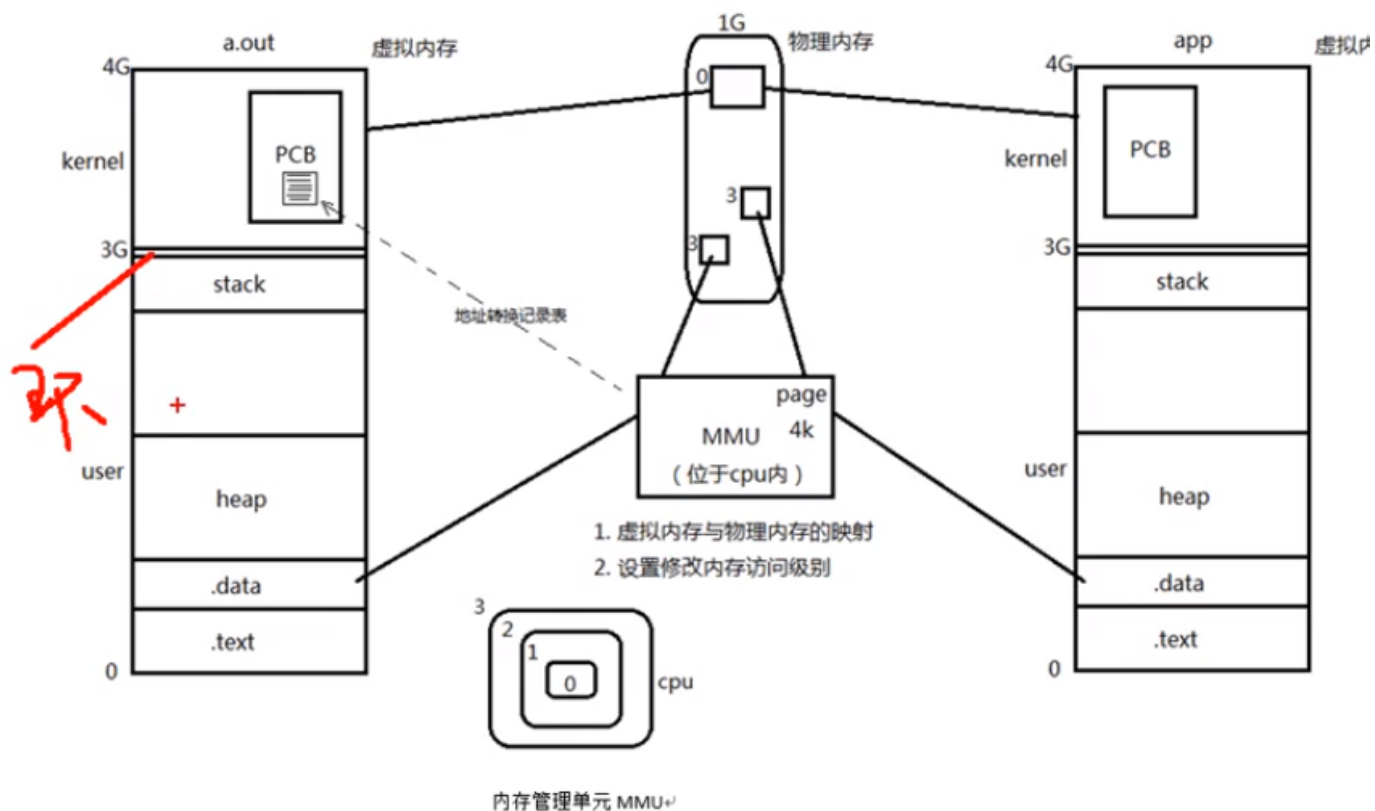
echo \$PATH 查看环境变量

path 环境变量里记录了一系列的值，当运行一个可执行文件时，系统会去环境变量记录的位置里查找这个文件并执行。

echo \$TERM 查看终端

echo \$LANG 查看语言

env 查看所有环境变量



80P-fork 函数原理

fork 函数：

```
pid_t fork(void)
```

创建子进程。父子进程各自返回。父进程返回子进程 pid。 子进程返回 0。

```
getpid();getppid();
```

循环创建 N 个子进程模型。 每个子进程标识自己的身份。

父子进程相同：

刚 fork 后。 data 段、text 段、堆、栈、环境变量、全局变量、宿主目录位置、进程工作目录位置、信号处理方式

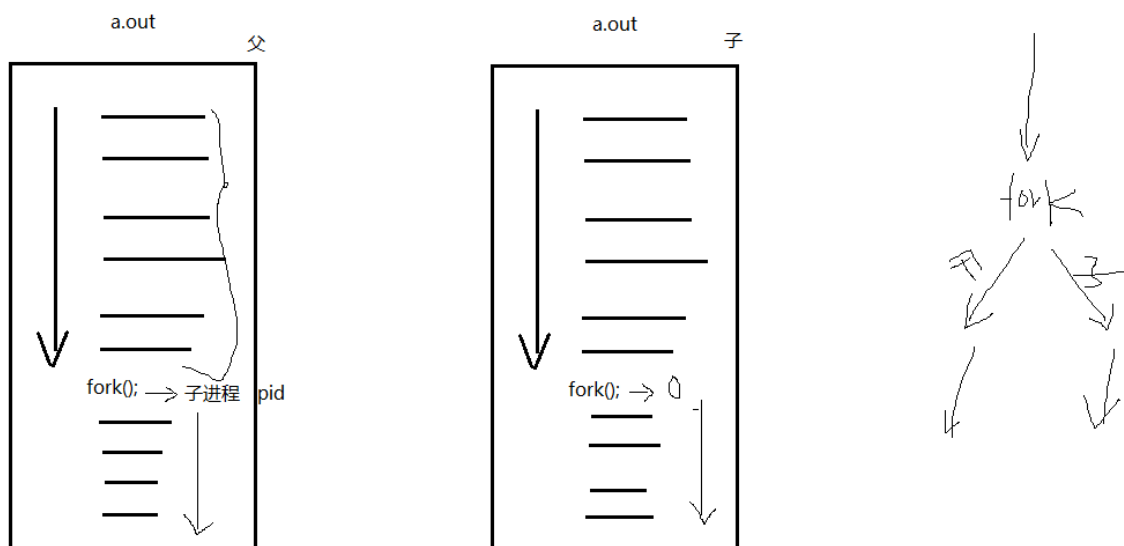
父子进程不同：

进程 id、返回值、各自的父进程、进程创建时间、闹钟、未决信号集

父子进程共享：

读时共享、写时复制。————— 全局变量。

1. 文件描述符 2. mmap 映射区。



81P-fork 创建子进程

下面是一个 fork 函数的例子，代码如下：

```
8 int main(int argc, char *argv[]){
9     printf("before fork-1-\n");
10    printf("before fork-2-\n");
11    printf("before fork-3-\n");
12    printf("before fork-4-\n");
13
14    pid_t pid = fork();
15    if(pid == -1){
16        perror("fork error");
17        exit(1);
18    }
19    else if(pid == 0){
20        printf("---child is created\n");
21    }
22    else if(pid > 0){
23        printf("---parent process : my child is %d\n", pid);
24    }
25
26    printf("====end of file\n");
27
28    return 0;
29 }
```

编译运行，如下：

```
zhcode@ubuntu:~/Code/code81$ make fork
gcc fork.c -o fork -Wall -g
zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
before fork-3-
before fork-4-
---parent process : my child is 3037
====end of file
zhcode@ubuntu:~/Code/code81$ ---child is created
====end of file
```

关于这里为啥终端提示符和输出信息混在了一起，循环创建多个子进程（后面第二节）那一节会进行分析，现在先不用管。

fork 之前的代码，父子进程都有，但是只有父进程执行了，子进程没有执行，fork 之后的代码，父子进程都有机会执行。

两个函数：

pid_t getpid()	获取当前进程 id
pid_t getppid()	获取当前进程的父进程 id

82P-getpid 和 getppid

来个小例子，就是在上面一个 fork 例子里面加入这两个函数：

```
8 int main(int argc, char *argv[]){
9     printf("before fork-1-\n");
10    printf("before fork-2-\n");
11
12    pid_t pid = fork();
13    if(pid == -1){
14        perror("fork error");
15        exit(1);
16    }
17    else if(pid == 0){
18        printf("child is created, pid= %d, parent-pid: %d\n", getpid(), getppid());
19    }
20    else if(pid > 0){
21        printf("p process: child is %d, my pid: %d, my ppid: %d\n", pid, getpid(), getppid());
22    }
23
24    printf("=====  
end of file\n");
25
26    return 0;
27 }
```

编译运行，如下：

```
zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
p process: child is 3140, my pid: 3139, my ppid: 2911
=====  
end of file
zhcode@ubuntu:~/Code/code81$ child is created, pid= 3140, parent-pid: 1630
=====  
end of file
```

这里，问题出现了。视频里，父进程子进程 id 能相互对应上，但这里没有对应上。

先看图里子进程的输出：

子进程 pid=3140， 父进程 pid=1630

再看图里父进程的输出：

子进程 pid=3140， 自己进程 pid=3139， 父进程 pid=2911

这很有问题，3139 作为父进程创建了 3140 这个子进程，然而 3140 这个子进程说 1630 是它的父进程。这里感受到了一股老王的气息。

于是，为了搞清楚这是不是个例，再运行了几次程序，结果如下：

```

zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
p process: child is 3201, my pid: 3200, my ppid: 2911
=====end of file
zhcode@ubuntu:~/Code/code81$ child is created, pid= 3201, parent-pid: 1630
=====end of file

zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
p process: child is 3212, my pid: 3211, my ppid: 2911
=====end of file
zhcode@ubuntu:~/Code/code81$ child is created, pid= 3212, parent-pid: 1630
=====end of file

zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
p process: child is 3214, my pid: 3213, my ppid: 2911
=====end of file
zhcode@ubuntu:~/Code/code81$ child is created, pid= 3214, parent-pid: 1630
=====end of file

```

从这个图可以看到，所有父进程的子进程 pid=父进程 pid+1，而子进程的父进程均为 1630，这个 1630 多次出现，显然不是偶然。

这里做出一个推测，假设图中所有进程都是父进程先结束，导致子进程成孤儿，于是回收到孤儿院，看起来合情合理。

修改一下代码，给父进程增加一个等待命令，这样能保证子进程完成时，父进程处于执行状态，子进程就不会成孤儿。同时，这里也解决了终端提示符和输出混在一起的问题，这个问题会在下一节分析，不用管，代码如下：

```

8 int main(int argc, char *argv[]){
9     printf("before fork-1-\n");
10    printf("before fork-2-\n");
11
12    pid_t pid = fork();
13    if(pid == -1){
14        perror("fork error");
15        exit(1);
16    }
17    else if(pid == 0){
18        printf("child is created, pid= %d, parent-pid: %d\n", getpid(), getppid());
19    }
20    else if(pid > 0){
21        sleep(1);
22        printf("p process: child is %d, my pid: %d, my ppid: %d\n", pid, getpid(), getppid());
23    }
24
25    printf("end of the file\n");
26    return 0;
27 }

```

编译运行，如下：

```

zhcode@ubuntu:~/Code/code81$ make fork
gcc fork.c -o fork -Wall -g
zhcode@ubuntu:~/Code/code81$ ./fork
before fork-1-
before fork-2-
child is created, pid= 2478, parent-pid: 2477
end of the file
p process: child is 2478, my pid: 2477, my ppid: 2259
end of the file
zhcode@ubuntu:~/Code/code81$

```

2477 子进程是 2478, 2478 父进程是 2477, 没有问题。
 错怪 1630 了, 它不是老王。

再看, 父进程的父进程 pid=2259, 查看一下这是个啥:

```

zhcode@ubuntu:~/Code/code81$ ps aux | grep 2259
zhcode      2259  0.0  0.2  24308  5372 pts/6    Ss   21:10   0:00 bash
zhcode      2483  0.0  0.0  15964  1020 pts/6    S+   21:31   0:00 grep --color=auto 2259
zhcode@ubuntu:~/Code/code81$

```

如图, 这是 bash, 其实我们写的所有进程都是 bash 的子进程
 那么疯狂收孤儿的 1630 呢, 如下:

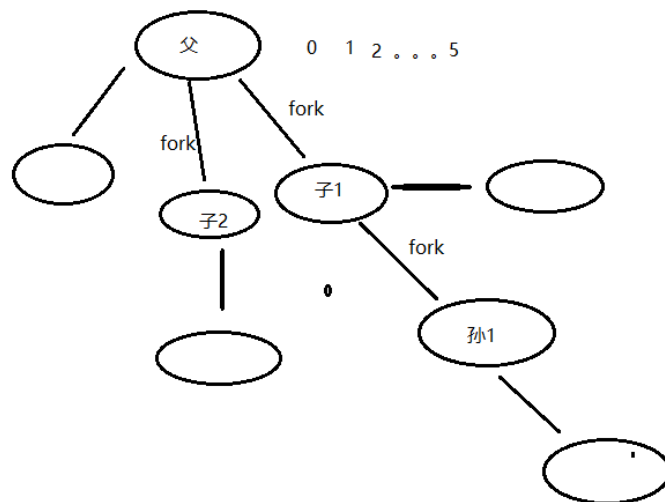
```

zhcode@ubuntu:~/Code/code81$ ps aux | grep 1630
zhcode      1630  0.0  0.2  48204  4676 ?        Ss   17:00   0:00 /sbin/upstart --user
zhcode      3591  0.0  0.0  15964   928 pts/4    S+   19:32   0:00 grep --color=auto 1630

```

这里的 upstart, 就是进程孤儿院。

83P-循环创建多个子进程



所以，直接用个 for 循环是要出事情的，因为子进程也会 fork 新的进程
这里，对调用 fork 的进程进行判定，只让父进程 fork 新的进程就行，代码如下：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int i;
10
11     for(i=0; i<5; i++){
12         if(fork() == 0)
13             break;
14     }
15     printf("I'm %dth child\n", i+1);
16
17     return 0;
18 }
```

编译执行，如图：

```
zhcode@ubuntu:~/Code/code81$ make loop_fork
gcc loop_fork.c -o loop_fork -Wall -g
zhcode@ubuntu:~/Code/code81$ ./loop_fork
I'm 6th child
zhcode@ubuntu:~/Code/code81$ I'm 5th child
I'm 4th child
I'm 2th child
I'm 3th child
I'm 1th child

zhcode@ubuntu:~/Code/code81$
```

出现了问题：进程多了一个，而且不是按顺序来的。这里多出的一个，是父进程，因为父进程才有 `i=5` 跳出循环这一步。所以，对父进程进行判定并处理
修改代码如下：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int i;
10
11     for(i=0; i<5; i++){
12         if(fork() == 0)
13             break;
14     }
15     if(i == 5)
16         printf("I'm parent \n");
17     else
18         printf("I'm %dth child\n", i+1);
19
20     return 0;
21 }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code81$ ./loop_fork
I'm parent
zhcode@ubuntu:~/Code/code81$ I'm 5th child
I'm 4th child
I'm 3th child
I'm 2th child
I'm 1th child
█
```

现在还有两个问题，

一个就是包括父进程在内的所有进程不是按顺序出现，多运行几次，发现是随机序列出现的。这要因为，对操作系统而言，这几个子进程几乎是同时出现的，它们和父进程一起争夺 cpu，谁抢到，谁打印，所以出现顺序是随机的。

第二问题就是终端提示符混在了输出里，这个是因为，`loop_fork` 是终端的子进程，一旦 `loop_fork` 执行完，终端就会打印提示符。就像之前没有子进程的程序，一旦执行完，就出现了终端提示符。这里也就是这个道理，`loop_fork` 执行完了，终端提示符出现，然而 `loop_fork` 的子进程还没执行完，所以输出就混在一起了。

下面通过 sleep 延时来解决父进程先结束这个问题。代码如下，就是给父进程加了个 sleep：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int i;
10
11     for(i=0; i<5; i++){
12         if(fork() == 0)
13             break;
14     }
15     if(i == 5){
16         printf("I'm parent \n");
17         sleep(1);
18     }
19     else
20         printf("I'm %dth child\n", i+1);
21
22     return 0;
23 }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code81$ make loop_fork2
gcc loop_fork2.c -o loop_fork2 -Wall -g
zhcode@ubuntu:~/Code/code81$ ./loop_fork2
I'm parent
I'm 5th child
I'm 4th child
I'm 2th child
I'm 3th child
I'm 1th child
zhcode@ubuntu:~/Code/code81$
```

可以看到，更改之后，父进程在所有子进程后结束，所以终端提示符最后出现。

这里和视频里有一点差异，我这里“I'm parent”是先于子进程的输出的，因为我这里是父进程在 sleep 之前就打印信息了，视频里是 sleep 之后打印。这个不是大问题，怎么写都行，因为父进程虽然在打印之前 fork 了子进程，照理来说子进程会和父进程抢 cpu，打印顺序会乱。但是由于父进程已经处于执行状态，所以一般来说父进程一定会先于子进程打印。当然这个对于不同操作系统不太一样，万一时间片再短点，父进程在打印之前，时间片到了，子进程抢到 cpu 就开始打印了，那么父进程打印信息就不一定在第一位了。道理是这么个道理，最好还是先 sleep 再打印吧，这样父进程一定是最后输出的。

最后来解决子进程乱序输出的问题,解决方法很简单,让第 1 个子进程少等,第二个子进程多等,后面子进程等待时间依次增加,这样就能实现有序输出。
代码如下:

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(int argc, char *argv[]){
9     int i;
10
11     for(i=0; i<5; i++){
12         if(fork() == 0)
13             break;
14     }
15     if(i == 5){
16         printf("I'm parent \n");
17         sleep(5);
18     }
19     else{
20         sleep(i);
21         printf("I'm %dth child\n", i+1);
22     }
23     return 0;
24 }
```

编译运行, 如图:

```
zhcode@ubuntu:~/Code/code81$ make loop_fork3
gcc loop_fork3.c -o loop_fork3 -Wall -g
zhcode@ubuntu:~/Code/code81$ ./loop_fork3
I'm parent
I'm 1th child
I'm 2th child
I'm 3th child
I'm 4th child
I'm 5th child
zhcode@ubuntu:~/Code/code81$
```

这下执行几次都这样, 有序输出, 问题不大。

84P-父子进程共享哪些内容

父子进程相同：

刚 fork 后。 data 段、text 段、堆、栈、环境变量、全局变量、宿主目录位置、进程工作目录位置、信号处理方式

父子进程不同：

进程 id、返回值、各自的父进程、进程创建时间、闹钟、未决信号集

父子进程共享：

读时共享、写时复制。————— 全局变量。

1. 文件描述符 2. mmap 映射区。

85P-父子进程共享

父子进程共享：

读时共享、写时复制。————— 全局变量。

1. 文件描述符 2. mmap 映射区。

86P-总结

```
./a.out ls.c /home/itcast/28_Linux ./abc/  
  
/home/itcast/28_Linux/testdir/
```

递归遍历目录: ls-R.c

1. 判断命令行参数, 获取用户要查询的目录名。 `int argc, char *argv[1]`
`argc == 1 --> ./`
2. 判断用户指定的是否是目录。 `stat S_ISDIR(); --> 封装函数 isFile() { }`
3. 读目录: `read_dir() {`
`opendir (dir)`
`while (readdir ()) {`
普通文件, 直接打印
目录:
拼接目录访问绝对路径。 `sprintf(path, "%s/%s", dir, d_name)`
递归调用自己。 --> `opendir (path) readdir closedir`
`}`
`closedir ()`
`}`
`read_dir() --> isFile() ---> read_dir()`

dup 和 dup2:

```
int dup(int oldfd);      文件描述符复制。  
  
oldfd: 已有文件描述符  
  
返回: 新文件描述符。  
  
int dup2(int oldfd, int newfd); 文件描述符复制。重定向。
```

fcntl 函数实现 dup:

```
int fcntl(int fd, int cmd, ....)
```

cmd: F_DUPFD

参 3: 被占用的, 返回最小可用的。

未被占用的, 返回=该值的文件描述符。

=====

进程:

程序: 死的。只占用磁盘空间。 ——脚本。

进程: 活的。运行起来的程序。占用内存、cpu 等系统资源。 ——戏。

PCB 进程控制块:

进程 id

文件描述符表

进程状态: 初始态、就绪态、运行态、挂起态、终止态。

进程工作目录位置

*umask 掩码

信号相关信息资源。

用户 id 和组 id

fork 函数:

```
pid_t fork(void)
```

创建子进程。父子进程各自返回。父进程返回子进程 pid。 子进程返回 0.

```
getpid();getppid();
```

循环创建 N 个子进程模型。 每个子进程标识自己的身份。

父子进程相同:

刚 fork 后。 data 段、text 段、堆、栈、环境变量、全局变量、宿主目录位置、进程工作目录位置、信号处理方式

父子进程不同：

进程 id、返回值、各自的父进程、进程创建时间、闹钟、未决信号集

父子进程共享：

读时共享、写时复制。————— 全局变量。

1. 文件描述符 2. mmap 映射区。

87P-复习

88P-父子进程 gdb 调试

`gdb` 调试：

设置父进程调试路径：`set follow-fork-mode parent`（默认）

设置子进程调试路径：`set follow-fork-mode child`

注意，一定要在 `fork` 函数调用之前设置才有效。

89P-exec 函数族

exec 函数族:

使进程执行某一程序。成功无返回值，失败返回 -1

`int execlp(const char *file, const char *arg, ...);` 借助 PATH 环境变量找寻待执行程序

参 1: 程序名

参 2: `argv0`

参 3: `argv1`

...: `argvN`

哨兵: `NULL`

`int execl(const char *path, const char *arg, ...);` 自己指定待执行程序路径。

`int execvp();`

`ps ajx --> pid ppid gid sid`

• exec 函数族 •

`fork` 创建子进程后执行的是和父进程相同的程序（但有可能执行不同的代码分支），子进程往往要调用一种 `exec` 函数以执行另一个程序。当进程调用一种 `exec` 函数时，该进程的用户空间代码和数据完全被新程序替换，从新程序的启动例程开始执行。调用 `exec` 并不创建新进程，所以调用 `exec` 前后该进程的 `id` 并未改变。•

将当前进程的 `.text`、`.data` 替换为所要加载的程序的 `.text`、`.data`，然后让进程从新的 `.text` 第一条指令开始执行，但进程 `ID` 不变，换核不换壳。•

其实有六种以 `exec` 开头的函数，统称 `exec` 函数：•

`int execl(const char *path, const char *arg, ...);`•

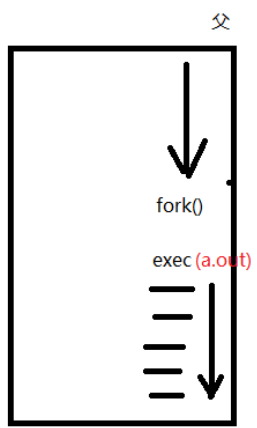
`int execlp(const char *file, const char *arg, ...);`•

`int execl(const char *path, const char *arg, ..., char *const envp[]);`•

`int execv(const char *path, char *const argv[]);`•

`int execvp(const char *file, char *const argv[]);`•

`int execve(const char *path, char *const argv[], char *const envp[]);`•



90P-execlp 和 ececl 函数

`int execlp(const char *file, const char *arg, ...)`

成功，无返回，失败返回-1

参数 1: 要加载的程序名字，该函数需要配合 PATH 环境变量来使用，当 PATH 所有目录搜索后没有参数 1 则返回出错。

该函数通常用来调用系统程序。如 `ls`、`date`、`cp`、`cat` 命令。

`execlp` 这里的 `p`，表示要借助环境变量来加载可执行文件

示例代码，通过 `execlp` 让子进程去执行 `ls` 命令：

```
8 int main(int argc, char *argv[]){
9
10     pid_t pid = fork();
11     if(pid == -1){
12         perror("fork error");
13         exit(1);
14     }
15     else if(pid == 0){
16         execlp("ls", "-l", "-d", "-h", NULL); // NULL is must
17         perror("exec error");
18         exit(1);
19     }
20     else if(pid > 0){
21         printf("I'm parent : %d\n", getpid());
22     }
23
24     return 0;
25 }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code90$ make fork_exec
gcc fork_exec.c -o fork_exec -Wall -g
zhcode@ubuntu:~/Code/code90$ ./fork_exec
I'm parent : 2698
zhcode@ubuntu:~/Code/code90$ .
```

只有父进程正确执行并输出了，子进程的 `ls` 输出有问题。

问题出在参数上，可变参数那里，是从 `argv[0]` 开始计算的。

修改代码，就是将缺失的 `argv[0]` 补上，然后让父进程延时 1 秒，保证终端提示符不和输出干扰。如下：

```

8 int main(int argc, char *argv[]){
9
10     pid_t pid = fork();
11     if(pid == -1){
12         perror("fork error");
13         exit(1);
14     }
15     else if(pid == 0){
16         // execlp("ls", "-l", "-d", "-h", NULL);
17         execlp("ls", "ls", "-l", "-h", NULL); // NULL is must
18         perror("exec error");
19         exit(1);
20     }
21     else if(pid > 0){
22         sleep(1);
23         printf("I'm parent : %d\n", getpid());
24     }
25
26     return 0;
27 }

```

编译执行，如下：

```

zhcode@ubuntu:~/Code/code90$ make fork_exec2
gcc fork_exec2.c -o fork_exec2 -Wall -g
zhcode@ubuntu:~/Code/code90$ ./fork_exec2
total 44K
-rw-rw-r-- 1 zhcode zhcode 556 May 26 11:06 fork.c
-rwxrwxr-x 1 zhcode zhcode 10K May 26 11:17 fork_exec
-rwxrwxr-x 1 zhcode zhcode 10K May 26 11:30 fork_exec2
-rw-rw-r-- 1 zhcode zhcode 478 May 26 11:30 fork_exec2.c
-rw-rw-r-- 1 zhcode zhcode 421 May 26 11:17 fork_exec.c
-rw-rw-r-- 1 zhcode zhcode 119 May 26 11:06 headcode.c
-rw-rw-r-- 1 zhcode zhcode 169 May 26 11:06 makefile
I'm parent : 2837
zhcode@ubuntu:~/Code/code90$

```

这个看起来就很科学了。于是子进程就能随意调用可执行程序了，这个可执行程序可以是系统的，也可以是自定义的。

下面使用 `execl` 来让子程序调用自定义的程序。

```
int execl(const char *path, const char *arg, ...)
```

这里要注意，和 `execlp` 不同的是，第一个参数是路径，不是文件名。

这个路径用相对路径和绝对路径都行。

调用的代码如下：

```

1 #include <stdio.h>
2
3 int main(){
4
5     printf("megumin is my wife!!!\n");
6
7     return 0;
8 }

```

exec 代码如下:

```

8 int main(int argc, char *argv[]){
9
10     pid_t pid = fork();
11     if(pid == -1){
12         perror("fork error");
13         exit(1);
14     }
15     else if(pid == 0){
16         execl("./mywife", "./mywidfe", NULL); // NULL is must
17         perror("exec error");
18         exit(1);
19     }
20     else if(pid > 0){
21         sleep(1);
22         printf("I'm parent : %d\n", getpid());
23     }
24
25     return 0;
26 }

```

编译执行, 如下:

```

zhcode@ubuntu:~/Code/code90$ make fork_exec3
gcc fork_exec3.c -o fork_exec3 -Wall -g
zhcode@ubuntu:~/Code/code90$ ./fork_exec3
megumin is my wife!!!
I'm parent : 2932
zhcode@ubuntu:~/Code/code90$

```

这就很强势了。

用 execl 也能执行 ls 这些, 把路径给出来就行, 但是这样麻烦, 所以对于系统指令一般还是用 execlp

91P-exec 函数族特性

写一个程序，使用 `execlp` 执行进程查看，并将结果输出到文件里。

要用到 `open`, `execlp`, `dup2`

代码如下：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pthread.h>
7
8 int main(){
9     int fd;
10    fd = open("ps.out", O_WRONLY|O_CREAT|O_TRUNC, 0644);
11    if(fd < 0){
12        perror("open ps.out error");
13        exit(1);
14    }
15
16    dup2(fd, STDOUT_FILENO);
17
18    execlp("ps", "ps", "aux", NULL);
19    close(fd); // have no need to
20    return 0;
21 }
```

编译执行，如下：

```
zhcode@ubuntu:~/Code/code91$ make exec_ps
gcc exec_ps.c -o exec_ps -Wall -g
zhcode@ubuntu:~/Code/code91$ ./exec_ps
zhcode@ubuntu:~/Code/code91$ ls
exec_ps  exec_ps.c  headcode.c  makefile  ps.out
zhcode@ubuntu:~/Code/code91$ cat ps.out
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0   0.2 119764  5884 ?        Ss   11:03   0:02 /sbin/init auto noprompt
root         2   0.0   0.0      0     0 ?        S    11:03   0:00 [kthreadd]
root         3   0.0   0.0      0     0 ?        S    11:03   0:00 [ksoftirqd/0]
root         5   0.0   0.0      0     0 ?        S<   11:03   0:00 [kworker/0:0H]
root         7   0.0   0.0      0     0 ?        S    11:03   0:00 [rcu_sched]
root         8   0.0   0.0      0     0 ?        S    11:03   0:00 [rcu_bh]
root         9   0.0   0.0      0     0 ?        S    11:03   0:00 [migration/0]
```

很强势，问题不大。

`exec` 函数族一般规律：

`exec` 函数一旦调用成功，即执行新的程序，不返回。只有失败才返回，错误值-1，所以通常我们直接在 `exec` 函数调用后直接调用 `perror()`，和 `exit()`，无需 `if` 判断。

<code>l(list)</code>	命令行参数列表
<code>p(path)</code>	搜索 file 时使用 <code>path</code> 变量
<code>v(vector)</code>	使用命令行参数数组
<code>e(environment)</code>	使用环境变量数组，不适用进程原有的环境变量，设置新加载程序运行的环境变量

事实上，只有 `execve` 才是真正的系统调用，其他 5 个函数最终都调用 `execve`，是库函数，所以 `execve` 在 man 手册第二节，其它函数在 man 手册第 3 节。

92P-孤儿进程和僵尸进程

孤儿进程：

父进程先于子进程终止，子进程沦为“孤儿进程”，会被 `init` 进程领养。

僵尸进程：

子进程终止，父进程尚未对子进程进行回收，在此期间，子进程为“僵尸进程”。`kill` 对其无效。这里要注意，每个进程结束后都必然会经历僵尸态，时间长短的差别而已。

子进程终止时，子进程残留资源 PCB 存放于内核中，PCB 记录了进程结束原因，进程回收就是回收 PCB。回收僵尸进程，得 `kill` 它的父进程，让孤儿院去回收它。

93P-wait 回收子进程

wait 函数： 回收子进程退出资源， 阻塞回收任意一个。

```
pid_t wait(int *status)
```

参数：（传出） 回收进程的状态。

返回值：成功： 回收进程的 pid

失败： -1, errno

函数作用 1： 阻塞等待子进程退出

函数作用 2： 清理子进程残留在内核的 pcb 资源

函数作用 3： 通过传出参数，得到子进程结束状态

获取子进程正常终止值：

WIFEXITED(status) --》 为真 --》调用 WEXITSTATUS(status) --》 得到 子进程 退出值。

获取导致子进程异常终止信号：

WIFSIGNALED(status) --》 为真 --》调用 WTERMSIG(status) --》 得到 导致子进程异常终止的信号编号。

一个进程终止时会关闭所有文件描述符，释放在用户空间分配的内存，但它的 PCB 还保留着，内核在其中保存了一些信息：如果是正常终止则保存着退出状态，如果是异常终止则保存着导致该进程终止的信号是哪个。这个进程的父进程可以调用 wait 或者 waitpid 获取这些信息，然后彻底清除掉这个进程。我们知道一个进程的退出状态可以在 shell 中用特殊变量 \$? 查看，因为 shell 是它的父进程，当它终止时，shell 调用 wait 或者 waitpid 得到它的退出状态，同时彻底清除掉这个进程。

```
pid_t wait(int *status)
```

其中 status 是传出参数

下面这个例子，使用 wait 来阻塞回收子进程。

```

9 int main(int argc, char *argv[]){
10     pid_t pid, wpid;
11     int status;
12
13     pid = fork();
14     if(pid == 0){
15         printf("child, my parent=%d. going to sleep 5s\n", getpid());
16         sleep(10);
17         printf("child die!!!!!!!!\n");
18     }
19     else if(pid > 0){
20         wpid = wait(&status);
21         if(wpid == -1){
22             perror("wait error");
23             exit(1);
24         }
25         printf("parent wait finish\n");
26     }
27     else{
28         perror("fork");
29         return 1;
30     }
31     return -1;
32 }

```

编译运行，如下：

```

zhcode@ubuntu:~/Code/code93$ make zoom_test
gcc zoom_test.c -o zoom_test -Wall -g
zhcode@ubuntu:~/Code/code93$ ./zoom_test
child, my parent=4129. going to sleep 5s
child die!!!!!!!!
parent wait finish
zhcode@ubuntu:~/Code/code93$

```

94P-获取子进程退出值和异常终止信号

获取子进程正常终止值：

WIFEXITED(status) --》 为真 --》调用 WEXITSTATUS(status) --》 得到 子进程 退出值。

获取导致子进程异常终止信号：

WIFSIGNALED(status) --》 为真 --》调用 WTERMSIG(status) --》 得到 导致子进程异常终止的信号编号。

下面这个代码捕获程序异常终止的信号并打印：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <unistd.h>
4. #include <sys/wait.h>
5.
6. int main(void)
7. {
8.     pid_t pid, wpid;
9.     int status;
10.
11.     pid = fork();
12.     if (pid == 0) {
13.         printf("---child, my id= %d, going to sleep 10s\n", getpid());
14.         sleep(10);
15.         printf("-----child die-----\n");
16.         return 73;
17.     } else if (pid > 0) {
18.         //wpid = wait(NULL);          // 不关心子进程结束原因
19.         wpid = wait(&status);        // 如果子进程未终止,父进程阻塞在这个函数上
20.         if (wpid == -1) {
21.             perror("wait error");
22.             exit(1);
23.         }
24.         if (WIFEXITED(status)) {      //为真,说明子进程正常终止.
25.             printf("child exit with %d\n", WEXITSTATUS(status));
26.
27.         }
28.         if (WIFSIGNALED(status)) {    //为真,说明子进程是被信号终止.
29.
30.             printf("child kill with signal %d\n", WTERMSIG(status));
31.         }
32.
33.         printf("-----parent wait finish: %d\n", wpid);
34.     } else {
```

```
35.     perror("fork");
36.     return 1;
37. }
38.
39.     return 0;
40. }
```

编译运行，如下所示：

```
zhcode@ubuntu:~/Code/code93$ make zoom_test2
gcc zoom_test2.c -o zoom_test2 -Wall -g
zhcode@ubuntu:~/Code/code93$ ./zoom_test2
---child, my id= 4257, going to sleep 10s
-----child die-----
child exit with 73
-----parent wait finish: 4257
zhcode@ubuntu:~/Code/code93$
```

这是子进程正常退出的情况。

下面发送信号使得子进程异常退出。

```
zhcode@ubuntu:~/Code/code93$ ./zoom_test2
---child, my id= 4301, going to sleep 10s
child kill with signal 9
-----parent wait finish: 4301
zhcode@ubuntu:~/Code/code93$
```

```
zhcode@ubuntu:~$ kill -9 4301
zhcode@ubuntu:~$
```

再测试一波，这次发送信号 11 试试

```
zhcode@ubuntu:~/Code/code93$ ./zoom_test2
---child, my id= 4320, going to sleep 10s
child kill with signal 11
-----parent wait finish: 4320
zhcode@ubuntu:~/Code/code93$
```

```
zhcode@ubuntu:~$ kill -11 4320
zhcode@ubuntu:~$
```

95P-waitpid 回收子进程

waitpid 函数：指定某一个进程进行回收。可以设置非阻塞。

```
waitpid(-1, &status, 0) == wait(&status);
```

```
pid_t waitpid(pid_t pid, int *status, int options)
```

参数：

pid：指定回收某一个子进程 pid

> 0：待回收的子进程 pid

-1：任意子进程

0：同组的子进程。

status：（传出） 回收进程的状态。

options：WNOHANG 指定回收方式为，非阻塞。

返回值：

> 0：表成功回收的子进程 pid

0：函数调用时， 参 3 指定了 WNOHANG， 并且，没有子进程结束。

-1：失败。errno

一次 wait/waitpid 函数调用，只能回收一个子进程。上一个例子，父进程产生了 5 个子进程，wait 会随机回收一个，捡到哪个算哪个。

96P-中午回顾

ps ajx --> pid ppid gid sid

97P-错误解析

在演示回收指定子进程的代码时出了问题，这里问题原因在于指定子进程的 pid 传递。父进程里的 pid 变量和子进程 pid 变量并不是同一个。子进程结束时，父进程的 pid 还是原来的 0。

原来的代码没有使用 fork 的返回值，导致父进程没有得到指定回收子进程的 pid。

默认情况下，父进程 fork 出来的子进程都属于同一个组。

pid_t waitpid(pid_t pid, int *status, int options)

参数：

pid: 指定回收某一个子进程 pid

> 0: 待回收的子进程 pid

-1: 任意子进程

0: 同组的子进程。

status: (传出) 回收进程的状态。

options: WNOHANG 指定回收方式为，非阻塞。

错误代码如下图所示，就不运行了，它不能回收指定的第 3 个子进程，每次

```
1. //指定回收一个子进程错误示例
2. #include <stdio.h>
3. #include <stdlib.h>
4. #include <string.h>
5. #include <unistd.h>
6. #include <sys/wait.h>
7. #include <pthread.h>
8.
9. int main(int argc, char *argv[])
10. {
11.     int i;
12.     pid_t pid, wpid;
13.
14.     for (i = 0; i < 5; i++) {
15.         if (fork() == 0) {           // 循环期间，子进程不 fork
16.             if (i == 2) {
17.                 pid = getpid();
18.                 printf("-----pid = %d\n", pid);
19.             }
20.             break;
21.         }
22.     }
23.
24.     if (5 == i) {                   // 父进程，从 表达式 2 跳出
```

```

25.     sleep(5);
26.
27.     //wait(NULL);                // 一次 wait/waitpid 函数调用,只能回收一个子进程.
28.     //wpid = waitpid(-1, NULL, WNOHANG); //回收任意子进程,没有结束的子进程,父进程直接返回 0
29.     //wpid = waitpid(pid, NULL, WNOHANG); //指定一个进程回收
30.
31.     printf("-----in parent , before waitpid, pid= %d\n", pid);
32.     wpid = waitpid(pid, NULL, 0); //指定一个进程回收
33.     if (wpid == -1) {
34.         perror("waitpid error");
35.         exit(1);
36.     }
37.     printf("I'm parent, wait a child finish : %d \n", wpid);
38.
39. } else {                // 子进程, 从 break 跳出
40.     sleep(i);
41.     printf("I'm %dth child, pid= %d\n", i+1, getpid());
42. }
43.
44. return 0;
45. }

```

编译并运行, 结果如下:

```

zhcode@ubuntu:~/Code/code93$ make loop_fork
gcc loop_fork.c -o loop_fork -Wall -g
zhcode@ubuntu:~/Code/code93$ ./loop_fork
-----pid = 4662
I'm 1th child, pid= 4660
I'm 2th child, pid= 4661
I'm 3th child, pid= 4662
I'm 4th child, pid= 4663
I'm 5th child, pid= 4664
-----in parent , before waitpid, pid= 0
I'm parent, wait a child finish : 4660
zhcode@ubuntu:~/Code/code93$ ./loop_fork
-----pid = 4668
I'm 1th child, pid= 4666
I'm 2th child, pid= 4667
I'm 3th child, pid= 4668
I'm 4th child, pid= 4669
I'm 5th child, pid= 4670
-----in parent , before waitpid, pid= 0
I'm parent, wait a child finish : 4666

```

这个代码错误如之前所述, 父进程里的 pid 还是 0, 因为父进程里没有获取指定子进程的 pid, 于是父进程里的 pid 还保持默认值。

下面是正确的示例，循环 fork 出 5 个子进程，并回收指定的子进程：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <sys/wait.h>
6. #include <pthread.h>
7.
8.
9. int main(int argc, char *argv[])
10. {
11.     int i;
12.     pid_t pid, wpid, tmpid;
13.
14.     for (i = 0; i < 5; i++) {
15.         pid = fork();
16.         if (pid == 0) {          // 循环期间，子进程不 fork
17.             break;
18.         }
19.         if (i == 2) {
20.             tmpid = pid;
21.             printf("-----pid = %d\n", tmpid);
22.         }
23.     }
24.
25.     if (5 == i) {              // 父进程，从 表达式 2 跳出
26. //         sleep(5);
27.
28.         //wait(NULL);          // 一次 wait/waitpid 函数调用,只能回收一个子进程.
29.         //wpid = waitpid(-1, NULL, WNOHANG);    //回收任意子进程,没有结束的子进程,父进程直接返回 0
30.         //wpid = waitpid(tmpid, NULL, 0);      //指定一个进程回收，阻塞等待
31.         printf("i am parent , before waitpid, pid = %d\n", tmpid);
32.
33.         //wpid = waitpid(tmpid, NULL, WNOHANG); //指定一个进程回收，不阻塞
34.         wpid = waitpid(tmpid, NULL, 0);        //指定一个进程回收，阻塞回收
35.         if (wpid == -1) {
36.             perror("waitpid error");
37.             exit(1);
38.         }
39.         printf("I'm parent, wait a child finish : %d \n", wpid);
40.
41.     } else {                   // 子进程，从 break 跳出
42.         sleep(i);
43.         printf("I'm %dth child, pid= %d\n", i+1, getpid());
44.     }
45.
46.     return 0;
47. }
```

编译执行，如下图所示：

```
zhcode@ubuntu:~/Code/code93$ ./test_waitpid
-----pid = 4628
i am parent , before waitpid, pid = 4628
I'm 1th child, pid= 4626
I'm 2th child, pid= 4627
I'm 3th child, pid= 4628
I'm parent, wait a child finish : 4628
zhcode@ubuntu:~/Code/code93$ I'm 4th child, pid= 4629
I'm 5th child, pid= 4630
```

如上图，指定回收的第三个进程，就回收的第三个。这里实现由两种，一个是阻塞等待回收指定进程，一个是非阻塞，但是用 sleep 延时父进程，以保证待回收的指定子进程已经执行结束。上面这个代码使用的阻塞回收，这个问题在于终端提示符会和输出混杂在一起。下面使用非阻塞回收+延时的方法，这样终端提示符就不会混在输出里。

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <sys/wait.h>
6. #include <pthread.h>
7.
8.
9. int main(int argc, char *argv[])
10. {
11.     int i;
12.     pid_t pid, wpid, tmpid;
13.
14.     for (i = 0; i < 5; i++) {
15.         pid = fork();
16.         if (pid == 0) {          // 循环期间，子进程不 fork
17.             break;
18.         }
19.         if (i == 2) {
20.             tmpid = pid;
21.             printf("-----pid = %d\n", tmpid);
22.         }
23.     }
24.
25.     if (5 == i) {              // 父进程，从 表达式 2 跳出
26.         sleep(5);
27.
28.         //wait(NULL);          // 一次 wait/waitpid 函数调用,只能回收一个子进程.
29.         //wpid = waitpid(-1, NULL, WNOHANG);    //回收任意子进程,没有结束的子进程,父进程直接返回 0
30.         //wpid = waitpid(tmpid, NULL, 0);       //指定一个进程回收，阻塞等待
31.         printf("i am parent , before waitpid, pid = %d\n", tmpid);
32.
33.         wpid = waitpid(tmpid, NULL, WNOHANG);    //指定一个进程回收，不阻塞
```

```

34.         //wpid = waitpid(tmpid, NULL, 0);           //指定一个进程回收, 阻塞回收
35.         if (wpid == -1) {
36.             perror("waitpid error");
37.             exit(1);
38.         }
39.         printf("I'm parent, wait a child finish : %d \n", wpid);
40.
41.     } else {           // 子进程, 从 break 跳出
42.         sleep(i);
43.         printf("I'm %dth child, pid= %d\n", i+1, getpid());
44.     }
45.
46.     return 0;
47. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code93$ make nohang
gcc nohang.c -o nohang -Wall -g
zhcode@ubuntu:~/Code/code93$ ./nohang
-----pid = 4726
I'm 1th child, pid= 4724
I'm 2th child, pid= 4725
I'm 3th child, pid= 4726
I'm 4th child, pid= 4727
I'm 5th child, pid= 4728
i am parent , before waitpid, pid = 4726
I'm parent, wait a child finish : 4726
zhcode@ubuntu:~/Code/code93$

```

98P-waitpid 回收多个子进程

waitpid 函数：指定某一个进程进行回收。可以设置非阻塞。

```
waitpid(-1, &status, 0) == wait(&status);
```

```
pid_t waitpid(pid_t pid, int *status, int options)
```

参数：

pid: 指定回收某一个子进程 pid

> 0: 待回收的子进程 pid

-1: 任意子进程

0: 同组的子进程。

status: (传出) 回收进程的状态。

options: WNOHANG 指定回收方式为，非阻塞。

返回值：

> 0 : 表成功回收的子进程 pid

0 : 函数调用时， 参 3 指定了 WNOHANG， 并且，没有子进程结束。

-1: 失败。errno

一次 wait/waitpid 函数调用，只能回收一个子进程。上一个例子，父进程产生了 5 个子进程，wait 会随机回收一个，捡到哪个算哪个。

总结：

wait、waitpid 一次调用，回收一个子进程。

想回收多个。while

下面这个例子，循环回收多个子进程：

```
1. // 回收多个子进程
2. #include <stdio.h>
3. #include <stdlib.h>
4. #include <string.h>
5. #include <unistd.h>
6. #include <sys/wait.h>
7. #include <pthread.h>
8.
9. int main(int argc, char *argv[])
```

```

10. {
11.     int i;
12.     pid_t pid, wpid;
13.
14.     for (i = 0; i < 5; i++) {
15.         pid = fork();
16.         if (pid == 0) {          // 循环期间, 子进程不 fork
17.             break;
18.         }
19.     }
20.
21.     if (5 == i) {              // 父进程, 从 表达式 2 跳出
22.         /*
23.         while ((wpid = waitpid(-1, NULL, 0))) {    // 使用阻塞方式回收子进程
24.             printf("wait child %d \n", wpid);
25.         }
26.         */
27.         while ((wpid = waitpid(-1, NULL, WNOHANG)) != -1) {    //使用非阻塞方式,回收子进程.
28.             if (wpid > 0) {
29.                 printf("wait child %d \n", wpid);
30.             } else if (wpid == 0) {
31.                 sleep(1);
32.                 continue;
33.             }
34.         }
35.
36.     } else {                  // 子进程, 从 break 跳出
37.         sleep(i);
38.         printf("I'm %dth child, pid= %d\n", i+1, getpid());
39.     }
40.
41.     return 0;
42. }

```

编译运行, 结果如下:

```

zhcode@ubuntu:~/Code/code93$ make waitpid_while
gcc waitpid_while.c -o waitpid_while -Wall -g
zhcode@ubuntu:~/Code/code93$ ./waitpid_while
I'm 1th child, pid= 4953
wait child 4953
I'm 2th child, pid= 4954
wait child 4954
I'm 3th child, pid= 4955
I'm 4th child, pid= 4956
wait child 4955
wait child 4956
I'm 5th child, pid= 4957
wait child 4957
zhcode@ubuntu:~/Code/code93$ █

```


可见，子进程运行完了，父进程就回收了。

99P-wait 和 waitpid 总结

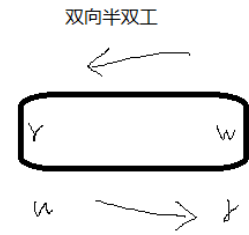
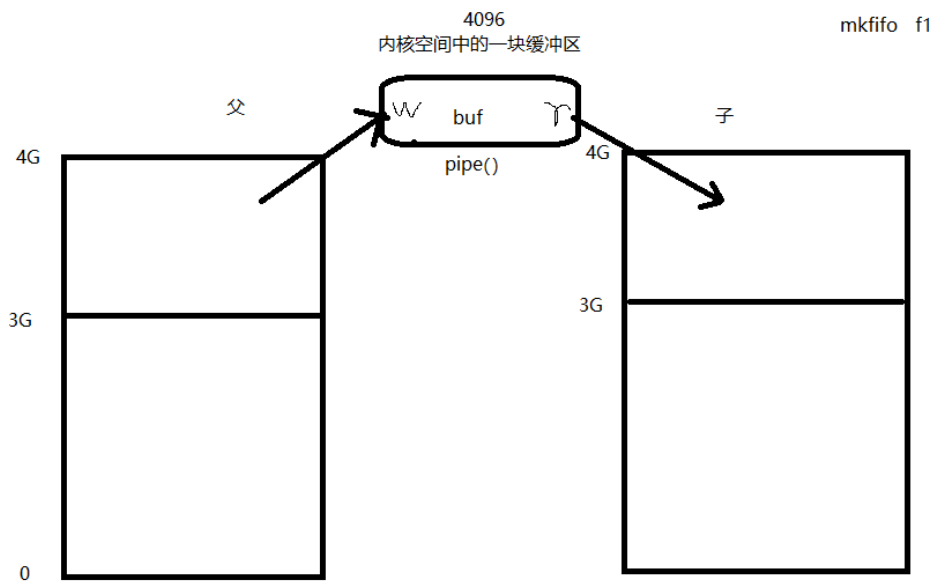
总结：

wait、waitpid 一次调用，回收一个子进程。

想回收多个。while

100P-进程间通信常见方式

IPC(InterProcess Communication)进程间通信



进程间通信的常用方式，特征：

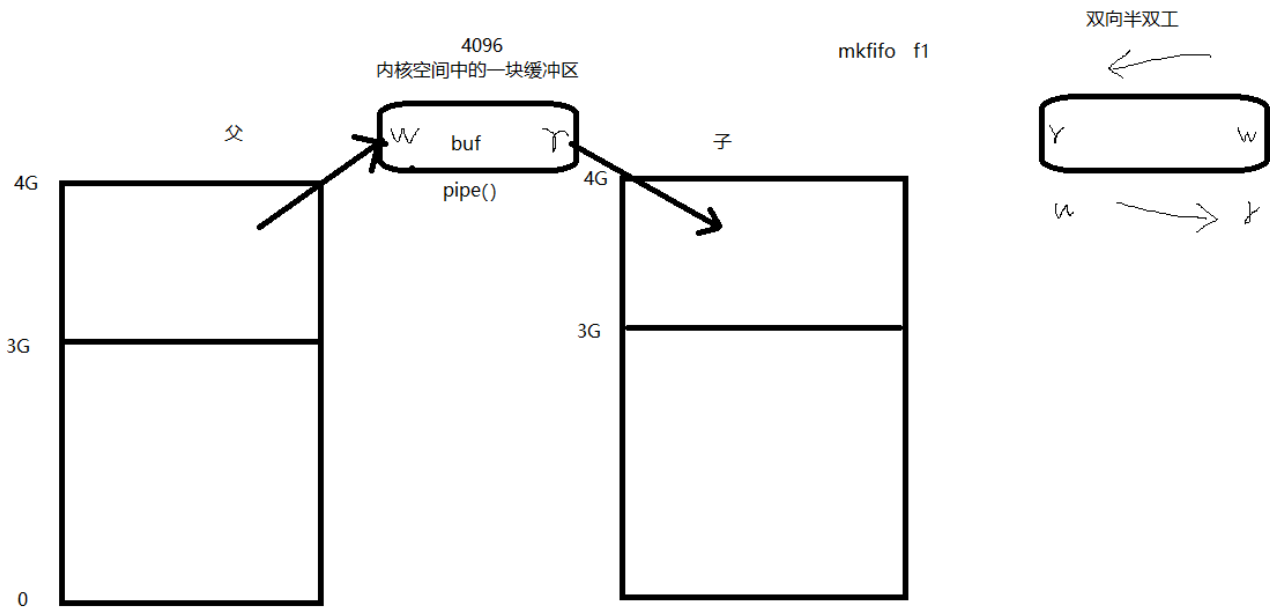
管道：简单

信号：开销小

mmap 映射：非血缘关系进程间

socket（本地套接字）：稳定

101P-管道的特质



管道：

实现原理： 内核借助环形队列机制，使用内核缓冲区实现。

特质： 1. 伪文件

2. 管道中的数据只能一次读取。

3. 数据在管道中，只能单向流动。

局限性： 1. 自己写，不能自己读。

2. 数据不可以反复读。

3. 半双工通信。

4. 血缘关系进程间可用。

102P-管道的基本用法

pipe 函数： 创建，并打开管道。

```
int pipe(int fd[2]);
```

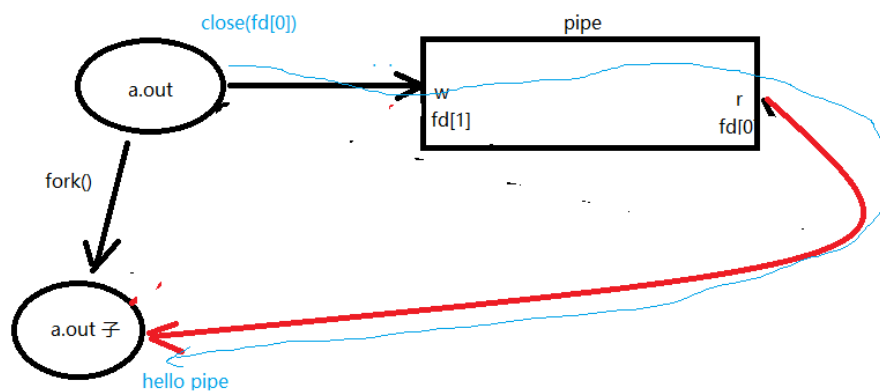
参数： fd[0]：读端。

fd[1]：写端。

返回值： 成功： 0

失败： -1 errno

管道通信原理：



一个管道通信的示例，父进程往管道里写，子进程从管道读，然后打印读取的内容：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void sys_err(const char *str)
9. {
10.     perror(str);
11.     exit(1);
12. }
13.
14. int main(int argc, char *argv[])
15. {
16.     int ret;
17.     int fd[2];
18.     pid_t pid;
19.
20.     char *str = "hello pipe\n";
```

```

21.     char buf[1024];
22.
23.     ret = pipe(fd);
24.     if (ret == -1)
25.         sys_err("pipe error");
26.
27.     pid = fork();
28.     if (pid > 0) {
29.         close(fd[0]);        // 关闭读段
30.         //sleep(3);
31.         write(fd[1], str, strlen(str));
32.         close(fd[1]);
33.     } else if (pid == 0) {
34.         close(fd[1]);        // 子进程关闭写段
35.         ret = read(fd[0], buf, sizeof(buf));
36.         printf("child read ret = %d\n", ret);
37.         write(STDOUT_FILENO, buf, ret);
38.
39.         close(fd[0]);
40.     }
41.
42.     return 0;
43. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code93$ make pipe
gcc pipe.c -o pipe -Wall -g
zhcode@ubuntu:~/Code/code93$ ./pipe
zhcode@ubuntu:~/Code/code93$ child read ret = 11
hello pipe

```

要是不想让终端提示和输出混杂在一起，就在父进程写入内容之后 sleep 一秒钟。

103P-管道读写行为

管道的读写行为：

读管道：

1. 管道有数据，read 返回实际读到的字节数。
2. 管道无数据：
 - 1) 无写端，read 返回 0 （类似读到文件尾）
 - 2) 有写端，read 阻塞等待。

写管道：

1. 无读端， 异常终止。（SIGPIPE 导致的）
2. 有读端：
 - 1) 管道已满， 阻塞等待
 - 2) 管道未满， 返回写出的字节个数。

104P-父子进程通信练习分析

练习：使用管道实现父子进程间通信，完成：`ls | wc -l` 假定父进程实现 `ls`，子进程实现 `wc`
`ls` 命令正常会将结果集写到 `stdout`，但现在会写入管道写端
`wc -l` 命令正常应该从 `stdin` 读取数据，但此时会从管道的读端读。

要用到 `pipe` `dup2` `exec`

105P-总结

gdb 调试:

设置父进程调试路径: `set follow-fork-mode parent` (默认)

设置子进程调试路径: `set follow-fork-mode child`

exec 函数族:

使进程执行某一程序。成功无返回值, 失败返回 -1

`int execlp(const char *file, const char *arg, ...);` 借助 PATH 环境变量找寻待执行程序

参 1: 程序名

参 2: `argv0`

参 3: `argv1`

...: `argvN`

哨兵: `NULL`

`int execl(const char *path, const char *arg, ...);` 自己指定待执行程序路径。

`int execvp();`

`ps ajx --> pid ppid gid sid`

孤儿进程:

父进程先于子进程终止, 子进程沦为“孤儿进程”, 会被 `init` 进程领养。

僵尸进程:

子进程终止, 父进程尚未对子进程进行回收, 在此期间, 子进程为“僵尸进程”。
`kill` 对其无效。

`wait` 函数: 回收子进程退出资源, 阻塞回收任意一个。

`pid_t wait(int *status)`

参数: (传出) 回收进程的状态。

返回值：成功： 回收进程的 pid

失败： -1, errno

函数作用 1： 阻塞等待子进程退出

函数作用 2： 清理子进程残留在内核的 pcb 资源

函数作用 3： 通过传出参数，得到子进程结束状态

获取子进程正常终止值：

WIFEXITED(status) --》 为真 --》调用 WEXITSTATUS(status) --》 得到 子进程 退出值。

获取导致子进程异常终止信号：

WIFSIGNALED(status) --》 为真 --》调用 WTERMSIG(status) --》 得到 导致子进程异常终止的信号编号。

waitpid 函数： 指定某一个进程进行回收。可以设置非阻塞。waitpid(-1, &status, 0)
== wait(&status);

pid_t waitpid(pid_t pid, int *status, int options)

参数：

pid: 指定回收某一个子进程 pid

> 0: 待回收的子进程 pid

-1: 任意子进程

0: 同组的子进程。

status: (传出) 回收进程的状态。

options: WNOHANG 指定回收方式为，非阻塞。

返回值：

> 0 : 表成功回收的子进程 pid

0 : 函数调用时， 参 3 指定了 WNOHANG， 并且，没有子进程结束。

-1: 失败。errno

总结：

wait、waitpid 一次调用，回收一个子进程。

想回收多个。while

=====

进程间通信的常用方式，特征：

管道：简单

信号：开销小

mmap 映射：非血缘关系进程间

socket（本地套接字）：稳定

管道：

实现原理： 内核借助环形队列机制，使用内核缓冲区实现。

特质； 1. 伪文件

2. 管道中的数据只能一次读取。

3. 数据在管道中，只能单向流动。

局限性：1. 自己写，不能自己读。

2. 数据不可以反复读。

3. 半双工通信。

4. 血缘关系进程间可用。

pipe 函数： 创建，并打开管道。

```
int pipe(int fd[2]);
```

参数： fd[0]：读端。

fd[1]：写端。

返回值： 成功： 0

失败： -1 errno

管道的读写行为：

读管道：

1. 管道有数据，read 返回实际读到的字节数。
2. 管道无数据：
 - 1) 无写端，read 返回 0 （类似读到文件尾）
 - 2) 有写端，read 阻塞等待。

写管道：

1. 无读端， 异常终止。（SIGPIPE 导致的）
2. 有读端：
 - 1) 管道已满， 阻塞等待
 - 2) 管道未满， 返回写出的字节个数。

106P-复习

普通文件，目录，软链接，这三个要占磁盘空间

管道，套接字，字符设备，块设备，不占磁盘空间，伪文件

107P-父子进程 lswc-l

练习：使用管道实现父子进程间通信，完成：ls | wc -l 假定父进程实现 ls，子进程实现 wc
ls 命令正常会将结果集写到 stdout，但现在会写入管道写端
wc -l 命令正常应该从 stdin 读取数据，但此时会从管道的读端读。

要用到 pipe dup2 exec

代码如下，还是蛮简单的：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void sys_err(const char *str)
9. {
10.     perror(str);
11.     exit(1);
12. }
13. int main(int argc, char *argv[])
14. {
15.     int fd[2];
16.     int ret;
17.     pid_t pid;
18.
19.     ret = pipe(fd);                // 父进程先创建一个管道,持有管道的读端和写端
20.     if (ret == -1) {
21.         sys_err("pipe error");
22.     }
23.
24.     pid = fork();                  // 子进程同样持有管道的读和写端
25.     if (pid == -1) {
26.         sys_err("fork error");
27.     }
28.     else if (pid > 0) {            // 父进程 读, 关闭写端
29.         close(fd[1]);
30.         dup2(fd[0], STDIN_FILENO); // 重定向 stdin 到 管道的 读端
31.         execlp("wc", "wc", "-l", NULL); // 执行 wc -l 程序
32.         sys_err("execlp wc error");
33.     }
34.     else if (pid == 0) {
35.         close(fd[0]);
36.         dup2(fd[1], STDOUT_FILENO); // 重定向 stdout 到 管道写端
37.         execlp("ls", "ls", NULL);   // 子进程执行 ls 命令
38.         sys_err("execlp ls error");
39.     }
```

```
40.  
41.     return 0;  
42. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code107$ make ls-wc-l  
gcc ls-wc-l.c -o ls-wc-l -Wall -g  
zhcode@ubuntu:~/Code/code107$ ./ls-wc-l  
4  
zhcode@ubuntu:~/Code/code107$
```

直接执行命令，如下：

```
zhcode@ubuntu:~/Code/code107$ ls | wc -l  
4  
zhcode@ubuntu:~/Code/code107$
```

其实代码和题目要求有出入，但是，问题不大，就这样吧。

108P-兄弟进程间通信

练习题：兄弟进程间通信

兄：ls

弟：wc -l

父：等待回收子进程

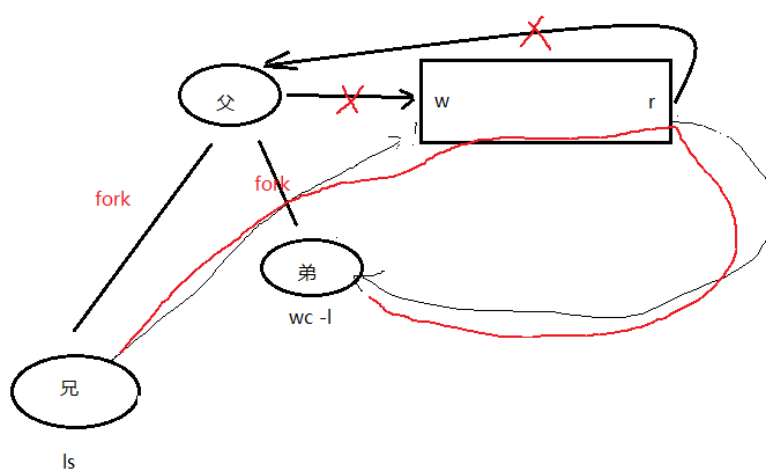
要求，使用循环创建 N 个子进程模型创建兄弟进程，使用循环因子 i 标识，注意管道读写行为

测试：

是否允许，一个 pipe 有一个写端多个读端 可

是否允许，一个 pipe 有多个写端一个读端 可

课后作业，统计当前系统中进程 ID 大于 10000 的进程个数



代码如下：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <sys/wait.h>
5. #include <unistd.h>
6. #include <errno.h>
7. #include <pthread.h>
8.
9. void sys_err(const char *str)
10. {
11.     perror(str);
12.     exit(1);
13. }
14. int main(int argc, char *argv[])
15. {
16.     int fd[2];
17.     int ret, i;
18.     pid_t pid;
19.
```



```

20.     ret = pipe(fd);
21.     if (ret == -1) {
22.         sys_err("pipe error");
23.     }
24.
25.     for(i = 0; i < 2; i++) {          // 表达式 2 出口,仅限父进程使用
26.         pid = fork();
27.         if (pid == -1) {
28.             sys_err("fork error");
29.         }
30.         if (pid == 0)                 // 子进程,出口
31.             break;
32.     }
33.
34.     if (i == 2) {                     // 父进程 . 不参与管道使用.
35.         close(fd[0]);                // 关闭管道的 读端/写端.
36.         close(fd[1]);
37.
38.         wait(NULL);                  // 回收子进程
39.         wait(NULL);
40.     } else if (i == 0) { // xiong
41.         close(fd[0]);
42.         dup2(fd[1], STDOUT_FILENO);  // 重定向 stdout
43.         execlp("ls", "ls", NULL);
44.         sys_err("execlp ls error");
45.     } else if (i == 1) { // 弟弟
46.         close(fd[1]);
47.         dup2(fd[0], STDIN_FILENO);   // 重定向 stdin
48.         execlp("wc", "wc", "-l", NULL);
49.         sys_err("execlp wc error");
50.     }
51.
52.     return 0;
53. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code107$ make pipe_broth
gcc pipe_broth.c -o pipe_broth -Wall -g
zhcode@ubuntu:~/Code/code107$ ./pipe_broth
6
zhcode@ubuntu:~/Code/code107$ ls | wc -l
6
zhcode@ubuntu:~/Code/code107$ █

```

这个代码需要注意一点，父进程不使用管道，所以一定要关闭父进程的管道，保证数据单向流动。

109P-多个读写端操作管道和管道缓冲区大小

下面是一个父进程读，俩子进程写的例子，也就是一个读端多个写端。需要调控写入顺序才行。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <sys/wait.h>
4. #include <string.h>
5. #include <stdlib.h>
6.
7. int main(void)
8. {
9.     pid_t pid;
10.    int fd[2], i, n;
11.    char buf[1024];
12.
13.    int ret = pipe(fd);
14.    if(ret == -1){
15.        perror("pipe error");
16.        exit(1);
17.    }
18.
19.    for(i = 0; i < 2; i++){
20.        if((pid = fork()) == 0)
21.            break;
22.        else if(pid == -1){
23.            perror("pipe error");
24.            exit(1);
25.        }
26.    }
27.
28.    if (i == 0) {
29.        close(fd[0]);
30.        write(fd[1], "1.hello\n", strlen("1.hello\n"));
31.    } else if(i == 1) {
32.        close(fd[0]);
33.        write(fd[1], "2.world\n", strlen("2.world\n"));
34.    } else {
35.        close(fd[1]); //父进程关闭写端,留读端读取数据
36.        sleep(1);
37.        n = read(fd[0], buf, 1024); //从管道中读数据
38.        write(STDOUT_FILENO, buf, n);
39.
40.        for(i = 0; i < 2; i++) //两个儿子 wait 两次
41.            wait(NULL);
42.    }
43.
44.    return 0;
45. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code107$ make pipe3
gcc pipe3.c -o pipe3 -Wall -g
zhcode@ubuntu:~/Code/code107$ ./pipe3
1.hello
2.world
zhcode@ubuntu:~/Code/code107$
```

这个例子需要注意，父进程必须等一下，不然可能俩子进程只写了一个，父进程就读完跑路了。

管道大小，默认 4096

```
zhcode@ubuntu:~/Code/code107$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 7722
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 7722
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
zhcode@ubuntu:~/Code/code107$
```

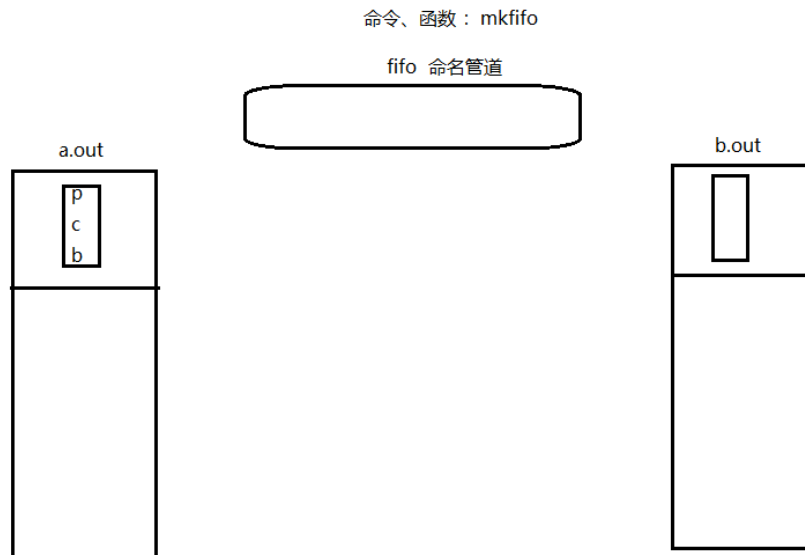
110P-命名管道 fifo 的创建和原理图

管道的优劣：

优点：简单，相比信号，套接字实现进程通信，简单很多

缺点：1. 只能单向通信，双向通信需建立两个管道

2. 只能用于有血缘关系的进程间通信。该问题后来使用 fifo 命名管道解决。



fifo 管道：可以用于无血缘关系的进程间通信。

命名管道：mkfifo

无血缘关系进程间通信：

读端，open fifo O_RDONLY

写端，open fifo O_WRONLY

fifo 操作起来像文件

下面的代码创建一个 fifo：

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/stat.h>
6 #include <errno.h>
7 #include <pthread.h>
8
9 void sys_err(const char *str)
10 {
11     perror(str);
12     exit(1);
13 }
14
15 int main(int argc, char *argv[])
16 {
17     int ret = mkfifo("testfifo", 0664);
18     if (ret == -1)
19         sys_err("mkfifo error");
20
21     return 0;
22 }
23

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code109$ ls
headcode.c  makefile  mytestfifo.c
zhcode@ubuntu:~/Code/code109$ make mytestfifo
gcc mytestfifo.c -o mytestfifo -Wall -g
zhcode@ubuntu:~/Code/code109$ ./mytestfifo
zhcode@ubuntu:~/Code/code109$ ls
headcode.c  makefile  mytestfifo  mytestfifo.c  testfifo
zhcode@ubuntu:~/Code/code109$

```

如图，管道就通过程序创建出来了。

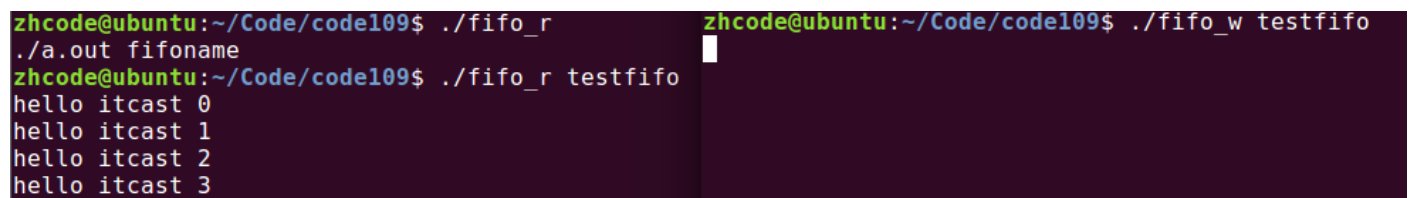
111P-fifo 实现非血缘关系进程间通信

下面这个例子，一个写 fifo，一个读 fifo，操作起来就像文件一样的：

```
15 int main(int argc, char *argv[])
16 {
17     int fd, i;
18     char buf[4096];
19
20     if (argc < 2) {
21         printf("Enter like this: ./a.out fifoname\n");
22         return -1;
23     }
24     fd = open(argv[1], O_WRONLY);          //打开管道文件
25     if (fd < 0)
26         sys_err("open");
27
28     i = 0;
29     while (1) {
30         sprintf(buf, "hello itcast %d\n", i++);
31
32         write(fd, buf, strlen(buf));      // 向管道写数据
33         sleep(1);
34     }
35     close(fd);
36
37     return 0;
38 }
39
"fifo_w.c" 39L, 599C written
```

```
15 int main(int argc, char *argv[])
16 {
17     int fd, len;
18     char buf[4096];
19
20     if (argc < 2) {
21         printf("./a.out fifoname\n");
22         return -1;
23     }
24     //int fd = mkfifo("testfifo", 644);
25     //open(fd, ...);
26     fd = open(argv[1], O_RDONLY);        // 打开管道文件
27     if (fd < 0)
28         sys_err("open");
29     while (1) {
30         len = read(fd, buf, sizeof(buf)); // 从管道的读端获取数据
31         write(STDOUT_FILENO, buf, len);
32         sleep(3);                         //多个读端时应增加睡眠秒数,放大效果.
33     }
34     close(fd);
35
36     return 0;
37 }
38
"fifo_r.c" 38L, 696C written
```

编译执行，如图：



```
zhcode@ubuntu:~/Code/code109$ ./fifor
./a.out fifoname
zhcode@ubuntu:~/Code/code109$ ./fifor testfifo
hello itcast 0
hello itcast 1
hello itcast 2
hello itcast 3
zhcode@ubuntu:~/Code/code109$ ./fifo_w testfifo
hello itcast 0
hello itcast 1
hello itcast 2
hello itcast 3
```

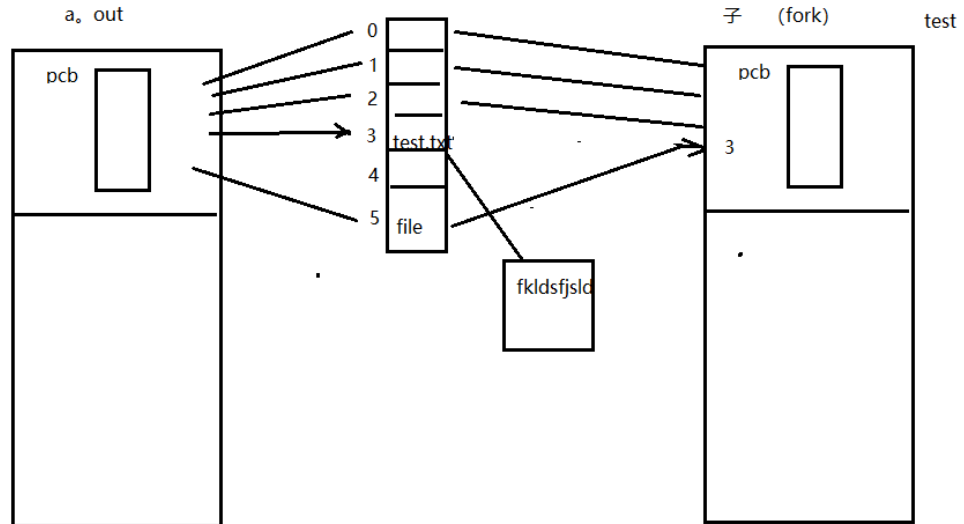
下面测试多个写管道，一个读管道，就是多开两个 `fifo.w`，就一个 `fifo.r`，这是可以的，懒得做了，就这样吧。

测试一个写端多个读端的时候，由于数据一旦被读走就没了，所以多个读端的并集才是写端的写入数据。

112P-文件用于进程间通信

文件实现进程间通信：

打开的文件是内核中的一块缓冲区。多个无血缘关系的进程，可以同时访问该文件。



文件通信这个，有没有血缘关系都行，只是有血缘关系的进程对于同一个文件，使用的同一个文件描述符，没有血缘关系的进程，对同一个文件使用的文件描述符可能不同。这些都不是问题，打开的是同一个文件就行。

113P-mmap 函数原型

存储映射 I/O (Memory-mapped I/O) 使一个磁盘文件与存储空间中的一个缓冲区相映射。于是从缓冲区中取数据，就相当于读文件中的相应字节。与此类似，将数据存入缓冲区，则相应的字节就自动写入文件。这样，就可在不使用 read 和 write 函数的情况下，使地址指针完成 I/O 操作。

使用这种方法，首先应该通知内核，将一个指定文件映射到存储区域中。这个映射工作可以通过 mmap 函数来实现。

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

创建共享内存映射

参数：

addr: 指定映射区的首地址。通常传 NULL，表示让系统自动分配

length: 共享内存映射区的大小。（<= 文件的实际大小）

prot: 共享内存映射区的读写属性。PROT_READ、PROT_WRITE、PROT_READ|PROT_WRITE

flags: 标注共享内存的共享属性。MAP_SHARED、MAP_PRIVATE

fd: 用于创建共享内存映射区的那个文件的 文件描述符。

offset: 默认 0，表示映射文件全部。偏移位置。需是 4k 的整数倍。

返回值：

成功：映射区的首地址。

失败：MAP_FAILED (void*(-1))， errno

flags 里面的 shared 意思是修改会反映到磁盘上
private 表示修改不反映到磁盘上

```
int munmap(void *addr, size_t length); 释放映射区。
```

addr: mmap 的返回值

length: 大小

114P-复习

115P-mmap 建立映射区

下面这个示例代码，使用 mmap 创建一个映射区（共享内存），并往映射区里写入内容：

```
14 int main(int argc, char *argv[]){
15     char *p = NULL;
16     int fd;
17
18     fd = open("testmap", O_RDWR|O_CREAT|O_TRUNC, 0644);
19
20     if(fd == -1)
21         sys_err("open error");
22 /*
23     lseek(fd, 10, SEEK_END);    // this two function equal to ftruncate
24     write(fd, "\0", 1);
25 */
26     ftruncate(fd, 20);
27     int len = lseek(fd, 0, SEEK_END);
28     p = mmap(NULL, len, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
29     if(p == MAP_FAILED)
30         sys_err("mmap error");
31     strcpy(p, "hello mmap");    // use p to write file
32     printf("-----%s\n", p);
33     int ret = munmap(p, len);
34     if(ret == -1)        sys_err("munmap error");
35     return 0;
36 }
"mmap.c" 36L, 777C written
```

编译运行，如下所示：

```
zhcode@ubuntu:~/Code/code115$ make mmap
gcc mmap.c -o mmap -Wall -g
zhcode@ubuntu:~/Code/code115$ ./mmap
-----hello mmap
zhcode@ubuntu:~/Code/code115$ ls
headcode.c  makefile  mmap  mmap.c  testmap
zhcode@ubuntu:~/Code/code115$ cat testmap
hello mmapzhcode@ubuntu:~/Code/code115$
```

116P-mmap 使用注意事项 1

使用注意事项：

1. 用于创建映射区的文件大小为 0，实际指定非 0 大小创建映射区，出 “总线错误”。
2. 用于创建映射区的文件大小为 0，实际制定 0 大小创建映射区，出 “无效参数”。
3. 用于创建映射区的文件读写属性为，只读。映射区属性为 读、写。出 “无效参数”。
4. 创建映射区，需要 read 权限。当访问权限指定为 “共享” MAP_SHARED 时， mmap 的读写权限，应该 <=文件的 open 权限。只写不行。
5. 文件描述符 fd，在 mmap 创建映射区完成即可关闭。后续访问文件，用 地址访问。
6. offset 必须是 4096 的整数倍。（MMU 映射的最小单位 4k ）
7. 对申请的映射区内存，不能越界访问。
8. munmap 用于释放的 地址，必须是 mmap 申请返回的地址。
9. 映射区访问权限为 “私有” MAP_PRIVATE，对内存所做的所有修改，只在内存有效，不会反应到物理磁盘上。
10. 映射区访问权限为 “私有” MAP_PRIVATE，只需要 open 文件时，有读权限，用于创建映射区即可。

117P-mmap 使用注意事项 2

都写在 116P 了，看上面就行。

mmap 函数的保险调用方式：

1. `fd = open ("文件名", O_RDWR);`
2. `mmap(NULL, 有效文件大小, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);`

118P-mmap 总结

1. 创建映射区的过程中，隐含着一次对映射文件的读操作
2. 当 MAP_SHARED 时，要求：映射区的权限应该 $\leq$ 文件打开的权限（出于对映射区的保护）。而 MAP_PRIVATE 则无所谓，因为 mmap 中的权限是对内存的限制
3. 映射区的释放与文件关闭无关。只要映射建立成功，文件可以立即关闭
4. 特别注意，当映射文件大小为 0 时，不能创建映射区。所以：用于映射的文件必须要有实际大小!!
mmap 使用时常常会出现总线错误，通常是由于共享文件存储空间大小引起的。如，400 字节大小的文件，在遍历映射区时，offset4096 字节，则会报出总线错误
5. munmap 传入的地址一定是 mmap 返回的地址。坚决杜绝指针++操作
6. 文件偏移量必须为 4K 的整数倍
7. mmap 创建映射区出错概率非常高，一定要检查返回值，确保映射区建立成功再进行后续操作。

119P-父子进程间 mmap 通信

父子进程使用 mmap 进程间通信：

父进程 先 创建映射区。 `open ( O_RDWR ) mmap( MAP_SHARED );`

指定 MAP_SHARED 权限

`fork()` 创建子进程。

一个进程读， 另外一个进程写。

下面这段代码，父子进程 mmap 通信，共享内存是一个 int 变量：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <unistd.h>
4. #include <fcntl.h>
5. #include <sys/mman.h>
6. #include <sys/wait.h>
7.
8. int var = 100;
9.
10. int main(void)
11. {
12.     int *p;
13.     pid_t pid;
14.
15.     int fd;
16.     fd = open("temp", O_RDWR|O_CREAT|O_TRUNC, 0644);
17.     if(fd < 0){
18.         perror("open error");
19.         exit(1);
20.     }
21.     ftruncate(fd, 4);
22.
23.     p = (int *)mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
24.     //p = (int *)mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_PRIVATE, fd, 0);
25.     if(p == MAP_FAILED){           //注意:不是 p == NULL
26.         perror("mmap error");
27.         exit(1);
28.     }
29.     close(fd);                     //映射区建立完毕,即可关闭文件
30.
31.     pid = fork();                  //创建子进程
32.     if(pid == 0){
33.         *p = 7000;                 // 写共享内存
34.         var = 1000;
35.         printf("child, *p = %d, var = %d\n", *p, var);
```

```

36.     } else {
37.         sleep(1);
38.         printf("parent, *p = %d, var = %d\n", *p, var);    // 读共享内存
39.         wait(NULL);
40.
41.         int ret = munmap(p, 4);    //释放映射区
42.         if (ret == -1) {
43.             perror("munmap error");
44.             exit(1);
45.         }
46.     }
47.
48.     return 0;
49. }

```

编译运行，如下所示：

```

zhcode@ubuntu:~/Code/code115$ make fork_mmap2
gcc fork_mmap2.c -o fork_mmap2 -Wall -g
zhcode@ubuntu:~/Code/code115$ ./fork_mmap2
child, *p = 7000, var = 1000
parent, *p = 7000, var = 100
zhcode@ubuntu:~/Code/code115$

```

如图，子进程修改 p 的值，也反映到了父进程上，这是因为共享内存定义为 shared 的。如果将共享内存定义为 private，运行结果如下：

```

zhcode@ubuntu:~/Code/code115$ make fork_mmap
gcc fork_mmap.c -o fork_mmap -Wall -g
zhcode@ubuntu:~/Code/code115$ ./fork_mmap
child, *p = 7000, var = 1000
parent, *p = 0, var = 100
zhcode@ubuntu:~/Code/code115$

```


120P-无血缘关系进程间 mmap 通信

无血缘关系进程间 mmap 通信：

【会写】

两个进程 打开同一个文件，创建映射区。

指定 flags 为 MAP_SHARED。

一个进程写入，另外一个进程读出。

【注意】：无血缘关系进程间通信。mmap：数据可以重复读取。

fifo：数据只能一次读取。

下面是两个无血缘关系的通信代码，先是写进程：

```
1. #include <stdio.h>
2. #include <sys/stat.h>
3. #include <sys/types.h>
4. #include <fcntl.h>
5. #include <unistd.h>
6. #include <stdlib.h>
7. #include <sys/mman.h>
8. #include <string.h>
9.
10. struct STU {
11.     int id;
12.     char name[20];
13.     char sex;
14. };
15.
16. void sys_err(char *str)
17. {
18.     perror(str);
19.     exit(1);
20. }
21.
22. int main(int argc, char *argv[])
23. {
24.     int fd;
25.     struct STU student = {10, "xiaoming", 'm'};
26.     char *mm;
27.
28.     if (argc < 2) {
29.         printf("./a.out file_shared\n");
30.         exit(-1);
31.     }
32.
```

```

33.     fd = open(argv[1], O_RDWR | O_CREAT, 0664);
34.     ftruncate(fd, sizeof(student));
35.
36.     mm = mmap(NULL, sizeof(student), PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
37.     if (mm == MAP_FAILED)
38.         sys_err("mmap");
39.
40.     close(fd);
41.
42.     while (1) {
43.         memcpy(mm, &student, sizeof(student));
44.         student.id++;
45.         sleep(1);
46.     }
47.
48.     munmap(mm, sizeof(student));
49.
50.     return 0;
51. }

```

然后是读进程：

```

1.  #include <stdio.h>
2.  #include <sys/stat.h>
3.  #include <fcntl.h>
4.  #include <unistd.h>
5.  #include <stdlib.h>
6.  #include <sys/mman.h>
7.  #include <string.h>
8.
9.  struct STU {
10.     int id;
11.     char name[20];
12.     char sex;
13. };
14.
15. void sys_err(char *str)
16. {
17.     perror(str);
18.     exit(-1);
19. }
20.
21. int main(int argc, char *argv[])
22. {
23.     int fd;
24.     struct STU student;
25.     struct STU *mm;
26.
27.     if (argc < 2) {

```

```

28.     printf("./a.out file_shared\n");
29.     exit(-1);
30. }
31.
32. fd = open(argv[1], O_RDONLY);
33. if (fd == -1)
34.     sys_err("open error");
35.
36. mm = mmap(NULL, sizeof(student), PROT_READ, MAP_SHARED, fd, 0);
37. if (mm == MAP_FAILED)
38.     sys_err("mmap error");
39.
40. close(fd);
41.
42. while (1) {
43.     printf("id=%d\tname=%s\t%c\n", mm->id, mm->name, mm->sex);
44.     sleep(2);
45. }
46. munmap(mm, sizeof(student));
47.
48. return 0;
49. }

```

编译并运行，结果如下：

```

zhcode@ubuntu:~/Code/code115$ make mmap_w      zhcode@ubuntu:~$ cd Code/code115
gcc mmap_w.c -o mmap_w -Wall -g               zhcode@ubuntu:~/Code/code115$ ./mmap_r mymmp
zhcode@ubuntu:~/Code/code115$ ./mmap_w mymmp  id=58   name=xiaoming   m
                                                id=60   name=xiaoming   m

```

如图，一读一写，问题不大。

多个写端一个读端也没问题，打开多个写进程即可，完事儿读进程会读到所有写进程写入的内容。这里要注意一个，内容被读走之后不会消失，所以如果读进程的读取时间间隔短，它会读到很多重复内容，就是因为写进程没来得及写入新内容。

121P-mmap 总结

无血缘关系进程间 mmap 通信：

【会写】

两个进程 打开同一个文件，创建映射区。

指定 flags 为 MAP_SHARED。

一个进程写入，另外一个进程读出。

【注意】：无血缘关系进程间通信。mmap：数据可以重复读取。

fifo：数据只能一次读取。

122P-mmap 匿名映射区

匿名映射：只能用于 血缘关系进程间通信。

```
p = (int *)mmap(NULL, 40, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
```

123P-总结

pipe 管道： 用于有血缘关系的进程间通信。 `ps aux | grep ls | wc -l`

父子进程间通信：

兄弟进程间通信：

fifo 管道： 可以用于无血缘关系的进程间通信。

命名管道： `mkfifo`

无血缘关系进程间通信：

读端，`open fifo O_RDONLY`

写端，`open fifo O_WRONLY`

文件实现进程间通信：

打开的文件是内核中的一块缓冲区。多个无血缘关系的进程，可以同时访问该文件。

共享内存映射：

`void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`
创建共享内存映射

参数：

addr： 指定映射区的首地址。通常传 NULL，表示让系统自动分配

length： 共享内存映射区的大小。（<= 文件的实际大小）

prot： 共享内存映射区的读写属性。PROT_READ、PROT_WRITE、PROT_READ|PROT_WRITE

flags： 标注共享内存的共享属性。MAP_SHARED、MAP_PRIVATE

fd： 用于创建共享内存映射区的那个文件的 文件描述符。

offset： 默认 0，表示映射文件全部。偏移位置。需是 4k 的整数倍。

返回值：

成功：映射区的首地址。

失败：MAP_FAILED (void*(-1)), errno

int munmap(void *addr, size_t length); 释放映射区。

addr: mmap 的返回值

length: 大小

使用注意事项：

1. 用于创建映射区的文件大小为 0，实际指定非 0 大小创建映射区，出 “总线错误”。
2. 用于创建映射区的文件大小为 0，实际制定 0 大小创建映射区，出 “无效参数”。
3. 用于创建映射区的文件读写属性为，只读。映射区属性为 读、写。出 “无效参数”。
4. 创建映射区，需要 read 权限。当访问权限指定为 “共享” MAP_SHARED 是，mmap 的读写权限，应该 <=文件的 open 权限。只写不行。
5. 文件描述符 fd，在 mmap 创建映射区完成即可关闭。后续访问文件，用 地址访问。
6. offset 必须是 4096 的整数倍。(MMU 映射的最小单位 4k)
7. 对申请的映射区内存，不能越界访问。
8. munmap 用于释放的 地址，必须是 mmap 申请返回的地址。
9. 映射区访问权限为 “私有” MAP_PRIVATE，对内存所做的所有修改，只在内存有效，不会反应到物理磁盘上。
10. 映射区访问权限为 “私有” MAP_PRIVATE，只需要 open 文件时，有读权限，用于创建映射区即可。

mmap 函数的保险调用方式：

1. fd = open (“文件名”, O_RDWR);
2. mmap(NULL, 有效文件大小, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

父子进程使用 mmap 进程间通信：

父进程 先 创建映射区。 open (O_RDWR) mmap(MAP_SHARED);

指定 MAP_SHARED 权限

fork() 创建子进程。

一个进程读， 另外一个进程写。

无血缘关系进程间 mmap 通信： **【会写】**

两个进程 打开同一个文件， 创建映射区。

指定 flags 为 MAP_SHARED。

一个进程写入， 另外一个进程读出。

【注意】： 无血缘关系进程间通信。mmap： 数据可以重复读取。

fifo： 数据只能一次读取。

匿名映射： 只能用于 血缘关系进程间通信。

```
p = (int *)mmap(NULL, 40, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
```


124P-复习

125P-信号的概念和机制

信号共性：

简单、不能携带大量信息、满足条件才发送。

信号的特质：

信号是软件层面上的“中断”。一旦信号产生，无论程序执行到什么位置，必须立即停止运行，处理信号，处理结束，再继续执行后续指令。

所有信号的产生及处理全部都是由【内核】完成的。

126P-与信号相关的概念

信号相关的概念：

产生信号：

1. 按键产生
2. 系统调用产生
3. 软件条件产生
4. 硬件异常产生
5. 命令产生

概念：

未决：产生与递达之间状态。

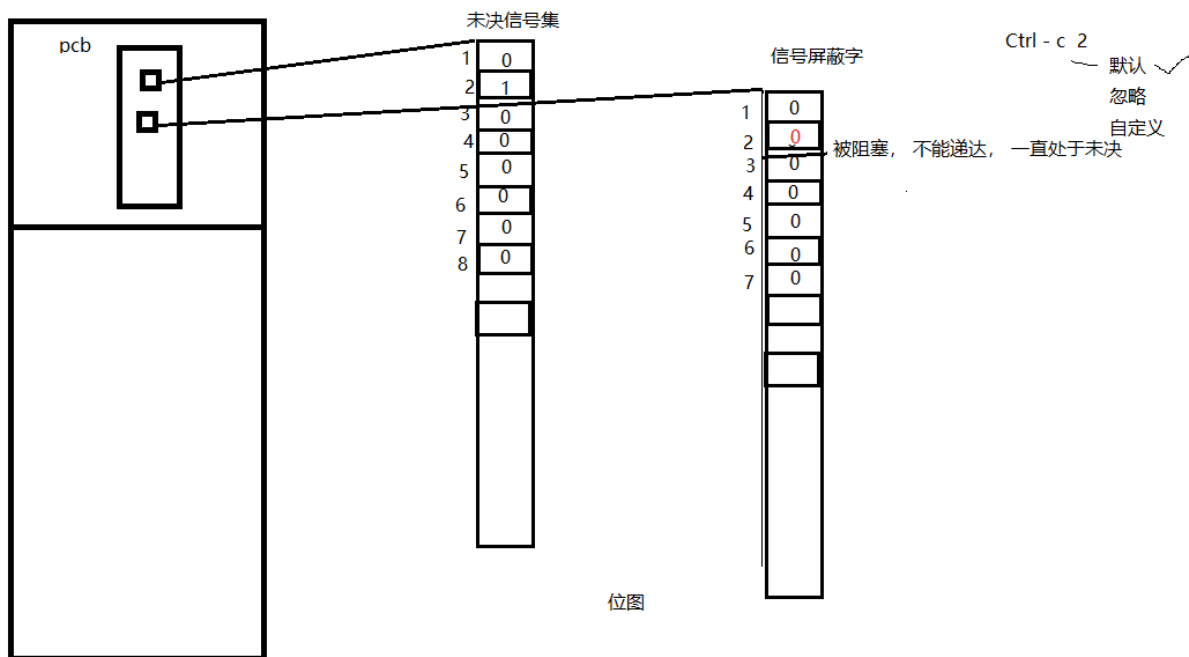
递达：产生并且送达到进程。直接被内核处理掉。

信号处理方式： 执行默认处理动作、忽略、捕捉（自定义）

阻塞信号集（信号屏蔽字）： 本质：位图。用来记录信号的屏蔽状态。一旦被屏蔽的信号，在解除屏蔽前，一直处于未决态。

未决信号集：本质：位图。用来记录信号的处理状态。该信号集中的信号，表示，已经产生，但尚未被处理。

127P-信号屏蔽字和未决信号集



概念：

未决：产生与递达之间状态。

递达：产生并且送达到进程。直接被内核处理掉。

信号处理方式： 执行默认处理动作、忽略、捕捉（自定义）

阻塞信号集（信号屏蔽字）： 本质：位图。用来记录信号的屏蔽状态。一旦被屏蔽的信号，在解除屏蔽前，一直处于未决态。

未决信号集：本质：位图。用来记录信号的处理状态。该信号集中的信号，表示，已经产生，但尚未被处理。

128P-信号四要素和常规信号一览

`kill -l` 查看当前系统中常规信号

信号 4 要素：

信号使用之前，应先确定其 4 要素，而后再用!!!

编号、名称、对应事件、默认处理动作。

129P-kill 函数和 kill 命令

kill 命令 和 kill 函数:

```
int kill (pid_t pid, int signum)
```

参数:

pid: > 0: 发送信号给指定进程

= 0: 发送信号给跟调用 kill 函数的那个进程处于同一进程组的进程。

< -1: 取绝对值, 发送信号给该绝对值所对应的进程组的所有组员。

= -1: 发送信号给, 有权限发送的所有进程。

signum: 待发送的信号

返回值:

成功: 0

失败: -1 errno

小例子, 子进程发送信号 kill 父进程:

```
10 void sys_err(const char *str) {
11     perror(str);
12     exit(1);
13 }
14
15 int main(int argc, char *argv[]){
16     pid_t pid = fork();
17
18     if(pid > 0){
19         printf("parent, pid = %d\n", getpid());
20         while(1);
21     }
22     else if(pid == 0){
23         printf("child pid = %d, ppid = %d\n", getpid(), getppid());
24         sleep(2);
25         kill(getppid(), SIGKILL);
26     }
27
28     return 0;
29 }
"kill.c" 29L, 484C written
```

编译运行, 结果如下:

```
zhcode@ubuntu:~/Code/code129$ make kill
gcc kill.c -o kill -Wall -g
zhcode@ubuntu:~/Code/code129$ ./kill
parent, pid = 3838
child pid = 3839, ppid = 3838
Killed
zhcode@ubuntu:~/Code/code129$
```

这里子进程不发送 kill 信号，发其他信号也行，比如段错误什么的。

kill -9 -groupname 杀一个进程组

130P-alarm 函数

alarm 函数：使用自然计时法。

定时发送 SIGALRM 给当前进程。

```
unsigned int alarm(unsigned int seconds);
```

seconds：定时秒数

返回值：上次定时剩余时间。

无错误现象。

alarm (0); 取消闹钟。

time 命令：查看程序执行时间。 实际时间 = 用户时间 + 内核时间 + 等待时间。 --》
优化瓶颈 IO

小例子就不写了，使用 alarm 函数计时，打印变量 i 的值。

131P-setitimer 函数

setitimer 函数:

```
int setitimer(int which, const struct itimerval *new_value, struct itimerval *old_value);
```

参数:

which: ITIMER_REAL: 采用自然计时。 ---> SIGALRM

ITIMER_VIRTUAL: 采用用户空间计时 ---> SIGVTALRM

ITIMER_PROF: 采用内核+用户空间计时 ---> SIGPROF

new_value: 定时秒数

类型: struct itimerval {

```
    struct timeval {  
        time_t      tv_sec;          /* seconds */  
        suseconds_t tv_usec;        /* microseconds */  
    } it_interval; ---> 周期定时秒数
```

```
    struct timeval {  
        time_t      tv_sec;  
        suseconds_t tv_usec;  
    } it_value; ---> 第一次定时秒数  
};
```

old_value: 传出参数, 上次定时剩余时间。

e. g.

```
struct itimerval new_t;  
struct itimerval old_t;  
  
new_t.it_interval.tv_sec = 0;  
new_t.it_interval.tv_usec = 0;  
new_t.it_value.tv_sec = 1;  
new_t.it_value.tv_usec = 0;
```

```
int ret = setitimer(&new_t, &old_t); 定时 1 秒
```

返回值:

成功: 0

失败: -1 errno

小例子，使用 `setitimer` 定时，向屏幕打印信息：

```
5 void myfunc(int signo)
6 {
7     printf("hello world\n");
8 }
9
10 int main(void)
11 {
12     struct itimerval it, oldit;
13
14     signal(SIGALRM, myfunc);    //注册SIGALRM信号的捕捉处理函数。
15
16     it.it_value.tv_sec = 2;
17     it.it_value.tv_usec = 0;
18
19     it.it_interval.tv_sec = 5;
20     it.it_interval.tv_usec = 0;
21
22     if(setitimer(ITIMER_REAL, &it, &oldit) == -1){
23         perror("setitimer error");
24         return -1;
25     }
26
27     while(1);
28
29     return 0;
30 }
31
```

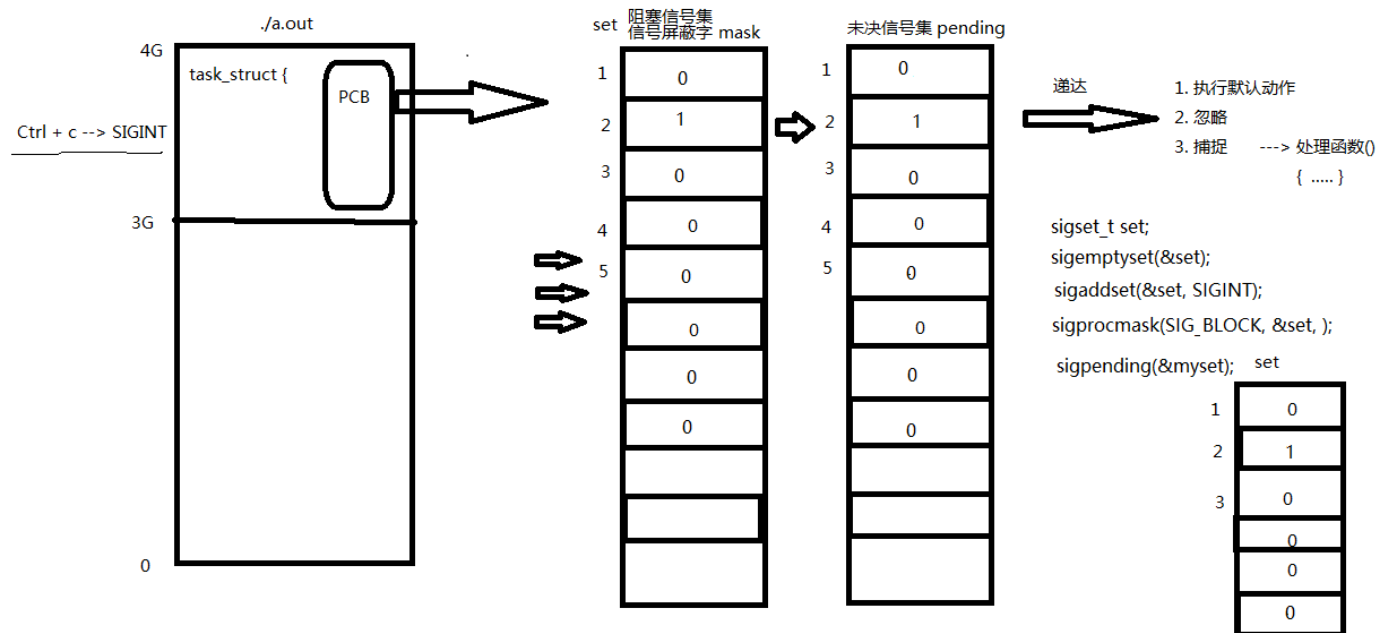
编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code129$ make setitimer
gcc setitimer.c -o setitimer -Wall -g
zhcode@ubuntu:~/Code/code129$ ./setitimer
hello world
hello world
hello world
^C
zhcode@ubuntu:~/Code/code129$
```

第一次信息打印是两秒间隔，之后都是 5 秒间隔打印一次。可以理解为第一次是有个定时器，什么时候触发打印，之后就是间隔时间。

132P-午后回顾

133P-信号集操作函数



信号集操作函数：

sigset_t set; 自定义信号集。

sigemptyset(sigset_t *set); 清空信号集

sigfillset(sigset_t *set); 全部置1

sigaddset(sigset_t *set, int signum); 将一个信号添加到集合中

sigdelset(sigset_t *set, int signum); 将一个信号从集合中移除

sigismember (const sigset_t *set, int signum); 判断一个信号是否在集合中。 在--》1, 不在--》0

设置信号屏蔽字和解除屏蔽：

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

how: SIG_BLOCK: 设置阻塞

SIG_UNBLOCK: 取消阻塞

SIG_SETMASK: 用自定义 set 替换 mask。

set: 自定义 set

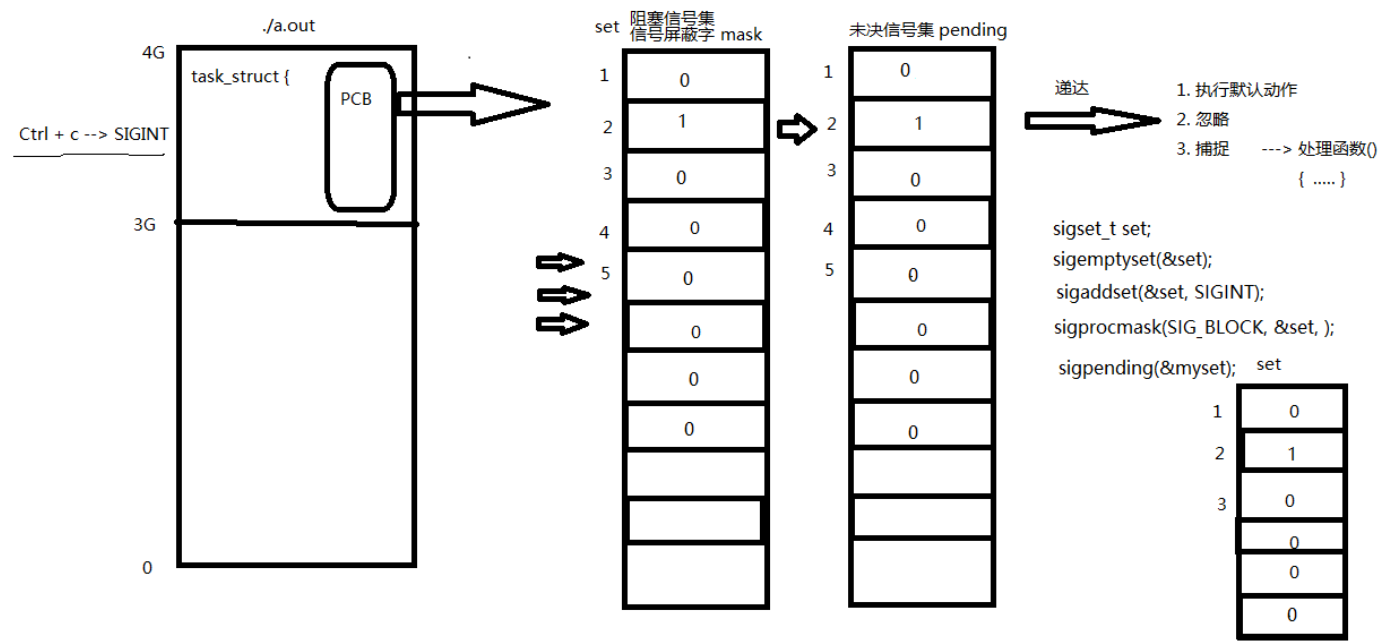
oldset: 旧有的 mask。

查看未决信号集：

```
int sigpending(sigset_t *set);
```

set: 传出的 未决信号集。

134P-信号操作函数使用原理分析



135P-信号集操作函数练习

信号列表：

```
zhcode@ubuntu:~/Code/code129$ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT      4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS      8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE     14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT     19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG      24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH    29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

其中 9 号和 19 号信号比较特殊，只能执行默认动作，不能忽略捕捉，不能设置阻塞。

下面这个小例子，利用自定义集合，来设置信号阻塞，我们输入被设置阻塞的信号，可以看到未决信号集发生变化：

```
1. #include <stdio.h>
2. #include <signal.h>
3. #include <stdlib.h>
4. #include <string.h>
5. #include <unistd.h>
6. #include <errno.h>
7. #include <pthread.h>
8.
9. void sys_err(const char *str)
10. {
11.     perror(str);
12.     exit(1);
13. }
14.
15. void print_set(sigset_t *set)
16. {
17.     int i;
18.     for (i = 1; i<32; i++) {
19.         if (sigismember(set, i))
20.             putchar('1');
21.         else
22.             putchar('0');
23.     }
24.     printf("\n");
25. }
26. int main(int argc, char *argv[])
27. {
28.     sigset_t set, oldset, pedset;
29.     int ret = 0;
```

```

30.
31.     sigemptyset(&set);
32.     sigaddset(&set, SIGINT);
33.     sigaddset(&set, SIGQUIT);
34.     sigaddset(&set, SIGBUS);
35.     sigaddset(&set, SIGKILL);
36.
37.     ret = sigprocmask(SIG_BLOCK, &set, &oldset);
38.     if (ret == -1)
39.         sys_err("sigprocmask error");
40.
41.     while (1) {
42.         ret = sigpending(&pedset);
43.         print_set(&pedset);
44.         sleep(1);
45.     }
46.
47.     return 0;
48. }

```

编译运行，如下图所示：

```

zhcode@ubuntu:~/Code/code129$ make sigfunc
gcc sigfunc.c -o sigfunc -Wall -g
zhcode@ubuntu:~/Code/code129$ ./sigfunc
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
^C0100000000000000000000000000000
01000000000000000000000000000000
01000000000000000000000000000000

```

可以看到，在输入 Ctrl+C 之后，进程捕捉到信号，但由于设置阻塞，没有处理，未决信号集对应位置变为 1。

136P-signal 实现信号捕捉

SYNOPSIS

```
#include <signal.h>

typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

参数:

signum : 待捕捉信号

handler: 捕捉信号后的操纵函数

返回值:

RETURN VALUE

signal() returns the previous value of the signal handler, or **SIG_ERR** on error. In the event of an error, **errno** is set to indicate the cause.

信号捕捉特性:

1. 捕捉函数执行期间, 信号屏蔽字 由 mask --> sa_mask , 捕捉函数执行结束。 恢复回 mask
2. 捕捉函数执行期间, 本信号自动被屏蔽(sa_flags = 0).
3. 捕捉函数执行期间, 被屏蔽信号多次发送, 解除屏蔽后只处理一次!

一个信号捕捉的小例子:

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <errno.h>
7 #include <pthread.h>
8
9 void sig_cath(int signo){
10     printf("catch you!!! %d \n", signo);
11     return;
12 }
13
14 int main(int argc, char *argv[])
15 {
16     signal(SIGINT, sig_cath); // register signal catch function
17
18     while(1);
19
20     return 0;
21 }

```

'signal.c' 21L, 339C written

编译运行，如下：

```

zhcode@ubuntu:~/Code/code129$ make signal
gcc signal.c -o signal -Wall -g
zhcode@ubuntu:~/Code/code129$ ./signal
^Ccatch you!!! 2
^Ccatch you!!! 2
^Ccatch you!!! 2

```

137P-sigaction 实现信号捕捉

sigaction 也是注册一个信号捕捉函数

SYNOPSIS

```
#include <signal.h>

int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);
```

The sigaction structure is defined as something like:

```
struct sigaction {
    void      (*sa_handler)(int);
    void      (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;
    int        sa_flags;
    void      (*sa_restorer)(void);
};
```

RETURN VALUE

sigaction() returns 0 on success; on error, -1 is returned, and errno is set to indicate the error.

下面的小例子，使用 sigaction 捕捉两个信号：

```
1. #include <stdio.h>
2. #include <signal.h>
3. #include <stdlib.h>
4. #include <string.h>
5. #include <unistd.h>
6. #include <errno.h>
7. #include <pthread.h>
8.
9. void sys_err(const char *str)
10. {
11.     perror(str);
12.     exit(1);
13. }
14.
15. void sig_catch(int signo)                // 回调函数
16. {
17.     if (signo == SIGINT) {
18.         printf("catch you!! %d\n", signo);
19.         sleep(10);
20.     }
21.
22.     else if (signo == SIGQUIT)
23.         printf("-----catch you!! %d\n", signo);
24.
25.     return ;
```

```

26. }
27.
28. int main(int argc, char *argv[])
29. {
30.     struct sigaction act, oldact;
31.
32.     act.sa_handler = sig_catch;           // set callback function name       设置回调函数
33.     sigemptyset(&(act.sa_mask));          // set mask when sig_catch working. 清空 sa_mask 屏蔽字，只在
                                           sig_catch 工作时有效
34.     //sigaddset(&act.sa_mask, SIGQUIT);
35.     act.sa_flags = 0;                     // usually use.                        默认值
36.
37.     int ret = sigaction(SIGINT, &act, &oldact); //注册信号捕捉函数
38.     if (ret == -1)
39.         sys_err("sigaction error");
40.     ret = sigaction(SIGQUIT, &act, &oldact); //注册信号捕捉函数
41.
42.     while (1);
43.
44.     return 0;
45. }

```

编译运行，如下：

```

zhcode@ubuntu:~/Code/code129$ make sigaction
gcc sigaction.c -o sigaction -Wall -g
zhcode@ubuntu:~/Code/code129$ ./sigaction
^Ccatch you!! 2
^\-----catch you!! 3
^Z
[4]+  Stopped                  ./sigaction
zhcode@ubuntu:~/Code/code129$

```

如图，两个信号都捕捉到了，并且输出了对应字符串。

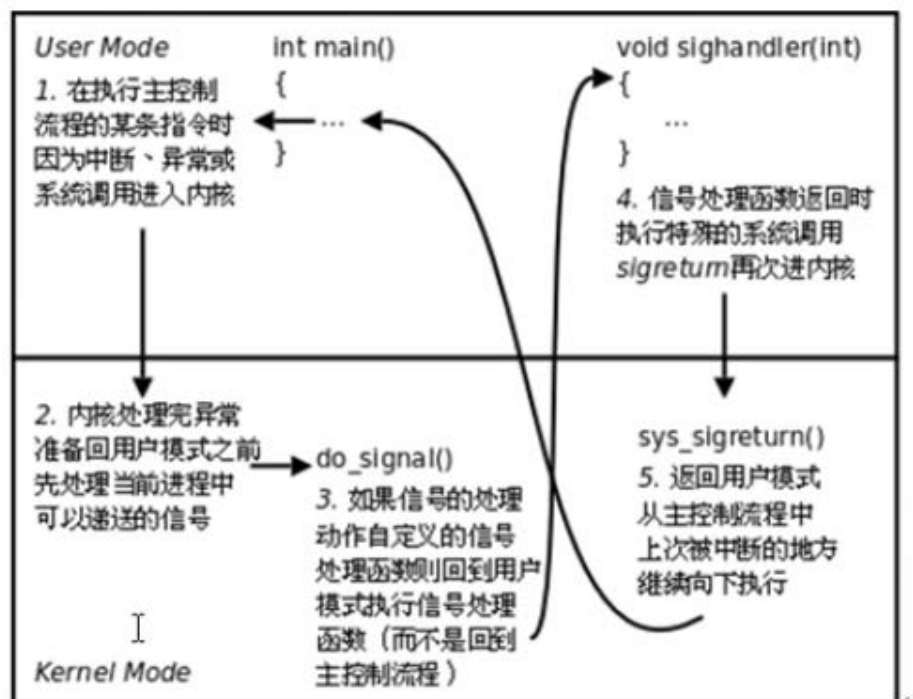
138P-信号捕捉的特性

信号捕捉特性：

1. 捕捉函数执行期间，信号屏蔽字 由 `mask` --> `sa_mask` ， 捕捉函数执行结束。 恢复回 `mask`
2. 捕捉函数执行期间，本信号自动被屏蔽(`sa_flags = 0`) .
3. 捕捉函数执行期间，被屏蔽信号多次发送，解除屏蔽后只处理一次！

139P-内核实现信号捕捉简析

■ 内核实现信号捕捉过程：



140P-借助信号捕捉回收子进程

SIGCHLD 的产生条件:

子进程终止时

子进程接收到 SIGSTOP

子进程处于停止态，接收到 SIGCONT 后唤醒时

下面是一个例子，创建子进程，并使用信号回收：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <signal.h>
6. #include <sys/wait.h>
7. #include <errno.h>
8. #include <pthread.h>
9.
10. void sys_err(const char *str)
11. {
12.     perror(str);
13.     exit(1);
14. }
15.
16. void catch_child(int signo)
17. {
18.     pid_t wpid;
19.
20.     wpid = wait(NULL);
21.     printf("-----catch child id %d\n", wpid);
22.
23.     return ;
24. }
25.
26. int main(int argc, char *argv[])
27. {
28.     pid_t pid;
29.     //阻塞
30.     int i;
31.     for (i = 0; i < 5; i++)
32.         if ((pid = fork()) == 0)           // 创建多个子进程
33.             break;
34.
35.     if (5 == i) {
36.         struct sigaction act;
37.
38.         act.sa_handler = catch_child;       // 设置回调函数
39.         sigemptyset(&act.sa_mask);         // 设置捕捉函数执行期间屏蔽字
40.         act.sa_flags = 0;                  // 设置默认属性，本信号自动屏蔽
```

```

41.
42.     sigaction(SIGCHLD, &act, NULL);           // 注册信号捕捉函数
43. //解除阻塞
44.
45.     printf("I'm parent, pid = %d\n", getpid());
46.
47.     while (1);
48.
49. } else {
50.     printf("I'm child pid = %d\n", getpid());
51.     return i;
52. }
53.
54. return 0;
55. }

```

编译运行，如下图所示：

```

zhcode@ubuntu:~/Code/code129$ make catch_child
gcc catch_child.c -o catch_child -Wall -g
zhcode@ubuntu:~/Code/code129$ ./catch_child
I'm parent, pid = 5625
I'm child pid = 5629
I'm child pid = 5628
I'm child pid = 5626
I'm child pid = 5627
I'm child pid = 5630
-----catch child id 5626

```

如图，只回收到 1 个子进程，多次执行，会发现回收到的子进程数量不是固定的。

原因分析：

问题出在，一次回调只回收一个子进程这里。同时出现多个子进程死亡时，其中一个子进程死亡信号被捕捉，父进程去处理这个信号，此时其他子进程死亡信号发送过来，由于相同信号的不排队原则，就只会回收累积信号中的一个子进程。

修改代码，回调函数中添加循环，一次回调可以回收多个子进程：

```

1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <signal.h>
6. #include <sys/wait.h>
7. #include <errno.h>
8. #include <pthread.h>
9.
10. void sys_err(const char *str)
11. {
12.     perror(str);

```



```

13.     exit(1);
14. }
15.
16. void catch_child(int signo)
17. {
18.     pid_t wpid;
19.
20.     while((wpid = wait(NULL)) != -1){
21.         printf("-----catch child id %d\n", wpid);
22.     }
23.     return ;
24. }
25.
26. int main(int argc, char *argv[])
27. {
28.     pid_t pid;
29. //阻塞
30.     int i;
31.     for (i = 0; i < 5; i++)
32.         if ((pid = fork()) == 0)           // 创建多个子进程
33.             break;
34.
35.     if (5 == i) {
36.         struct sigaction act;
37.
38.         act.sa_handler = catch_child;       // 设置回调函数
39.         sigemptyset(&act.sa_mask);         // 设置捕捉函数执行期间屏蔽字
40.         act.sa_flags = 0;                  // 设置默认属性，本信号自动屏蔽
41.
42.         sigaction(SIGCHLD, &act, NULL);    // 注册信号捕捉函数
43. //解除阻塞
44.
45.         printf("I'm parent, pid = %d\n", getpid());
46.
47.         while (1);
48.
49.     } else {
50.         printf("I'm child pid = %d\n", getpid());
51.         return i;
52.     }
53.
54.     return 0;
55. }

```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code129$ make catch_child2
gcc catch_child2.c -o catch_child2 -Wall -g
zhcode@ubuntu:~/Code/code129$ ./catch_child2
I'm parent, pid = 5972
I'm child pid = 5977
I'm child pid = 5976
I'm child pid = 5975
I'm child pid = 5974
I'm child pid = 5973
-----catch child id 5973
-----catch child id 5974
-----catch child id 5975
-----catch child id 5976
-----catch child id 5977
^Z
[35]+  Stopped                  ./catch_child2
zhcode@ubuntu:~/Code/code129$
```

这下就是回收了所有子进程，很强势。

还有一个问题需要注意，这里有可能父进程还没注册完捕捉函数，子进程就死亡了，解决这个问题的方法，首先是让子进程 sleep，但这个不太科学。在 fork 之前注册也行，但这个也不是很科学。最科学的方法是在 int i 之前设置屏蔽，等父进程注册完捕捉函数再解除屏蔽。这样即使子进程先死亡了，信号也因为被屏蔽而无法到达父进程。解除屏蔽过后，父进程就能处理累积起来的信号了。

141P-慢速系统调用中断

慢速系统调用：

可能会使进程永久阻塞的一类。如果在阻塞期间收到一个信号，该系统调用就被中断，不再继续执行(早期)，也可以设定系统调用是否重启。如 read, write, pause...

142P-总结

信号共性：

简单、不能携带大量信息、满足条件才发送。

信号的特质：

信号是软件层面上的“中断”。一旦信号产生，无论程序执行到什么位置，必须立即停止运行，处理信号，处理结束，再继续执行后续指令。

所有信号的产生及处理全部都是由【内核】完成的。

信号相关的概念：

产生信号：

1. 按键产生
2. 系统调用产生
3. 软件条件产生
4. 硬件异常产生
5. 命令产生

概念：

未决：产生与递达之间状态。

递达：产生并且送达到进程。直接被内核处理掉。

信号处理方式： 执行默认处理动作、忽略、捕捉（自定义）

阻塞信号集（信号屏蔽字）： 本质：位图。用来记录信号的屏蔽状态。一旦被屏蔽的信号，在解除屏蔽前，一直处于未决态。

未决信号集：本质：位图。用来记录信号的处理状态。该信号集中的信号，表示，已经产生，但尚未被处理。

信号 4 要素：

信号使用之前，应先确定其 4 要素，而后再用!!!

编号、名称、对应事件、默认处理动作。

kill 命令 和 kill 函数:

```
int kill (pid_t pid, int signum)
```

参数:

pid: > 0: 发送信号给指定进程

= 0: 发送信号给跟调用 kill 函数的那个进程处于同一进程组的进程。

< -1: 取绝对值, 发送信号给该绝对值所对应的进程组的所有组员。

= -1: 发送信号给, 有权限发送的所有进程。

signum: 待发送的信号

返回值:

成功: 0

失败: -1 errno

alarm 函数: 使用自然计时法。

定时发送 SIGALRM 给当前进程。

```
unsigned int alarm(unsigned int seconds);
```

seconds: 定时秒数

返回值: 上次定时剩余时间。

无错误现象。

alarm (0); 取消闹钟。

time 命令 : 查看程序执行时间。 实际时间 = 用户时间 + 内核时间 + 等待时间。 --》
优化瓶颈 IO

setitimer 函数:

```
int setitimer(int which, const struct itimerval *new_value, struct itimerval *old_value);
```

参数:

which: ITIMER_REAL: 采用自然计时。 ——> SIGALRM

ITIMER_VIRTUAL: 采用用户空间计时 ---> SIGVTALRM

ITIMER_PROF: 采用内核+用户空间计时 ---> SIGPROF

new_value: 定时秒数

```
类型: struct itimerval {  
  
    struct timeval {  
        time_t      tv_sec;           /* seconds */  
        suseconds_t tv_usec;         /* microseconds */  
    } it_interval; ---> 周期定时秒数  
  
    struct timeval {  
        time_t      tv_sec;  
        suseconds_t tv_usec;  
    } it_value; ---> 第一次定时秒数  
};
```

old_value: 传出参数, 上次定时剩余时间。

e. g.

```
struct itimerval new_t;  
struct itimerval old_t;  
  
new_t.it_interval.tv_sec = 0;  
new_t.it_interval.tv_usec = 0;  
new_t.it_value.tv_sec = 1;  
new_t.it_value.tv_usec = 0;  
  
int ret = setitimer(&new_t, &old_t); 定时 1 秒
```

返回值:

成功: 0

失败: -1 errno

其他几个发信号函数:

```
int raise(int sig);
```

```
void abort(void);
```

信号集操作函数:

```
sigset_t set; 自定义信号集。
```

`sigemptyset(sigset_t *set);` 清空信号集

`sigfillset(sigset_t *set);` 全部置 1

`sigaddset(sigset_t *set, int signum);` 将一个信号添加到集合中

`sigdelset(sigset_t *set, int signum);` 将一个信号从集合中移除

`sigismember (const sigset_t *set, int signum);` 判断一个信号是否在集合中。 在--》1, 不在--》0

设置信号屏蔽字和解除屏蔽:

`int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);`

how: SIG_BLOCK: 设置阻塞

SIG_UNBLOCK: 取消阻塞

SIG_SETMASK: 用自定义 set 替换 mask。

set: 自定义 set

oldset: 旧有的 mask。

查看未决信号集:

`int sigpending(sigset_t *set);`

set: 传出的 未决信号集。

【信号捕捉】:

`signal();`

【sigaction();】 重点!!!

信号捕捉特性:

1. 捕捉函数执行期间, 信号屏蔽字 由 mask --> sa_mask , 捕捉函数执行结束。 恢复回 mask
2. 捕捉函数执行期间, 本信号自动被屏蔽(sa_flg = 0).
3. 捕捉函数执行期间, 被屏蔽信号多次发送, 解除屏蔽后只处理一次!

借助信号完成 子进程回收。

143P-复习子进程借助信号回收

信号回收子进程的完整代码，如下：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <signal.h>
6. #include <sys/wait.h>
7. #include <errno.h>
8. #include <pthread.h>
9.
10. void sys_err(const char *str)
11. {
12.     perror(str);
13.     exit(1);
14. }
15.
16. void catch_child(int signo)           // 有子进程终止，发送 SIGCHLD 信号时，该函数会被内核回调
17. {
18.     pid_t wpid;
19.     int status;
20.
21.     //while((wpid = wait(NULL)) != -1) {
22.     while((wpid = waitpid(-1, &status, 0)) != -1) {           // 循环回收,防止僵尸进程出现.
23.         if (WIFEXITED(status))
24.             printf("-----catch child id %d, ret=%d\n", wpid, WEXITSTATUS(status));
25.     }
26.
27.     return ;
28. }
29.
30. int main(int argc, char *argv[])
31. {
32.     pid_t pid;
33.     //阻塞
34.     sigset_t set;
35.     sigemptyset(&set);
36.     sigaddset(&set, SIGCHLD);
37.
38.     sigprocmask(SIG_BLOCK, &set, NULL);
39.
40.     int i;
41.     for (i = 0; i < 15; i++)
42.         if ((pid = fork()) == 0)           // 创建多个子进程
43.             break;
44.
45.     if (15 == i) {
```



```

46.     struct sigaction act;
47.
48.     act.sa_handler = catch_child;           // 设置回调函数
49.     sigemptyset(&act.sa_mask);             // 设置捕捉函数执行期间屏蔽字
50.     act.sa_flags = 0;                      // 设置默认属性，本信号自动屏蔽
51.
52.     sigaction(SIGCHLD, &act, NULL);        // 注册信号捕捉函数
53.     //解除阻塞
54.     sigprocmask(SIG_UNBLOCK, &set, NULL);
55.
56.     printf("I'm parent, pid = %d\n", getpid());
57.
58.     while (1);
59.
60. } else {
61.     printf("I'm child pid = %d\n", getpid());
62.     return i;
63. }
64.
65. return 0;
66. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code129$ make catch_child3
gcc catch_child3.c -o catch_child3 -Wall -g
zhcode@ubuntu:~/Code/code129$ ./catch_child3
I'm parent, pid = 2432
I'm child pid = 2438
I'm child pid = 2437
I'm child pid = 2439
I'm child pid = 2436
I'm child pid = 2440
I'm child pid = 2435
I'm child pid = 2441
I'm child pid = 2442
I'm child pid = 2443
I'm child pid = 2434
I'm child pid = 2444
I'm child pid = 2447
I'm child pid = 2445
I'm child pid = 2446
I'm child pid = 2433
-----catch child id 2433, ret=0
-----catch child id 2434, ret=1
-----catch child id 2435, ret=2
-----catch child id 2436, ret=3
-----catch child id 2437, ret=4
-----catch child id 2438, ret=5
-----catch child id 2439, ret=6
-----catch child id 2440, ret=7

```

所有子进程都回收了，就很强势。

144P-会话

会话：多个进程组的集合

创建会话的 6 点注意事项：

1. 调用进程不能是进程组组长，该进程变成新会话首进程
2. 该进程成为一个新进程组的组长进程
3. 需要 root 权限（ubuntu 不需要）
4. 新会话丢弃原有的控制终端，该会话没有控制终端
5. 该调用进程是组长进程，则出错返回
6. 建立新会话时，先调用 fork，父进程终止，子进程调用 setsid

getsid 函数：

`pid_t getsid(pid_t pid)` 获取当前进程的会话 id
成功返回调用进程会话 ID，失败返回-1，设置 error

setsid 函数：

`pid_t setsid(void)` 创建一个会话，并以自己的 ID 设置进程组 ID，同时也是新会话的 ID
成功返回调用进程的会话 ID，失败返回-1，设置 error

145P-守护进程创建步骤分析

守护进程：

daemon 进程。通常运行于操作系统后台，脱离控制终端。一般不与用户直接交互。周期性的等待某个事件发生或周期性执行某一动作。

不受用户登录注销影响。通常采用以 d 结尾的命名方式。

创建守护进程，最关键的一步是调用 setsid 函数创建一个新的 Session，并成为 Session Leader

守护进程创建步骤：

1. fork 子进程，让父进程终止。
2. 子进程调用 setsid() 创建新会话
3. 通常根据需要，改变工作目录位置 chdir()，防止目录被卸载。
4. 通常根据需要，重设 umask 文件权限掩码，影响新文件的创建权限。
-- 432 r---wx-w- 422 022 -- 755 0345 -
5. 通常根据需要，关闭/重定向 文件描述符
6. 守护进程 业务逻辑。while ()

146P-守护进程创建

下面这个例子，创建一个守护进程：

```
1. #include <stdio.h>
2. #include <sys/stat.h>
3. #include <fcntl.h>
4. #include <stdlib.h>
5. #include <string.h>
6. #include <unistd.h>
7. #include <errno.h>
8. #include <pthread.h>
9.
10. void sys_err(const char *str)
11. {
12.     perror(str);
13.     exit(1);
14. }
15.
16. int main(int argc, char *argv[])
17. {
18.     pid_t pid;
19.     int ret, fd;
20.
21.     pid = fork();
22.     if (pid > 0)           // 父进程终止
23.         exit(0);
24.
25.     pid = setsid();        // 创建新会话
26.     if (pid == -1)
27.         sys_err("setsid error");
28.
29.     ret = chdir("/home/zhcode/Code/code146");    // 改变工作目录位置
30.     if (ret == -1)
31.         sys_err("chdir error");
32.
33.     umask(0022);          // 改变文件访问权限掩码
34.
35.     close(STDIN_FILENO);   // 关闭文件描述符 0
36.
37.     fd = open("/dev/null", O_RDWR); // fd --> 0
38.     if (fd == -1)
39.         sys_err("open error");
40.
41.     dup2(fd, STDOUT_FILENO); // 重定向 stdout 和 stderr
42.     dup2(fd, STDERR_FILENO);
43.
44.     while (1);             // 模拟 守护进程业务.
45.
```

```
46.     return 0;
47. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code146$ make daemon
gcc daemon.c -o daemon -Wall -g
zhcode@ubuntu:~/Code/code146$ ./daemon
zhcode@ubuntu:~/Code/code146$
```

查看进程列表，如下：

zhcode	2214	98.9	0.0	4216	76	?	Rs	10:48	5:05	./daemon
root	2252	0.0	0.0	0	0	?	S	10:53	0:00	[kworker/0:0]
zhcode	2256	0.0	0.1	39100	3388	pts/4	R+	10:53	0:00	ps aux

这个 daemon 进程就不会受到用户登录注销影响。
要想终止，就必须用 kill 命令

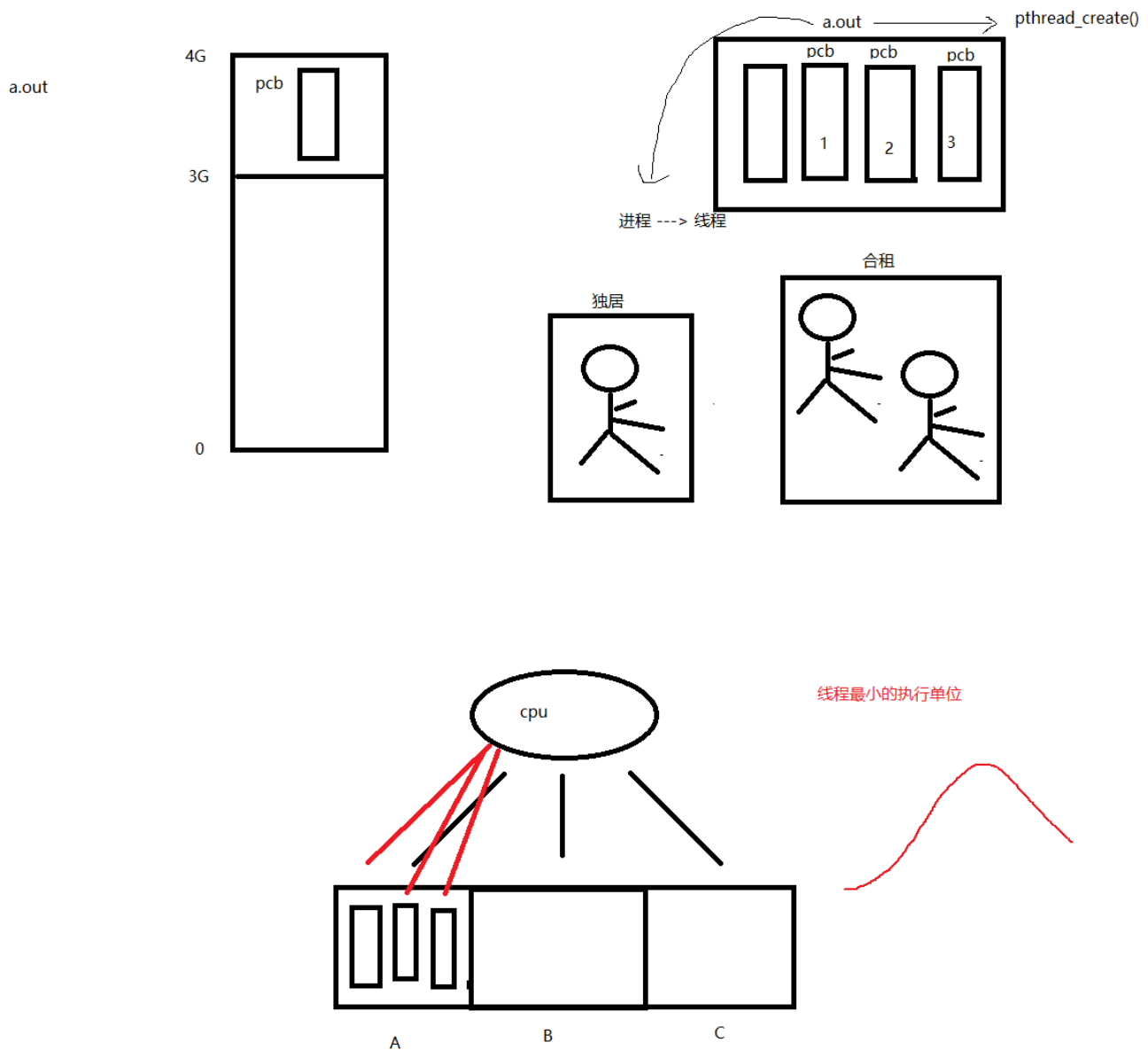
147P-线程概念

线程概念：

进程：有独立的 进程地址空间。有独立的 pcb。 分配资源的最小单位。

线程：有独立的 pcb。没有独立的进程地址空间。 最小单位的执行。

ps -Lf 进程 id ---> 线程号。LWP --> cpu 执行的最小单位。



ps -Lf 进程号 查看进程的线程

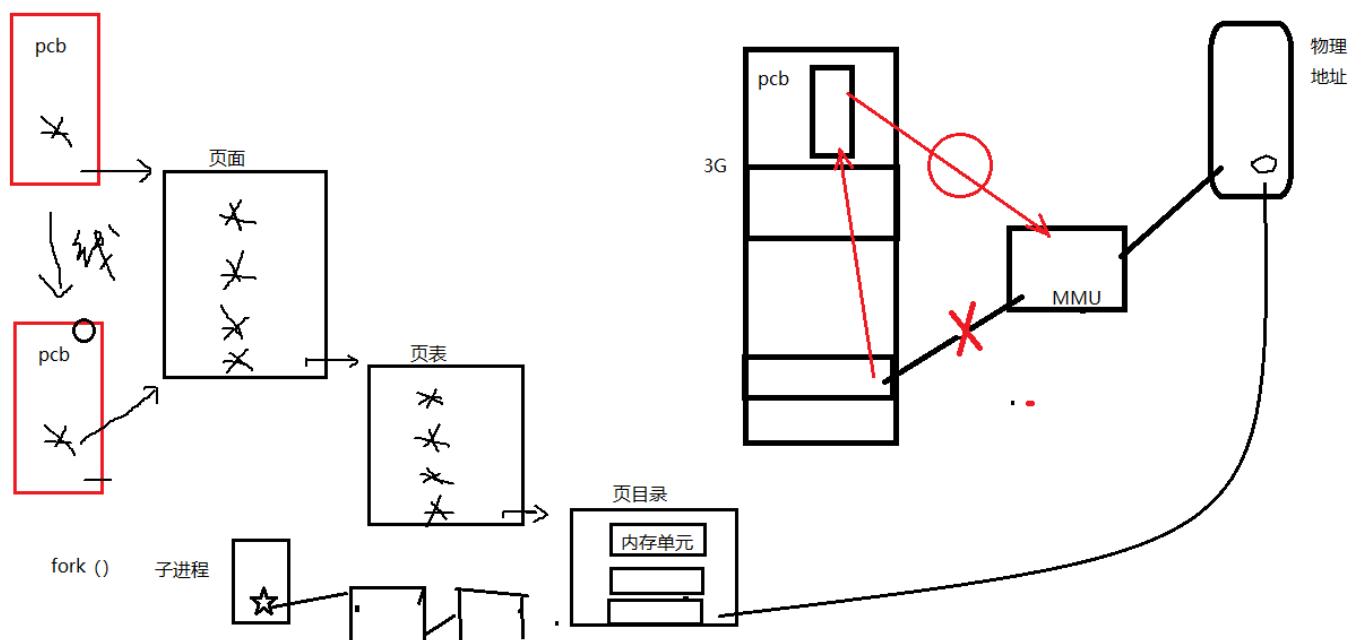
148P-三级映射

Linux 内核线程实现原理

类 Unix 系统中，早期是没有“线程”概念的，80 年代才引入，借助进程机制实现出了线程的概念。因此在这类系统中，进程和线程关系密切。

1. 轻量级进程(light-weight process)，也有 PCB，创建线程使用的底层函数和进程一样，都是 clone
2. 从内核里看进程和线程是一样的，都有各自不同的 PCB，但是 PCB 中指向内存资源的三级页表是相同的
3. 进程可以蜕变成线程
4. 线程可看做寄存器和栈的集合
5. 在 linux 下，线程是最小的执行单位；进程是最小的分配资源单位

察看 LWP 号：ps -Lf pid 查看指定线程的 lwp 号。



149P-线程共享和非共享

线程共享：

独享 栈空间（内核栈、用户栈）

共享 `./text./data ./rodataa ./bss heap` ---> 共享【全局变量】(errno)

150P-中午复习

151P-创建线程

`pthread_t pthread_self(void);` 获取线程 id。线程 id 是在进程地址空间内部，用来标识线程身份的 id 号。

返回值：本线程 id

检查出错返回：线程中。

```
fprintf(stderr, "xxx error: %s\n", strerror(ret));
```

`int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);` 创建子线程。

参 1：传出参数，表新创建的子线程 id

参 2：线程属性。传 NULL 表使用默认属性。

参 3：子线程回调函数。创建成功，`pthread_create` 函数返回时，该函数会被自动调用。

参 4：参 3 的参数。没有的话，传 NULL

返回值：成功：0

失败：errno

下面这个例子，创建一个子线程去执行其他任务：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void sys_err(const char *str){
9.     perror(str);
10.    exit(1);
11. }
12.
13. void *tfn(void *arg){
14.    printf("thread: pid = %d, tid = %lu\n", getpid(), pthread_self());
15.
16.    return NULL;
```

```

17. }
18.
19. int main(int argc, char *argv[]){
20.     pthread_t tid;
21.
22.     printf("main: pid = %d, tid = %lu\n", getpid(), pthread_self());
23.
24.     int ret = pthread_create(&tid, NULL, tfn, NULL);
25.     if (ret != 0) {
26.         perror("pthread_create error");
27.     }
28.
29.     return 0;
30. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code151$ make pthrd_cratt
gcc pthrd_cratt.c -o pthrd_cratt -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./pthrd_cratt
main: pid = 3220, tid = 139626669016832
zhcode@ubuntu:~/Code/code151$

```

可以看到，子线程的打印信息并未出现。原因在于，主线程执行完之后，就销毁了整个进程的地址空间，于是子线程就无法打印。简单粗暴的方法就是让主线程睡 1 秒，等子线程执行。

代码变化如下：

```

24     int ret = pthread_create(&tid, NULL, tfn, NULL);
25     if (ret != 0) {
26         perror("pthread_create error");
27     }
28
29     sleep(1);
30
31     return 0;
32 }

```

编译执行，如下：

```

zhcode@ubuntu:~/Code/code151$ make pthrd_cratt
gcc pthrd_cratt.c -o pthrd_cratt -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./pthrd_cratt
main: pid = 3374, tid = 140551227545344
thread: pid = 3374, tid = 140551219214080
zhcode@ubuntu:~/Code/code151$

```

152P-循环创建多个子线程

下面的例子，循环创建多个子线程：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void sys_err(const char *str){
9.     perror(str);
10.    exit(1);
11. }
12.
13. void *tfn(void *arg){
14.     int i = (int)arg;
15.     sleep(i);
16.     printf("--I'm %dth thread: pid = %d, tid = %lu\n",i+1, getpid(), pthread_self());
17.
18.     return NULL;
19. }
20.
21. int main(int argc, char *argv[]){
22.     int i;
23.     int ret;
24.     pthread_t tid;
25.
26.     for(i=0;i<5;i++){
27.         ret = pthread_create(&tid, NULL, tfn, (void *)i);
28.         if (ret != 0) {
29.             sys_err("pthread_create error");
30.         }
31.     }
32.     sleep(i);
33.     printf("I'm main, pid = %d, tid = %lu\n", getpid(), pthread_self());
34.
35.     return 0;
36. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code151$ ./pthrd_more
--I'm 1th thread: pid = 3533, tid = 140219065702144
--I'm 2th thread: pid = 3533, tid = 140219057309440
--I'm 3th thread: pid = 3533, tid = 140219048916736
--I'm 4th thread: pid = 3533, tid = 140219040524032
--I'm 5th thread: pid = 3533, tid = 140219032131328
I'm main, pid = 3533, tid = 140219074033408
zhcode@ubuntu:~/Code/code151$
```

编译时会出现类型强转的警告，指针 4 字节转 int 的 8 字节，不过不存在精度损失，忽略就行。

153P-错误分析

在 152P 的代码中，如果将 `i` 取地址后再传入线程创建函数里，就是说当前传的是：`(void *)i`
改成：`(void *)&i`
相应的，修改回调函数：`int i = *((int *)arg)`
运行代码，会出现如下结果：

```
zhcode@ubuntu:~/Code/code151$ ./pthrd more
--I'm 6th thread: pid = 3617, tid = 140378375792384
--I'm 6th thread: pid = 3617, tid = 140378384185088
--I'm 6th thread: pid = 3617, tid = 140378392577792
--I'm 6th thread: pid = 3617, tid = 140378400970496
--I'm 6th thread: pid = 3617, tid = 140378409363200
I'm main, pid = 3617, tid = 140378417694464
zhcode@ubuntu:~/Code/code151$
```

如果多次运行都只有主线程的输出，将主线程等待时长从 `i` 改为大于 6 的数即可。因为子线程等待时间 `i` 是不定的，但都小于等于 6 秒，由于抢 cpu 时没抢过主线程，导致没有子线程的输出。

错误原因在于，子线程如果用引用传递 `i`，会去读取主线程里的 `i` 值，而主线程里的 `i` 是动态变化的，不固定。所以，应该采用值传递，不用引用传递。

154P-线程间全局变量共享

直接看个代码，在子线程里更改全局变量，看主线程里的该变量有啥变化：

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int var = 100;
7
8 void *tfn(void *arg){
9     var = 200;
10    printf("thread, var = %d\n", var);
11
12    return NULL;
13 }
14
15 int main(void){
16    printf("At first var = %d\n", var);
17
18    pthread_t tid;
19    pthread_create(&tid, NULL, tfn, NULL);
20    sleep(1);
21
22    printf("after pthread_create, var = %d\n", var);
23
24    return 0;
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code151$ make glb_thread
gcc glb_thread.c -o glb_thread -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./glb_thread
At first var = 100
thread, var = 200
after pthread_create, var = 200
zhcode@ubuntu:~/Code/code151$
```

可以看到，子线程里更改全局变量后，主线程里也跟着发生变化。

155P-pthread_exit 退出

`void pthread_exit(void *retval);` 退出当前线程。

`retval`: 退出值。 无退出值时, `NULL`

`exit()`; 退出当前进程。

`return`: 返回到调用者那里去。

`pthread_exit()`: 退出当前线程。

如果在回调函数里加一段代码:

```
if(i == 2)
```

```
    exit(0);
```

看起来好像是退出了第三个子线程, 然而运行时, 发现后续的 4, 5 也没了。这是因为, `exit` 是退出进程。

一、修改一下, 换成:

```
if(i == 2)
```

```
    return NULL;
```

这样运行一下, 发现后续线程不会凉凉, 说明 `return` 是可以达到退出线程的目的。然而真正意义上, `return` 是返回到函数调用者那里去, 线程并没有退出。

二、再修改一下, 再定义一个函数 `func`, 直接返回那种

```
void *func(void) {
```

```
    return NULL;
```

```
}
```

```
if(i == 2)
```

```
    func();
```

运行, 发现 1, 2, 3, 4, 5 线程都还在, 说明没有达到退出目的。

三、再次修改:

```
void *func(void) {
```

```
    pthread_exit(NULL);
```

```
    return NULL;
```

```
}
```

```
if(i == 2)
```

```
    func();
```

编译运行, 发现 3 没了, 看起来很科学的样子。 `pthread_exit` 表示将当前线程退出。放在函数里, 还是直接调用, 都可以。

回到之前说的一个问题，由于主线程可能先于子线程结束，所以子线程的输出可能不会打印出来，当时是用主线程 `sleep` 等待子线程结束来解决的。现在就可以使用 `pthread_exit` 来解决了。方法就是将 `return 0` 替换为 `pthread_exit`，只退出当前线程，不会对其他线程造成影响。

`void pthread_exit(void *retval);` 退出当前线程。

`retval`: 退出值。 无退出值时，`NULL`

`exit()`: 退出当前进程。

`return`: 返回到调用者那里去。

`pthread_exit()`: 退出当前线程。

156P-pthread_join

`int pthread_join(pthread_t thread, void **retval);` 阻塞 回收线程。

thread: 待回收的线程 id

retval: 传出参数。 回收的那个线程的退出值。

线程异常借助, 值为 -1。

返回值: 成功: 0

失败: errno

下面这个是回收线程并获取子线程返回值的小例子:

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. struct thrd {
9.     int var;
10.    char str[256];
11. };
12.
13. void sys_err(const char *str)
14. {
15.     perror(str);
16.     exit(1);
17. }
18.
19. void *tfn(void *arg)
20. {
21.     struct thrd *tval;
22.
23.     tval = malloc(sizeof(tval));
24.     tval->var = 100;
25.     strcpy(tval->str, "hello thread");
26.
27.     return (void *)tval;
28. }
29.
30. int main(int argc, char *argv[])
31. {
32.     pthread_t tid;
33.
```

```
34.     struct thrd *retval;
35.
36.     int ret = pthread_create(&tid, NULL, tfn, NULL);
37.     if (ret != 0)
38.         sys_err("pthread_create error");
39.
40.     //int pthread_join(pthread_t thread, void **retval);
41.     ret = pthread_join(tid, (void **)&retval);
42.     if (ret != 0)
43.         sys_err("pthread_join error");
44.
45.     printf("child thread exit with var= %d, str= %s\n", retval->var, retval->str);
46.
47.     pthread_exit(NULL);
48.
49. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code151$ make pthread_join
gcc pthread_join.c -o pthread_join -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./pthread_join
child thread exit with var= 100, str= hello thread
zhcode@ubuntu:~/Code/code151$
```

157P-pthread_join 作业

使用 `pthread_join` 函数将循环创建的多个子线程回收
这里 `tid` 要使用数组来存

158P-pthread_cancel 函数

`int pthread_cancel(pthread_t thread);` 杀死一个线程。 需要到达取消点（保存点）

thread: 待杀死的线程 id

返回值: 成功: 0

失败: errno

如果, 子线程没有到达取消点, 那么 `pthread_cancel` 无效。

我们可以在程序中, 手动添加一个取消点。使用 `pthread_testcancel()`;

成功被 `pthread_cancel()` 杀死的线程, 返回 -1. 使用 `pthread_join` 回收。

小例子, 主线程调用 `pthread_cancel` 杀死子线程

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8.
9. void *tfn(void *arg){
10.     while (1) {
11.         printf("thread: pid = %d, tid = %lu\n", getpid(), pthread_self());
12.         sleep(1);
13.     }
14.
15.     return NULL;
16. }
17.
18. int main(int argc, char *argv[]){
19.     pthread_t tid;
20.
21.     int ret = pthread_create(&tid, NULL, tfn, NULL);
22.     if (ret != 0) {
23.         fprintf(stderr, "pthread_create error:%s\n", strerror(ret));
24.         exit(1);
25.     }
26.
27.     printf("main: pid = %d, tid = %lu\n", getpid(), pthread_self());
28.
29.     sleep(5);
30.
```

```

31.     ret = pthread_cancel(tid);           // 终止线程
32.     if (ret != 0) {
33.         fprintf(stderr, "pthread_cancel error:%s\n", strerror(ret));
34.         exit(1);
35.     }
36.
37.     while (1);
38.
39.     pthread_exit((void *)0);
40. }

```

编译运行，如下：

```

zhcode@ubuntu:~/Code/code151$ make pthread_join
gcc pthread_join.c -o pthread_join -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./pthread_join
child thread exit with var= 100, str= hello thread
zhcode@ubuntu:~/Code/code151$ vi pthread_cancel.c
zhcode@ubuntu:~/Code/code151$ make pthread_cancel
gcc pthread_cancel.c -o pthread_cancel -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./pthread_cancel
main: pid = 4578, tid = 140350113515264
thread: pid = 4578, tid = 140350105184000
thread: pid = 4578, tid = 140350105184000
thread: pid = 4578, tid = 140350105184000
thread: pid = 4578, tid = 140350105184000
thread: pid = 4578, tid = 140350105184000
^C
zhcode@ubuntu:~/Code/code151$

```

可以看到，主线程确实 kill 了子线程。

这里要注意一点，pthread_cancel 工作的必要条件是进入内核，如果 tfn 真的奇葩到没有进入内核，则 pthread_cancel 不能杀死线程，此时需要手动设置取消点，就是 pthread_testcancel()

159P-检查出错返回

`int pthread_detach(pthread_t thread);` 设置线程分离

thread: 待分离的线程 id

返回值: 成功: 0

失败: errno

下面这个例子, 使用 detach 分离线程, 照理来说, 分离后的线程会自动回收:

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8.
9. void *tfn(void *arg)
10. {
11.     printf("thread: pid = %d, tid = %lu\n", getpid(), pthread_self());
12.
13.     return NULL;
14. }
15.
16. int main(int argc, char *argv[])
17. {
18.     pthread_t tid;
19.
20.     int ret = pthread_create(&tid, NULL, tfn, NULL);
21.     if (ret != 0) {
22.         perror("pthread_create error");
23.     }
24.     ret = pthread_detach(tid);           // 设置线程分离` 线程终止,会自动清理 pcb,无需回收
25.     if (ret != 0) {
26.         perror("pthread_detach error");
27.     }
28.
29.     sleep(1);
30.
31.     ret = pthread_join(tid, NULL);
32.     printf("join ret = %d\n", ret);
33.     if (ret != 0) {
34.         perror("pthread_join error");
35.     }
36.
```

```

37.     printf("main: pid = %d, tid = %lu\n", getpid(), pthread_self());
38.
39.     pthread_exit((void *)0);
40. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code151$ make detach
gcc detach.c -o detach -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./detach
thread: pid = 4761, tid = 139950241339136
join ret = 22
pthread_join error: Success
main: pid = 4761, tid = 139950249670400
zhcode@ubuntu:~/Code/code151$

```

这里，问题出现了，join 出错了，但是没打印错误原因。之前的 perror 方法检查线程错误是有问题的。应该使用 strerror，修改代码如下：

```

1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8.
9. void *tfn(void *arg)
10. {
11.     printf("thread: pid = %d, tid = %lu\n", getpid(), pthread_self());
12.
13.     return NULL;
14. }
15.
16. int main(int argc, char *argv[])
17. {
18.     pthread_t tid;
19.
20.     int ret = pthread_create(&tid, NULL, tfn, NULL);
21.     if (ret != 0) {
22.         fprintf(stderr, "pthread_create error: %s\n", strerror(ret));
23.         exit(1);
24.     }
25.     ret = pthread_detach(tid); // 设置线程分离`线程终止,会自动清理 pcb,无需回收
26.     if (ret != 0) {
27.         fprintf(stderr, "pthread_detach error: %s\n", strerror(ret));
28.         exit(1);
29.     }
30.
31.     sleep(1);
32.

```



```
33.     ret = pthread_join(tid, NULL);
34.     printf("join ret = %d\n", ret);
35.     if (ret != 0) {
36.         fprintf(stderr, "pthread_join error: %s\n", strerror(ret));
37.         exit(1);
38.     }
39.
40.     printf("main: pid = %d, tid = %lu\n", getpid(), pthread_self());
41.
42.     pthread_exit((void *)0);
43. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code151$ make detach2
gcc detach2.c -o detach2 -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./detach2
thread: pid = 4792, tid = 140618359441152
join ret = 22
pthread_join error: Invalid argument
zhcode@ubuntu:~/Code/code151$
```

160P-线程分离 pthread_detach

上一节的出错，是因为线程分离后，系统会自动回收资源，用 `pthread_join` 去回收已经被系统回收的线程，那个线程号就是无效参数。

161P-进程和线程控制原语对比

线程控制原语

进程控制原语

`pthread_create()`

`fork();`

`pthread_self()`

`getpid();`

`pthread_exit()`

`exit();` `/ return`

`pthread_join()`

`wait()/waitpid()`

`pthread_cancel()`

`kill()`

`pthread_detach()`

162P-线程属性设置分离线程

线程属性：

设置分离属性。

`pthread_attr_t attr` 创建一个线程属性结构体变量

`pthread_attr_init(&attr);` 初始化线程属性

`pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);` 设置线程属性为 分离态

`pthread_create(&tid, &attr, tfn, NULL);` 借助修改后的 设置线程属性 创建为分离态的新线程

`pthread_attr_destroy(&attr);` 销毁线程属性

调整线程状态，使线程创建出来就是分离态，代码如下：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8.
9. void *tfn(void *arg)
10. {
11.     printf("thread: pid = %d, tid = %lu\n", getpid(), pthread_self());
12.
13.     return NULL;
14. }
15.
16. int main(int argc, char *argv[])
17. {
18.     pthread_t tid;
19.
20.     pthread_attr_t attr;
21.
22.     int ret = pthread_attr_init(&attr);
23.     if (ret != 0) {
24.         fprintf(stderr, "attr_init error:%s\n", strerror(ret));
25.         exit(1);
26.     }
27.
```

```

28.     ret = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);    // 设置线程属性为 分离属
    性
29.     if (ret != 0) {
30.         fprintf(stderr, "attr_setdetachstate error:%s\n", strerror(ret));
31.         exit(1);
32.     }
33.
34.     ret = pthread_create(&tid, &attr, tfn, NULL);
35.     if (ret != 0) {
36.         perror("pthread_create error");
37.     }
38.
39.     ret = pthread_attr_destroy(&attr);
40.     if (ret != 0) {
41.         fprintf(stderr, "attr_destroy error:%s\n", strerror(ret));
42.         exit(1);
43.     }
44.
45.     ret = pthread_join(tid, NULL);
46.     if (ret != 0) {
47.         fprintf(stderr, "pthread_join error:%s\n", strerror(ret));
48.         exit(1);
49.     }
50.
51.     printf("main: pid = %d, tid = %lu\n", getpid(), pthread_self());
52.
53.     pthread_exit((void *)0);
54. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code151$ make attr_detach
gcc attr_detach.c -o attr_detach -Wall -g -lpthread
zhcode@ubuntu:~/Code/code151$ ./attr_detach
pthread_join error:Invalid argument
zhcode@ubuntu:~/Code/code151$ 

```

如图，pthread_join 报错，说明线程已经自动回收，设置分离成功。

163P-线程使用注意事项

• 线程使用注意事项

1. 主线程退出其他线程不退出，主线程应调用 `pthread_exit`

2. 避免僵尸线程

`pthread_join`

`pthread_detach` I

`pthread_create` 指定分离属性

被 join 线程可能在 join 函数返回前就释放完自己的所有内存资源，所以不应当返回被回收线程栈中的值;

3. `malloc` 和 `mmap` 申请的内存可以被其他线程释放

4. 应避免在多线程模型中调用 `fork` 除非，马上 `exec`，子进程中只有调用 `fork` 的线程存在，其他线程在子进程中均 `pthread_exit`

5. 信号的复杂语义很难和多线程共存，应避免在多线程引入信号机制

164P-总结

守护进程:

daemon 进程。通常运行与操作系统后台，脱离控制终端。一般不与用户直接交互。周期性的等待某个事件发生或周期性执行某一动作。

不受用户登录注销影响。通常采用以 d 结尾的命名方式。

守护进程创建步骤:

1. fork 子进程，让父进程终止。
2. 子进程调用 setsid() 创建新会话
3. 通常根据需要，改变工作目录位置 chdir()，防止目录被卸载。
4. 通常根据需要，重设 umask 文件权限掩码，影响新文件的创建权限。 022 -- 755 0345 -- 432 r---wx-w- 422
5. 通常根据需要，关闭/重定向 文件描述符
6. 守护进程 业务逻辑。while ()

线程概念:

进程: 有独立的 进程地址空间。有独立的 pcb。 分配资源的最小单位。

线程: 有独立的 pcb。没有独立的进程地址空间。 最小单位的执行。

ps -Lf 进程 id ---> 线程号。LWP --> cpu 执行的最小单位。

线程共享:

独享 栈空间 (内核栈、用户栈)

共享 ./text./data ./rodataa ./bss heap ---> 共享【全局变量】(errno)

线程控制原语:

pthread_t pthread_self(void); 获取线程 id。线程 id 是在进程地址空间内部，用来标识线程身份的 id 号。

返回值: 本线程 id

检查出错返回： 线程中。

```
fprintf(stderr, "xxx error: %s\n", strerror(ret));
```

```
int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void  
*(*start_rountn)(void *), void *arg); 创建子线程。
```

参 1: 传出参数，表新创建的子线程 id

参 2: 线程属性。传 NULL 表使用默认属性。

参 3: 子线程回调函数。创建成功，ptherad_create 函数返回时，该函数会被自动调用。

参 4: 参 3 的参数。没有的话，传 NULL

返回值：成功：0

失败：errno

循环创建 N 个子线程：

```
for (i = 0; i < 5; i++)
```

```
pthread_create(&tid, NULL, tfn, (void *)i); // 将 int 类型 i, 强转成 void  
*, 传参。
```

void pthread_exit(void *retval); 退出当前线程。

retval: 退出值。无退出值时，NULL

exit(); 退出当前进程。

return: 返回到调用者那里去。

pthread_exit(): 退出当前线程。

int pthread_join(pthread_t thread, void **retval); 阻塞 回收线程。

thread: 待回收的线程 id

retval: 传出参数。回收的那个线程的退出值。

线程异常借助，值为 -1。

返回值：成功：0

失败：errno

`int pthread_detach(pthread_t thread);` 设置线程分离

thread: 待分离的线程 id

返回值：成功：0

失败：errno

`int pthread_cancel(pthread_t thread);` 杀死一个线程。 需要到达取消点（保存点）

thread: 待杀死的线程 id

返回值：成功：0

失败：errno

如果，子线程没有到达取消点， 那么 `pthread_cancel` 无效。

我们可以在程序中，手动添加一个取消点。使用 `pthread_testcancel()`;

成功被 `pthread_cancel()` 杀死的线程，返回 -1. 使用 `pthread_join` 回收。

线程控制原语

进程控制原语

`pthread_create()`

`fork()`;

`pthread_self()`

`getpid()`;

`pthread_exit()`

`exit()`; / return

`pthread_join()`

`wait()/waitpid()`

`pthread_cancel()`

`kill()`

`pthread_detach()`

线程属性：

设置分离属性。

`pthread_attr_t attr` 创建一个线程属性结构体变量

`pthread_attr_init(&attr);` 初始化线程属性

`pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);` 设置线程属性为 分离态

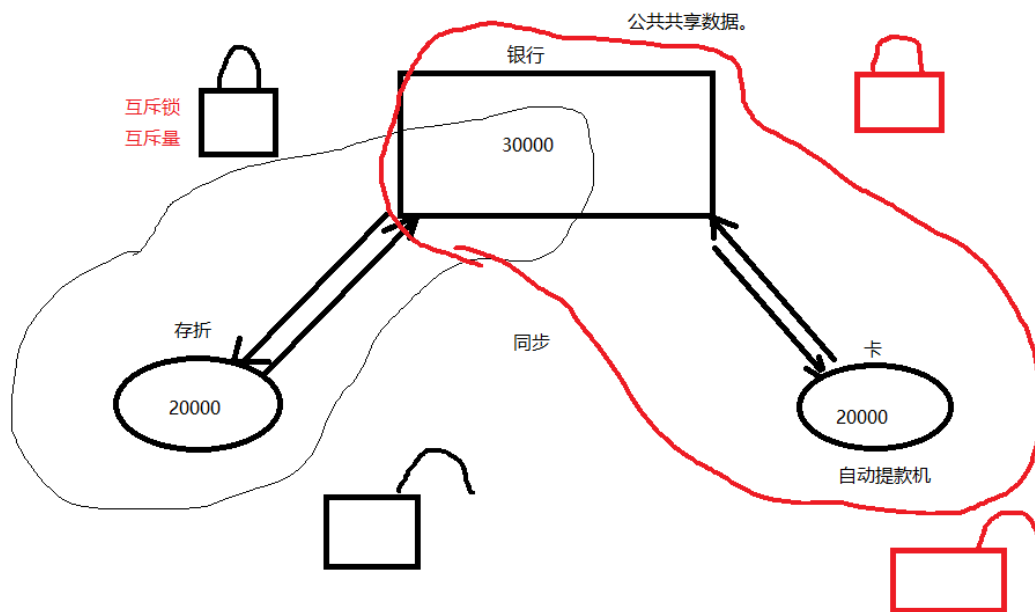
`pthread_create(&tid, &attr, tfn, NULL);` 借助修改后的 设置线程属性 创建为分离态的新线程

`pthread_attr_destroy(&attr);` 销毁线程属性

165P-线程同步概念

线程同步：

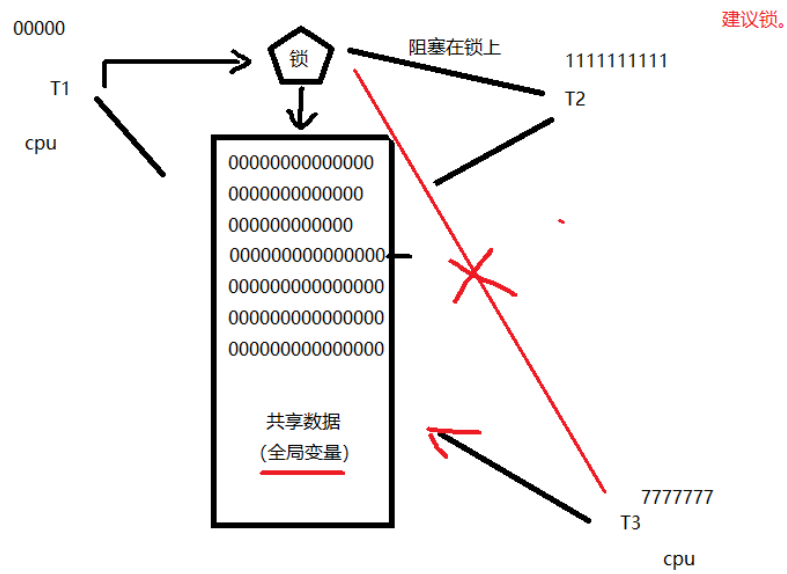
协同步调，对公共区域数据按序访问。防止数据混乱，产生与时间有关的错误。



数据混乱的原因：

1. 资源共享(独享资源则不会)
2. 调度随机(意味着数据访问会出现竞争)
3. 线程间缺乏必要同步机制

166P-锁使用的注意事项



锁的使用：

建议锁！对公共数据进行保护。所有线程【应该】在访问公共数据前先拿锁再访问。但，锁本身不具备强制性。

167P-借助互斥锁管理共享数据实现同步

下面一个小例子，数据共享导致的混乱：

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <pthread.h>
4. #include <stdlib.h>
5. #include <unistd.h>
6.
7. void *tfn(void *arg)
8. {
9.     srand(time(NULL));
10.
11.     while (1) {
12.
13.         printf("hello ");
14.         sleep(rand() % 3); /*模拟长时间操作共享资源，导致 cpu 易主，产生与时间有关的错误*/
15.         printf("world\n");
16.         sleep(rand() % 3);
17.     }
18.
19.     return NULL;
20. }
21.
22. int main(void)
23. {
24.     pthread_t tid;
25.     srand(time(NULL));
26.
27.     pthread_create(&tid, NULL, tfn, NULL);
28.     while (1) {
29.
30.         printf("HELLO ");
31.         sleep(rand() % 3);
32.         printf("WORLD\n");
33.         sleep(rand() % 3);
34.
35.     }
36.     pthread_join(tid, NULL);
37.
38.     return 0;
39. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code167$ make pthread shared
gcc pthread_shared.c -o pthread_shared -Wall -g -lpthread
zhcode@ubuntu:~/Code/code167$ ./pthread_shared
HELLO hello WORLD
world
HELLO WORLD
hello HELLO world
hello WORLD
HELLO world
WORLD
hello HELLO world
```

如图，输出结果是主线程和子线程交叉的。

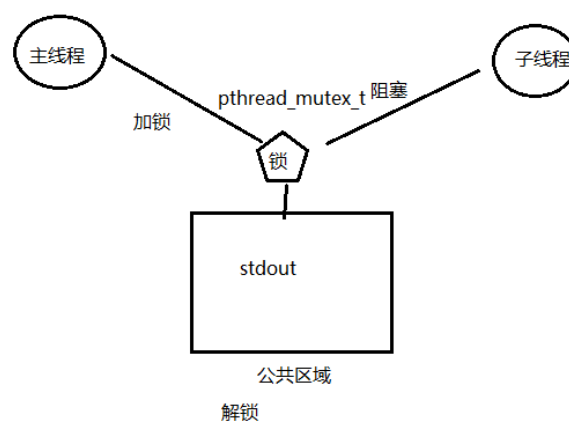
主要应用函数：

pthread_mutex_init	函数
pthread_mutex_destory	函数
pthread_mutex_lock	函数
pthread_mutex_trylock	函数
pthread_mutex_unlock	函数

以上 5 个函数的返回值都是：成功返回 0，失败返回错误号

pthread_mutex_t 类型，其本质是一个结构体。为简化理解，应用时可忽略其实现细节，简单当成整数看待

pthread_mutex_t mutex; 变量 mutex 只有两种取值：0, 1



1. pthread_mutex_t lock; 创建锁
2. pthread_mutex_init; 初始化
3. pthread_mutex_lock; 加锁
4. 访问共享数据 (stdout)
5. pthread_mutex_unlock(); 解锁
6. pthread_mutex_destroy; 销毁锁

使用 mutex (互斥量、互斥锁) 一般步骤：

pthread_mutex_t 类型。

1. pthread_mutex_t lock; 创建锁

2 pthread_mutex_init; 初始化 1

3. pthread_mutex_lock;加锁 1-- --> 0
4. 访问共享数据 (stdout)
5. pthread_mutex_unlock();解锁 0++ --> 1
6. pthread_mutex_destroy; 销毁锁

```
int pthread_mutex_init(pthread_mutex_t *restrict mutex,
                      const pthread_mutexattr_t *restrict attr)
```

这里的 restrict 关键字，表示指针指向的内容只能通过这个指针进行修改
restrict 关键字：

用来限定指针变量。被该关键字限定的指针变量所指向的内存操作，必须由本指针完成。
初始化互斥量：

```
pthread_mutex_t mutex;
```

1. pthread_mutex_init(&mutex, NULL); 动态初始化。
2. pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; 静态初始化。

修改上面的代码，使用锁实现互斥访问共享区：

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <pthread.h>
4. #include <stdlib.h>
5. #include <unistd.h>
6.
7. pthread_mutex_t mutex;      // 定义一把互斥锁
8.
9. void *tfn(void *arg)
10. {
11.     srand(time(NULL));
12.
13.     while (1) {
14.         pthread_mutex_lock(&mutex);      // 加锁
15.         printf("hello ");
16.         sleep(rand() % 3);      // 模拟长时间操作共享资源，导致 cpu 易主，产生与时间有关的错误
17.         printf("world\n");
18.         pthread_mutex_unlock(&mutex);      // 解锁
19.         sleep(rand() % 3);
20.     }
21.
22.     return NULL;
23. }
```

```

24.
25. int main(void)
26. {
27.     pthread_t tid;
28.     srand(time(NULL));
29.     int ret = pthread_mutex_init(&mutex, NULL);    // 初始化互斥锁
30.     if(ret != 0){
31.         fprintf(stderr, "mutex init error:%s\n", strerror(ret));
32.         exit(1);
33.     }
34.
35.     pthread_create(&tid, NULL, tfn, NULL);
36.     while (1) {
37.         pthread_mutex_lock(&mutex);    // 加锁
38.         printf("HELLO ");
39.         sleep(rand() % 3);
40.         printf("WORLD\n");
41.         pthread_mutex_unlock(&mutex);    // 解锁
42.         sleep(rand() % 3);
43.     }
44.     pthread_join(tid, NULL);
45.
46.     pthread_mutex_destory(&mutex);    // 销毁互斥锁
47.
48.     return 0;
49. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code167$ make pthread_mutex
gcc pthread_mutex.c -o pthread_mutex -Wall -g -lpthread
pthread_mutex.c: In function 'main':
pthread_mutex.c:46:2: warning: implicit declaration of function 'pthread_mutex_destory' [-Wimplicit-function-declaration]
pthread_mutex_destory(&mutex);    // 销毁互斥锁
^
zhcode@ubuntu:~/Code/code167$ ls
headcode.c  makefile  pthread_mutex  pthread_mutex.c  pthread_shared  pthread_shared.c
zhcode@ubuntu:~/Code/code167$ ./pthread_mutex
HELLO WORLD
hello world
HELLO WORLD
hello world
^C
zhcode@ubuntu:~/Code/code167$

```

可以看到，主线程和子线程在访问共享区时就没有交叉输出的情况了。

168P-互斥锁使用技巧

注意事项：

尽量保证锁的粒度， 越小越好。（访问共享数据前，加锁。访问结束【立即】解锁。）

互斥锁，本质是结构体。 我们可以看成整数。 初值为 1。（`pthread_mutex_init()` 函数调用成功。）

加锁： `--`操作， 阻塞线程。

解锁： `++`操作， 唤醒阻塞在锁上的线程。

try 锁：尝试加锁，成功`--`。失败，返回。同时设置错误号 `EBUSY`

169P-try 锁

try 锁：尝试加锁，成功--，加锁失败直接返回错误号(如 EBUSY)，不阻塞

170P-读写锁操作函数原型

·读写锁特性：

1. 读写锁是“写模式加锁”时， 解锁前，所有对该锁加锁的线程都会被阻塞。
2. 读写锁是“读模式加锁”时， 如果线程以读模式对其加锁会成功；如果线程以写模式加锁会阻塞。
3. 读写锁是“读模式加锁”时， 既有试图以写模式加锁的线程，也有试图以读模式加锁的线程。那么读写锁会阻塞随后的读模式锁请求。优先满足写模式锁。读锁、写锁并行阻塞，写锁优先级高。

读写锁也叫共享-独占锁。当读写锁以读模式锁住时，它是以共享模式锁住的；当它以写模式锁住时，它是以独占模式锁住的。写独占、读共享。

读写锁非常适合于对数据结构读的次数远大于写的情况。

读写锁：

锁只有一把。以读方式给数据加锁——读锁。以写方式给数据加锁——写锁。

读共享，写独占。

写锁优先级高。

相较于互斥量而言，当读线程多的时候，提高访问效率

```
pthread_rwlock_t  rwlock;
```

```
pthread_rwlock_init(&rwlock, NULL);
```

```
pthread_rwlock_rdlock(&rwlock);    try
```

```
pthread_rwlock_wrlock(&rwlock);    try
```

```
pthread_rwlock_unlock(&rwlock);
```

```
pthread_rwlock_destroy(&rwlock);
```

以上函数都是成功返回 0，失败返回错误号。

pthread_rwlock_t 类型 用于定义一个读写锁变量

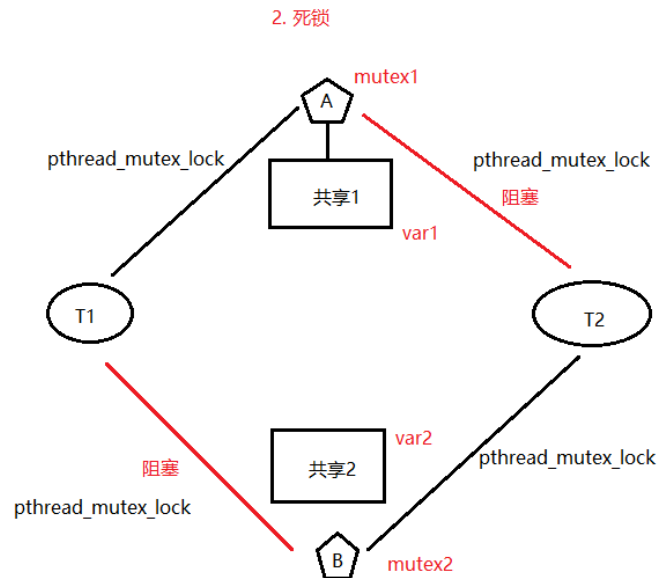
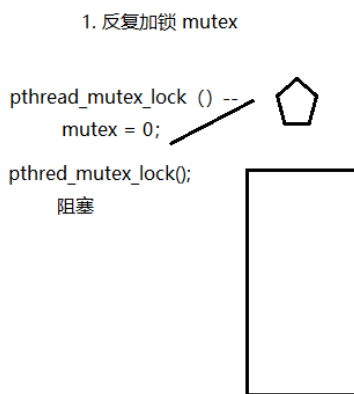
```
pthread_rwlock_t  rwlock
```

171P-两种死锁

【死锁】:

是使用锁不恰当导致的现象:

1. 对一个锁反复 lock。
2. 两个线程，各自持有一把锁，请求另一把。



172P-读写锁原理

读写锁：

锁只有一把。以读方式给数据加锁——读锁。以写方式给数据加锁——写锁。

读共享，写独占。

写锁优先级高。

相较于互斥量而言，当读线程多的时候，提高访问效率

```
pthread_rwlock_t  rwlock;
```

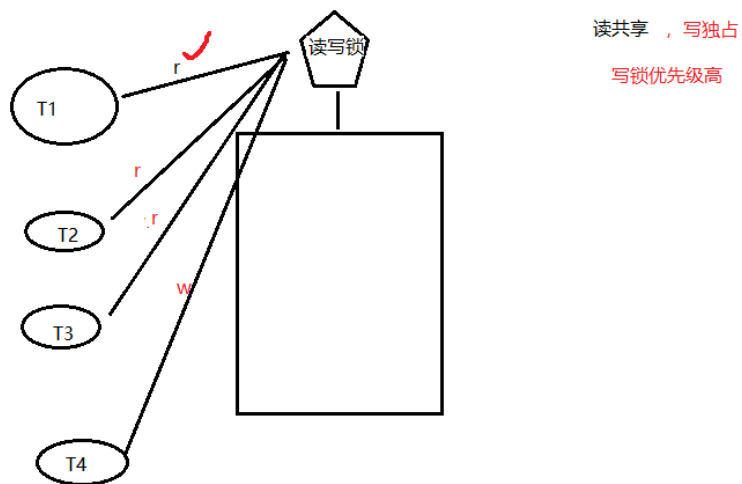
```
pthread_rwlock_init(&rwlock, NULL);
```

```
pthread_rwlock_rdlock(&rwlock);    try
```

```
pthread_rwlock_wrlock(&rwlock);    try
```

```
pthread_rwlock_unlock(&rwlock);
```

```
pthread_rwlock_destroy(&rwlock);
```



• 读写锁特性：

1. 读写锁是“写模式加锁”时， 解锁前，所有对该锁加锁的线程都会被阻塞。
2. 读写锁是“读模式加锁”时， 如果线程以读模式对其加锁会成功；如果线程以写模式加锁会阻塞。
3. 读写锁是“读模式加锁”时， 既有试图以写模式加锁的线程，也有试图以读模式加锁的线程。那么读写锁会阻塞随后的读模式锁请求。优先满足写模式锁。读锁、写锁并行阻塞，写锁优先级高。

读写锁也叫共享-独占锁。当读写锁以读模式锁住时，它是以共享模式锁住的；当它以写模式锁住时，它是以独占模式锁住的。写独占、读共享。

读写锁非常适合于对数据结构读的次数远大于写的情况。

173P-rwlock

一个读写锁的例子，核心还是读共享，写独占。

写锁优先级高。

```
1.  /* 3 个线程不定时 "写" 全局资源，5 个线程不定时 "读" 同一全局资源 */
2.
3.  #include <stdio.h>
4.  #include <unistd.h>
5.  #include <pthread.h>
6.
7.  int counter;                //全局资源
8.  pthread_rwlock_t rwlock;
9.
10. void *th_write(void *arg)
11. {
12.     int t;
13.     int i = (int)arg;
14.
15.     while (1) {
16.         t = counter;        // 保存写之前的值
17.         usleep(1000);
18.
19.         pthread_rwlock_wrlock(&rwlock);
20.         printf("====write %d: %lu: counter=%d ++counter=%d\n", i, pthread_self(), t, ++counter)
21.         ;
22.         pthread_rwlock_unlock(&rwlock);
23.
24.         usleep(9000);        // 给 r 锁提供机会
25.     }
26.     return NULL;
27.
28. void *th_read(void *arg)
29. {
30.     int i = (int)arg;
31.
32.     while (1) {
33.         pthread_rwlock_rdlock(&rwlock);
34.         printf("-----read %d: %lu: %d\n", i, pthread_self(), counter);
35.         pthread_rwlock_unlock(&rwlock);
36.
37.         usleep(2000);        // 给写锁提供机会
38.     }
39.     return NULL;
40. }
41.
```

```

42. int main(void)
43. {
44.     int i;
45.     pthread_t tid[8];
46.
47.     pthread_rwlock_init(&rwlock, NULL);
48.
49.     for (i = 0; i < 3; i++)
50.         pthread_create(&tid[i], NULL, th_write, (void *)i);
51.
52.     for (i = 0; i < 5; i++)
53.         pthread_create(&tid[i+3], NULL, th_read, (void *)i);
54.
55.     for (i = 0; i < 8; i++)
56.         pthread_join(tid[i], NULL);
57.
58.     pthread_rwlock_destroy(&rwlock);           //释放读写锁
59.
60.     return 0;
61. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code167$ ./rwlock
-----read 3: 140051476412160: 0
-----read 2: 140051484804864: 0
-----read 1: 140051493197568: 0
-----read 4: 140051468019456: 0
-----read 0: 140051501590272: 0
=====write 2: 140051509982976: counter=0 ++counter=1
=====write 1: 140051518375680: counter=0 ++counter=2
=====write 0: 140051526768384: counter=0 ++counter=3
-----read 2: 140051484804864: 3

```

程序输出飞快，随便截个图，如上图。

由于读共享，写独占。写锁优先级高。前面 5 个 read 一定先于后面的 write 到达的，不然 write 会抢到锁先进行写操作。

174P-午后复习

175P-静态初始化条件变量和互斥量

条件变量：

本身不是锁！ 但是通常结合锁来使用。 mutex
主要应用函数：

▪ 主要应用函数：

pthread_cond_init 函数↵

pthread_cond_destroy 函数↵

pthread_cond_wait 函数↵

pthread_cond_timedwait 函数↵

pthread_cond_signal 函数↵

pthread_cond_broadcast 函数↵

以上 6 个函数的返回值都是：成功返回 0， 失败直接返回错误号。↵

pthread_cond_t 类型 用于定义条件变量↵

pthread_cond_t cond;↵

pthread_cond_t cond;

初始化条件变量：

1. pthread_cond_init(&cond, NULL); 动态初始化。
2. pthread_cond_t cond = PTHREAD_COND_INITIALIZER; 静态初始化。

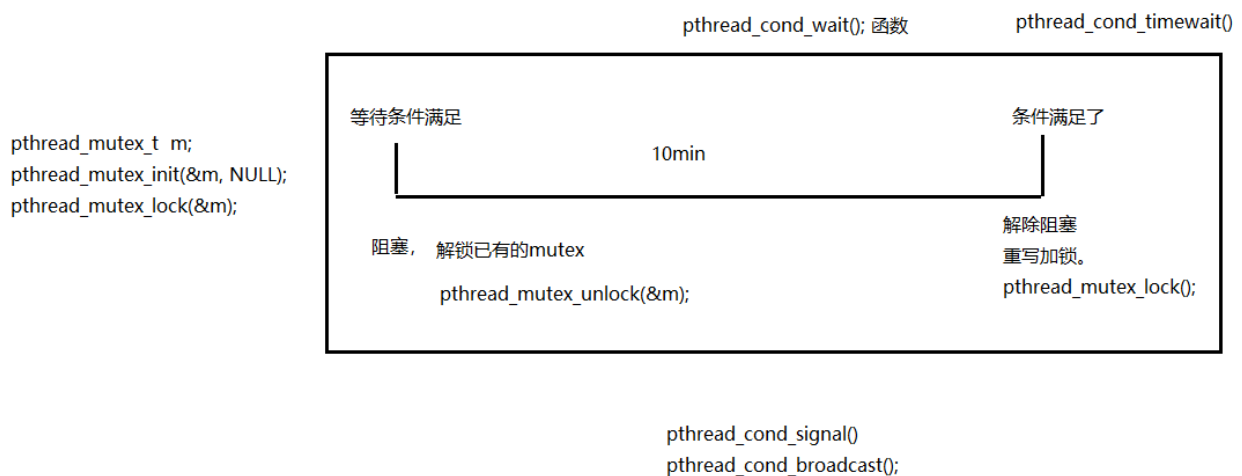
176P-条件变量和相关函数 wait

阻塞等待条件：

```
pthread_cond_wait(&cond, &mutex);
```

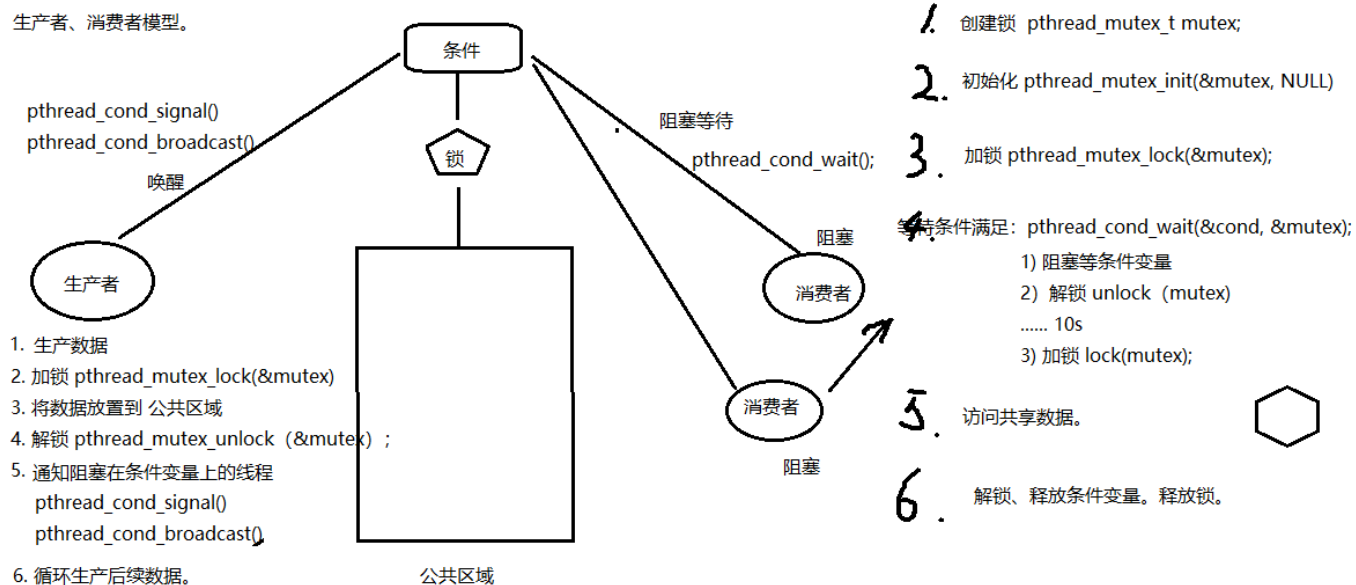
作用：

- 1) 阻塞等待条件变量满足
- 2) 解锁已经加锁成功的信号量（相当于 `pthread_mutex_unlock(&mutex)`），12 两步为一个原子操作
- 3) 当条件满足，函数返回时，解除阻塞并重新申请获取互斥锁。重新加锁信号量（相当于，`pthread_mutex_lock(&mutex);`）



177P-条件变量的生产者消费者模型分析

生产者、消费者模型。



178P-条件变量生产者消费者代码预览

代码如下：

```
1.  /*借助条件变量模拟 生产者-消费者 问题*/
2.  #include <stdlib.h>
3.  #include <unistd.h>
4.  #include <pthread.h>
5.  #include <stdio.h>
6.
7.  /*链表作为公享数据,需被互斥量保护*/
8.  struct msg {
9.      struct msg *next;
10.     int num;
11. };
12.
13. struct msg *head;
14.
15. /* 静态初始化 一个条件变量 和 一个互斥量*/
16. pthread_cond_t has_product = PTHREAD_COND_INITIALIZER;
17. pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
18.
19. void *consumer(void *p)
20. {
21.     struct msg *mp;
22.
23.     for (;;) {
24.         pthread_mutex_lock(&lock);
25.         while (head == NULL) {           //头指针为空,说明没有节点    可以为if吗
26.             pthread_cond_wait(&has_product, &lock);  // 解锁, 并阻塞等待
27.         }
28.         mp = head;
29.         head = mp->next;                 //模拟消费掉一个产品
30.         pthread_mutex_unlock(&lock);
31.
32.         printf("-Consume %lu---%d\n", pthread_self(), mp->num);
33.         free(mp);
34.         sleep(rand() % 5);
35.     }
36. }
37.
38. void *producer(void *p)
39. {
40.     struct msg *mp;
41.
42.     for (;;) {
43.         mp = malloc(sizeof(struct msg));
44.         mp->num = rand() % 1000 + 1;      //模拟生产一个产品
45.         printf("-Produce -----%d\n", mp->num);
```

```

46.
47.     pthread_mutex_lock(&lock);
48.     mp->next = head;
49.     head = mp;
50.     pthread_mutex_unlock(&lock);
51.
52.     pthread_cond_signal(&has_product); //将等待在该条件变量上的一个线程唤醒
53.     sleep(rand() % 5);
54. }
55. }
56.
57. int main(int argc, char *argv[])
58. {
59.     pthread_t pid, cid;
60.     srand(time(NULL));
61.
62.     pthread_create(&pid, NULL, producer, NULL);
63.     pthread_create(&cid, NULL, consumer, NULL);
64.
65.     pthread_join(pid, NULL);
66.     pthread_join(cid, NULL);
67.
68.     return 0;
69. }

```

编译运行，结果如下：

```

zhcode@ubuntu:~/Code/code167$ ./consumer
-Produce -----14
-Consume 139866583148288---14
-Produce -----20
-Consume 139866583148288---20
-Produce -----825
-Produce -----827
-Consume 139866583148288---827
-Consume 139866583148288---825
-Produce -----847
-Consume 139866583148288---847

```

可以看到，消费者按序消费，毕竟是链表。

179P-条件变量实现生产者消费者代码

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void err_thread(int ret, char *str)
9. {
10.     if (ret != 0) {
11.         fprintf(stderr, "%s:%s\n", str, strerror(ret));
12.         pthread_exit(NULL);
13.     }
14. }
15.
16. struct msg {
17.     int num;
18.     struct msg *next;
19. };
20.
21. struct msg *head;
22.
23. pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;    // 定义/初始化一个互斥量
24. pthread_cond_t has_data = PTHREAD_COND_INITIALIZER;    // 定义/初始化一个条件变量
25.
26. void *producer(void *arg)
27. {
28.     while (1) {
29.         struct msg *mp = malloc(sizeof(struct msg));
30.
31.         mp->num = rand() % 1000 + 1;    // 模拟生产一个数据
32.         printf("--produce %d\n", mp->num);
33.
34.         pthread_mutex_lock(&mutex);    // 加锁 互斥量
35.         mp->next = head;    // 写公共区域
36.         head = mp;
37.         pthread_mutex_unlock(&mutex);    // 解锁 互斥量
38.
39.         pthread_cond_signal(&has_data);    // 唤醒阻塞在条件变量 has_data 上的线程。
40.
41.         sleep(rand() % 3);
42.     }
43.
44.     return NULL;
45. }
```

```

46.
47. void *consumer(void *arg)
48. {
49.     while (1) {
50.         struct msg *mp;
51.
52.         pthread_mutex_lock(&mutex);           // 加锁 互斥量
53.         if (head == NULL) {
54.             pthread_cond_wait(&has_data, &mutex); // 阻塞等待条件变量, 解锁
55.         }                                       // pthread_cond_wait 返回时, 重写加
           锁 mutex
56.
57.         mp = head;
58.         head = mp->next;
59.
60.         pthread_mutex_unlock(&mutex);         // 解锁 互斥量
61.         printf("-----consumer:%d\n", mp->num);
62.
63.         free(mp);
64.         sleep(rand()%3);
65.     }
66.
67.     return NULL;
68. }
69.
70. int main(int argc, char *argv[])
71. {
72.     int ret;
73.     pthread_t pid, cid;
74.
75.     srand(time(NULL));
76.
77.     ret = pthread_create(&pid, NULL, producer, NULL); // 生产者
78.     if (ret != 0)
79.         err_thread(ret, "pthread_create producer error");
80.
81.     ret = pthread_create(&cid, NULL, consumer, NULL); // 消费者
82.     if (ret != 0)
83.         err_thread(ret, "pthread_create consuer error");
84.
85.     pthread_join(pid, NULL);
86.     pthread_join(cid, NULL);
87.
88.     return 0;
89. }

```

编译运行，结果如下：


```
zhcode@ubuntu:~/Code/code167$ make consumer2
gcc consumer2.c -o consumer2 -Wall -g -lpthread
zhcode@ubuntu:~/Code/code167$ ./consumer2
--produce 580
-----consumer:580
--produce 284
-----consumer:284
--produce 102
-----consumer:102
--produce 829
--produce 865
-----consumer:865
-----consumer:829
--produce 704
-----consumer:704
--produce 645
--produce 899
--produce 151
```

180P-多个消费者使用 while 做

先看一个手动创建多个消费者的代码：

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4. #include <unistd.h>
5. #include <errno.h>
6. #include <pthread.h>
7.
8. void err_thread(int ret, char *str)
9. {
10.     if (ret != 0) {
11.         fprintf(stderr, "%s:%s\n", str, strerror(ret));
12.         pthread_exit(NULL);
13.     }
14. }
15.
16. struct msg {
17.     int num;
18.     struct msg *next;
19. };
20.
21. struct msg *head;
22.
23. pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;    // 定义/初始化一个互斥量
24. pthread_cond_t has_data = PTHREAD_COND_INITIALIZER;    // 定义/初始化一个条件变量
25.
26. void *producer(void *arg)
27. {
28.     while (1) {
29.         struct msg *mp = malloc(sizeof(struct msg));
30.
31.         mp->num = rand() % 1000 + 1;    // 模拟生产一个数据
32.         printf("--produce %d\n", mp->num);
33.
34.         pthread_mutex_lock(&mutex);    // 加锁 互斥量
35.         mp->next = head;    // 写公共区域
36.         head = mp;
37.         pthread_mutex_unlock(&mutex);    // 解锁 互斥量
38.
39.         pthread_cond_signal(&has_data);    // 唤醒阻塞在条件变量 has_data 上的线程.
40.
41.         sleep(rand() % 3);
42.     }
43.
44.     return NULL;
```

```

45. }
46.
47. void *consumer(void *arg)
48. {
49.     while (1) {
50.         struct msg *mp;
51.
52.         pthread_mutex_lock(&mutex);           // 加锁 互斥量
53.         if (head == NULL) {
54.             pthread_cond_wait(&has_data, &mutex); // 阻塞等待条件变量, 解锁
55.         }                                       // pthread_cond_wait 返回时, 重写加
            锁 mutex
56.
57.         mp = head;
58.         head = mp->next;
59.
60.         pthread_mutex_unlock(&mutex);         // 解锁 互斥量
61.         printf("-----consumer id = %lu  :%d\n", pthread_self(), mp->num);
62.
63.         free(mp);
64.         sleep(rand()%3);
65.     }
66.
67.     return NULL;
68. }
69.
70. int main(int argc, char *argv[])
71. {
72.     int ret;
73.     pthread_t pid, cid;
74.
75.     srand(time(NULL));
76.
77.     ret = pthread_create(&pid, NULL, producer, NULL); // 生产者
78.     if (ret != 0)
79.         err_thread(ret, "pthread_create producer error");
80.
81.     ret = pthread_create(&cid, NULL, consumer, NULL); // 消费者
82.     if (ret != 0)
83.         err_thread(ret, "pthread_create consuer error");
84.
85.     ret = pthread_create(&cid, NULL, consumer, NULL); // 消费者
86.     if (ret != 0)
87.         err_thread(ret, "pthread_create consuer error");
88.
89.     ret = pthread_create(&cid, NULL, consumer, NULL); // 消费者
90.     if (ret != 0)
91.         err_thread(ret, "pthread_create consuer error");
92.

```

```
93.     pthread_join(pid, NULL);
94.     pthread_join(cid, NULL);
95.
96.     return 0;
97. }
```

编译运行，结果如下：

```
zhcode@ubuntu:~/Code/code167$ make consumer3
gcc consumer3.c -o consumer3 -Wall -g -lpthread
zhcode@ubuntu:~/Code/code167$ ./consumer3
--produce 167
-----consumer id = 140683728320256 :167
--produce 279
-----consumer id = 140683736712960 :279
--produce 629
-----consumer id = 140683745105664 :629
```

这个代码还是有问题的。比如，两个消费者都阻塞在条件变量上，就是说没有数据可以消费。完事儿都把锁还回去了，生产者此时生产了一个数据，会同时唤醒两个因条件变量阻塞的消费者，完事儿两个消费者去抢锁。结果就是 A 消费者拿到锁，开始消费数据，B 消费者阻塞在锁上。之后 A 消费完数据，把锁归还，B 被唤醒，然而此时已经没有数据供 B 消费了。所以这里有个逻辑错误，消费者阻塞在条件变量那里应该使用 while 循环。这样 A 消费完数据后，B 做的第一件事不是去拿锁，而是判定条件变量。

181P-条件变量 signal 注意事项

`pthread_cond_signal()`：唤醒阻塞在条件变量上的（至少）一个线程。

`pthread_cond_broadcast()`： 唤醒阻塞在条件变量上的 所有线程。

182P-信号量概念及其相关操作函数

信号量：

应用于线程、进程间同步。

相当于 初始化值为 N 的互斥量。 N 值，表示可以同时访问共享数据区的线程数。

函数：

`sem_t sem;` 定义类型。

`int sem_init(sem_t *sem, int pshared, unsigned int value);`

参数：

`sem:` 信号量

`pshared:` 0: 用于线程间同步

1: 用于进程间同步

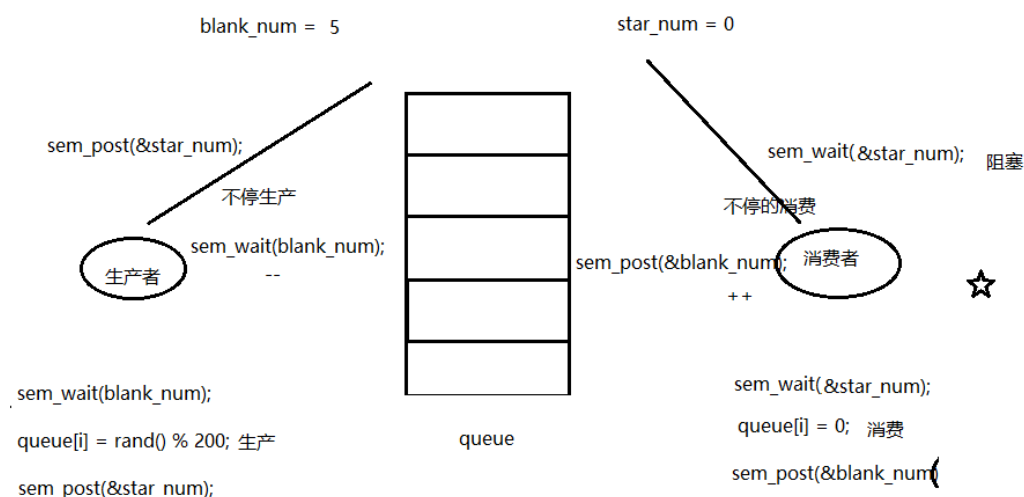
`value:` N 值。（指定同时访问的线程数）

`sem_destroy();`

`sem_wait();` 一次调用，做一次-- 操作，当信号量的值为 0 时，再次 -- 就会阻塞。
(对比 `pthread_mutex_lock`)

`sem_post();` 一次调用，做一次++ 操作。当信号量的值为 N 时，再次 ++ 就会阻塞。
(对比 `pthread_mutex_unlock`)

183P-信号量实现的生产者消费者



代码如下:

```
1.  /*信号量实现 生产者 消费者问题*/
2.
3.  #include <stdlib.h>
4.  #include <unistd.h>
5.  #include <pthread.h>
6.  #include <stdio.h>
7.  #include <semaphore.h>
8.
9.  #define NUM 5
10.
11. int queue[NUM]; //全局数组实现环形队列
12. sem_t blank_number, product_number; //空格子信号量, 产品信号量
13.
14. void *producer(void *arg)
15. {
16.     int i = 0;
17.
18.     while (1) {
19.         sem_wait(&blank_number); //生产者将空格子数--,为0则阻塞等待
20.         queue[i] = rand() % 1000 + 1; //生产一个产品
21.         printf("----Produce---%d\n", queue[i]);
22.         sem_post(&product_number); //将产品数++
23.
24.         i = (i+1) % NUM; //借助下标实现环形
25.         sleep(rand()%1);
26.     }
27. }
```

```

28.
29. void *consumer(void *arg)
30. {
31.     int i = 0;
32.
33.     while (1) {
34.         sem_wait(&product_number);           //消费者将产品数--,为0则阻塞等待
35.         printf("-Consume---%d\n", queue[i]);
36.         queue[i] = 0;                         //消费一个产品
37.         sem_post(&blank_number);             //消费掉以后,将空格子数++
38.
39.         i = (i+1) % NUM;
40.         sleep(rand()%3);
41.     }
42. }
43.
44. int main(int argc, char *argv[])
45. {
46.     pthread_t pid, cid;
47.
48.     sem_init(&blank_number, 0, NUM);          //初始化空格子信号量为5,线程间共享 -- 0
49.     sem_init(&product_number, 0, 0);          //产品数为0
50.
51.     pthread_create(&pid, NULL, producer, NULL);
52.     pthread_create(&cid, NULL, consumer, NULL);
53.
54.     pthread_join(pid, NULL);
55.     pthread_join(cid, NULL);
56.
57.     sem_destroy(&blank_number);
58.     sem_destroy(&product_number);
59.
60.     return 0;
61. }

```

编译运行，结果如下：


```
zhcode@ubuntu:~/Code/code167$ make semaphore
gcc semaphore.c -o semaphore -Wall -g -lpthread
zhcode@ubuntu:~/Code/code167$ ./semaphore
----Produce---384
-Consume---384
----Produce---916
-Consume---916
----Produce---387
----Produce---650
----Produce---363
----Produce---691
----Produce---764
-Consume---387
----Produce---427
-Consume---650
----Produce---212
-Consume---363
```

184P-总结

线程同步：

协同步调，对公共区域数据按序访问。防止数据混乱，产生与时间有关的错误。

锁的使用：

建议锁！对公共数据进行保护。所有线程【应该】在访问公共数据前先拿锁再访问。但，锁本身不具备强制性。

使用 mutex (互斥量、互斥锁) 一般步骤：

pthread_mutex_t 类型。

1. pthread_mutex_t lock; 创建锁
- 2 pthread_mutex_init; 初始化 1
3. pthread_mutex_lock; 加锁 1-- --> 0
4. 访问共享数据 (stdout)
5. pthread_mutex_unlock(); 解锁 0++ --> 1
6. pthread_mutex_destroy; 销毁锁

初始化互斥量：

pthread_mutex_t mutex;

1. pthread_mutex_init(&mutex, NULL); 动态初始化。
2. pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; 静态初始化。

注意事项：

尽量保证锁的粒度， 越小越好。（访问共享数据前，加锁。访问结束【立即】解锁。）

互斥锁，本质是结构体。 我们可以看成整数。 初值为 1。（pthread_mutex_init() 函数调用成功。）

加锁： --操作， 阻塞线程。

解锁： ++操作， 唤醒阻塞在锁上的线程。

try 锁：尝试加锁，成功--。失败，返回。同时设置错误号 EBUSY

restrict 关键字：

用来限定指针变量。被该关键字限定的指针变量所指向的内存操作，必须由本指针完成。

【死锁】：

是使用锁不恰当导致的现象：

1. 对一个锁反复 lock。
2. 两个线程，各自持有一把锁，请求另一把。

读写锁：

锁只有一把。以读方式给数据加锁——读锁。以写方式给数据加锁——写锁。

读共享，写独占。

写锁优先级高。

相较于互斥量而言，当读线程多的时候，提高访问效率

```
pthread_rwlock_t  rwlock;
```

```
pthread_rwlock_init(&rwlock, NULL);
```

```
pthread_rwlock_rdlock(&rwlock);    try
```

```
pthread_rwlock_wrlock(&rwlock);    try
```

```
pthread_rwlock_unlock(&rwlock);
```

```
pthread_rwlock_destroy(&rwlock);
```

条件变量：

本身不是锁！ 但是通常结合锁来使用。 mutex

```
pthread_cond_t  cond;
```

初始化条件变量：

1. pthread_cond_init(&cond, NULL); 动态初始化。
2. pthread_cond_t cond = PTHREAD_COND_INITIALIZER; 静态初始化。

阻塞等待条件：

```
pthread_cond_wait(&cond, &mutex);
```

作用： 1) 阻塞等待条件变量满足

2) 解锁已经加锁成功的信号量 （相当于 pthread_mutex_unlock(&mutex)）

3) 当条件满足，函数返回时，重新加锁信号量 （相当于，pthread_mutex_lock(&mutex);）

pthread_cond_signal(): 唤醒阻塞在条件变量上的（至少）一个线程。

pthread_cond_broadcast(): 唤醒阻塞在条件变量上的所有线程。

【要求，能够借助条件变量，完成生产者消费者】

信号量：

应用于线程、进程间同步。

相当于 初始化值为 N 的互斥量。 N 值，表示可以同时访问共享数据区的线程数。

函数：

sem_t sem; 定义类型。

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

参数：

sem: 信号量

pshared: 0: 用于线程间同步

1: 用于进程间同步

value: N 值。（指定同时访问的线程数）

```
sem_destroy();
```

sem_wait(); 一次调用，做一次-- 操作，当信号量的值为 0 时，再次 -- 就会阻塞。
(对比 pthread_mutex_lock)

sem_post(); 一次调用，做一次++ 操作。当信号量的值为 N 时，再次 ++ 就会阻塞。
(对比 pthread_mutex_unlock)